

UE TAC

TD 1 : Introduction à Android

et Kotlin et Jetpack Compose...

Un / votre téléphone...

- Soit votre téléphone personnel (Android !) + câble USB pour charger/tester vos applis
- Soit un téléphone/tablette de la fac en prêt à chaque séance
 - Venez en chercher un en échange de votre carte étudiant

Votre niveau en Android (et autres technos) ?

- 1 : Pas d'expérience
- 2 : Quelques connaissances
- 3 : Plusieurs projets
- 4 : Confirmé
- Complétez le document maintenant :
<https://forms.gle/6A8QKzXgskwjS3vZ7>

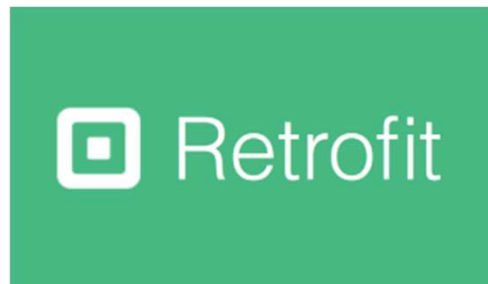
Dans ce cours ...

- Vous découvrirez
 - Bases du développement d'application Android (Kotlin et Compose)
 - Bibliothèques "à la mode"
 - Quelques bonnes pratiques
- Ce que l'on attend ...
 - Curiosité sur les concepts
 - Travail binôme ET personnel
 - Ponctualité et respect

Nous verrons...



JetPack

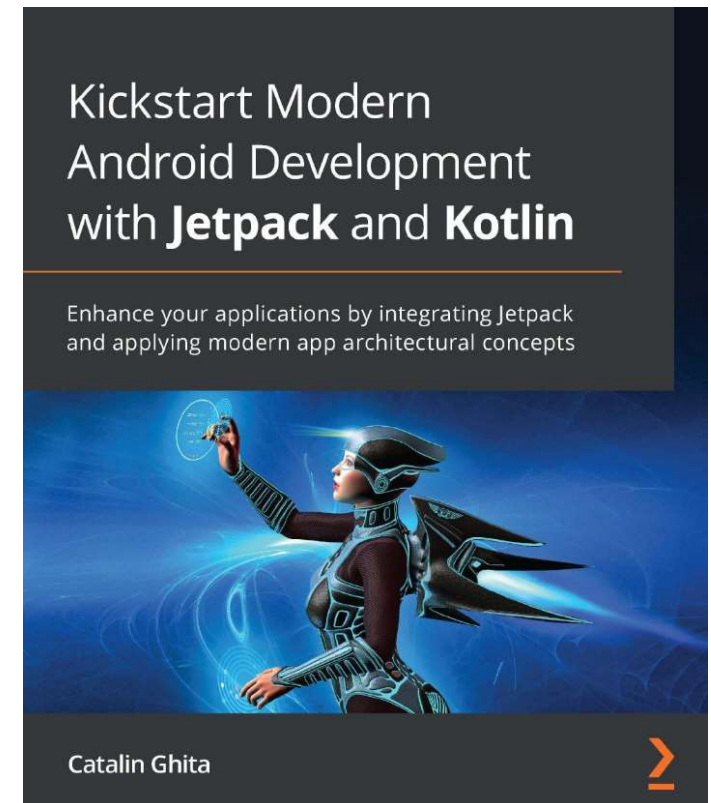


Documentation officielle : RTFM !

- Design : <https://developer.android.com/design/>
- Tous les concepts : <https://developer.android.com/guide>
- Exemple :
 - App Basics : Application Fundamentals
<https://developer.android.com/guide/components/fundamentals>
- Une question sur le fonctionnement d'un élément d'Android ?
-> **Recherche sur developer.android.com D'ABORD !**

Livre

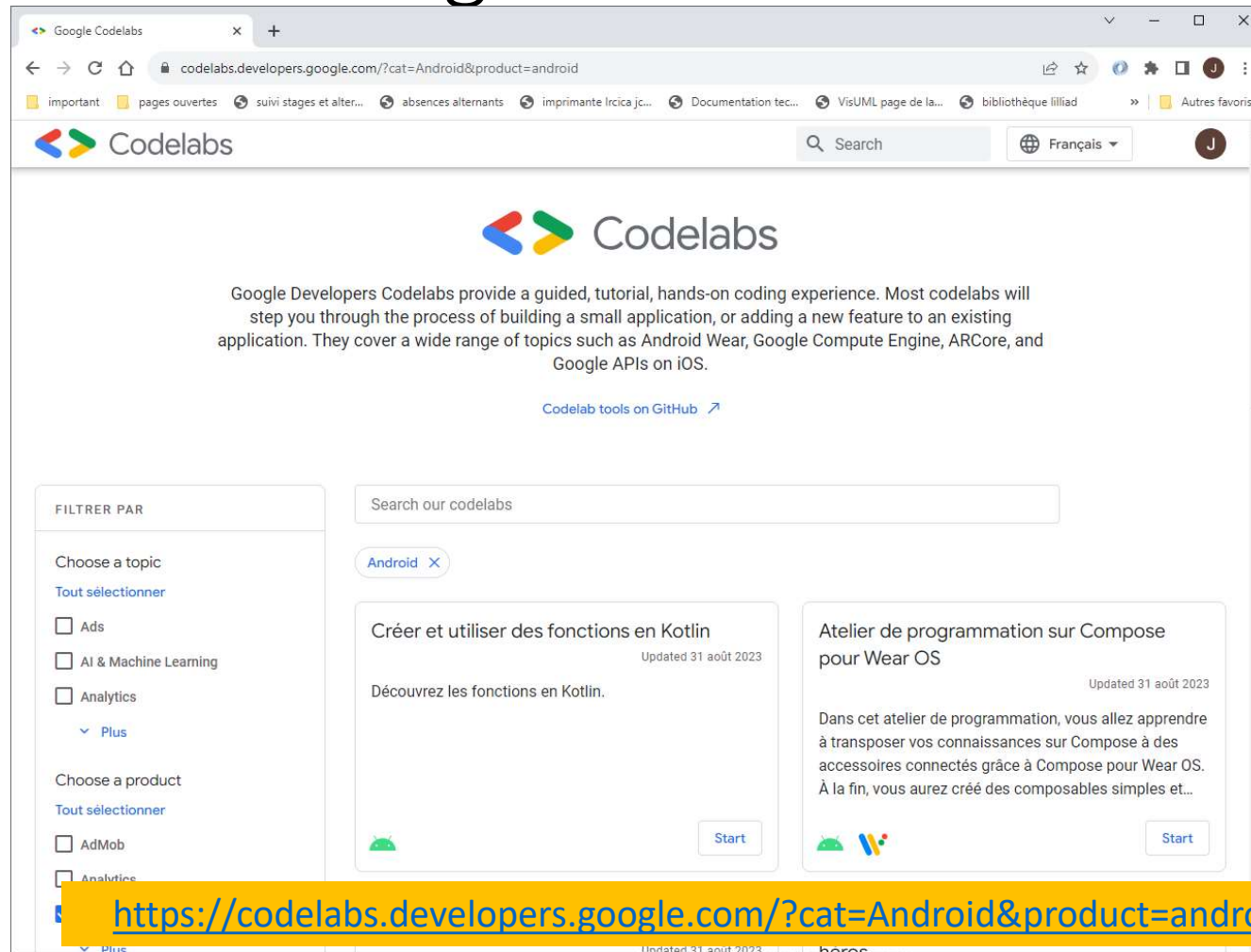
- <https://univ-scholarvox-com.ressources-electroniques.univ-lille.fr/reader/docid/88930974/page/1>
 - disponible à Liliad en version électronique
- Code source :
<https://github.com/PacktPublishing/Kickstart-Modern-Android-Development-with-Jetpack-and-Kotlin>



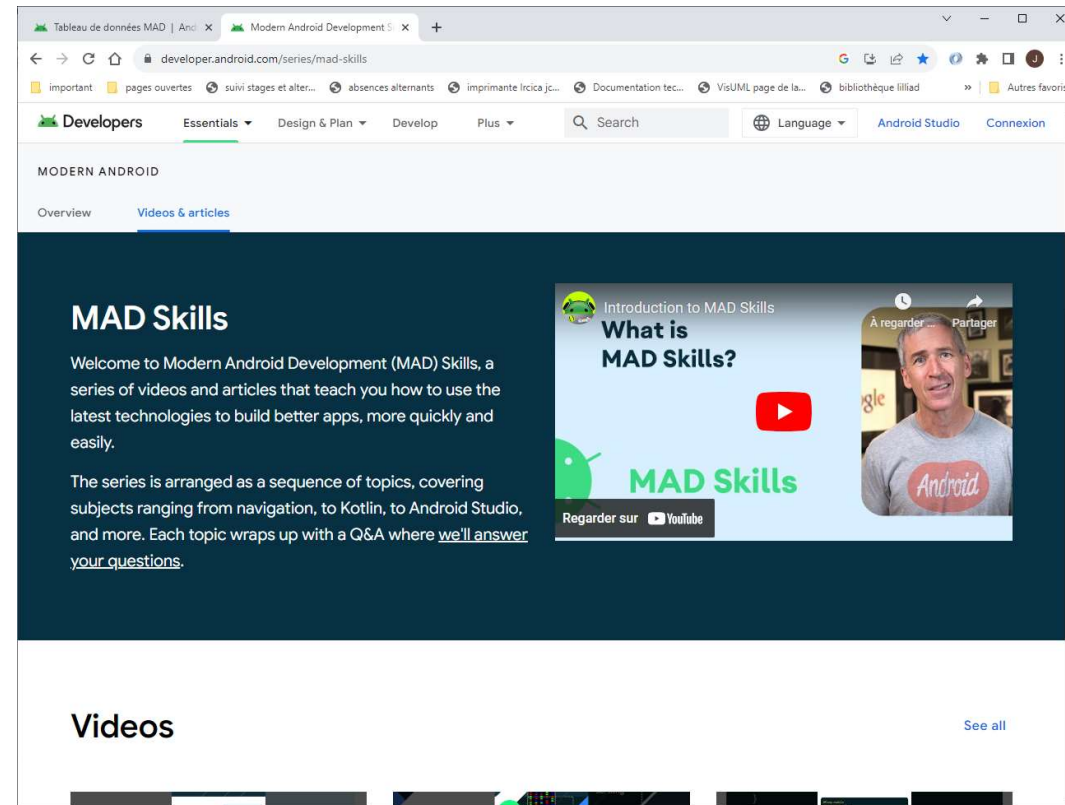
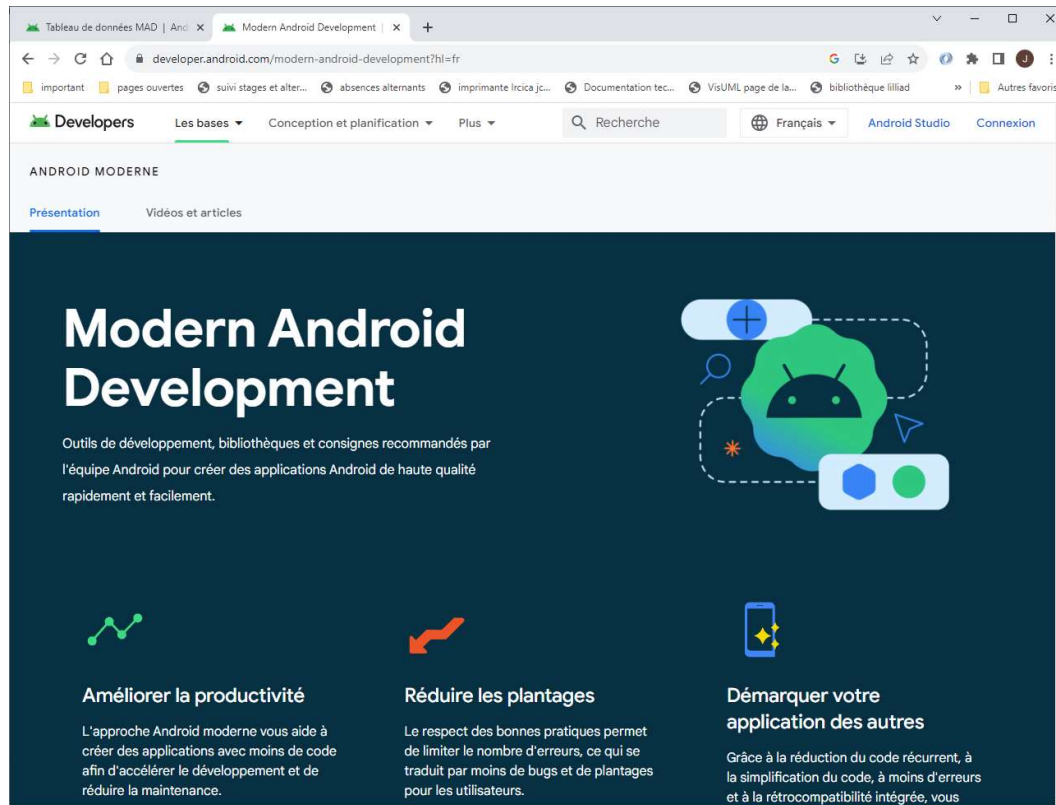
Livre

- Approche très pédagogique
- Très bonne lecture en complément de TAC

Des « cours » Google : codelabs



Des « cours » Google : MAD Skills



Des « cours » Google : MAD Skills



The screenshot shows a web browser window displaying the Google MAD Skills page. The browser's address bar shows the URL `developer.android.com/modern-android-development/scorecard?hl=fr`. The page features a dark blue background with a colorful illustration of various Android development tools and concepts. The main heading is "Quel est votre score MAD ?". Below this, there is a paragraph explaining that MAD (Modern Android Development) is a model for creating better applications, and another paragraph mentioning a plugin for Android Studio to track MAD scores. A video player is embedded on the right side of the page, showing a video titled "What's your MAD score?" with the hashtag #MADscore.

Tableau de données MAD | And... X +

developer.android.com/modern-android-development/scorecard?hl=fr

important pages ouvertes suivi stages et alter... absences alternants imprimante lrica jc... Documentation tec... VisUML page de la... bibliothèque lilliad Autres favoris

Developers Plate-forme Android Studio Kotlin Plus Recherche Français Android Studio Connexion

Quel est votre score MAD ?

De Kotlin à Jetpack, en passant par Android Studio et Android App Bundle, Modern Android Development (MAD) est le modèle dont vous avez besoin pour créer de meilleures applications.

Avec le tableau de données MAD, nous avons créé un plug-in pour Android Studio afin que vous puissiez voir (et montrer) à quel point vous êtes un développeur Android moderne.

What's your MAD score?
What's your MAD score?
#MADscore

<https://developer.android.com/modern-android-development/scorecard>

AVANT TOUT ! Découvrons Kotlin...

- Qu'est-ce que Kotlin ?
- Qui a une idée ?

Kotlin (si vous êtes débutant total)

- Kotlin est un **langage de programmation moderne** et polyvalent conçu pour être simple, sûr et expressif. Il a été développé par JetBrains et est destiné à fonctionner sur la machine virtuelle Java (JVM). Kotlin est utilisé pour créer des applications Android, des applications serveur, des applications web et bien plus encore.
- Kotlin est un **langage orienté objet**, ce qui signifie que vous pouvez créer des objets avec des propriétés et des méthodes pour représenter des entités du monde réel. Il **prend également en charge la programmation fonctionnelle**, ce qui signifie que vous pouvez **manipuler des fonctions comme des objets** et les utiliser de manière expressive.
- Les principales caractéristiques de Kotlin sont les suivantes :
 - **Concision** : Kotlin offre une **syntaxe concise** qui réduit la quantité de code nécessaire pour accomplir une tâche. Cela le rend plus lisible et plus agréable à écrire.
 - **Sécurité** : Kotlin met l'accent sur la **sécurité du code** en évitant des erreurs courantes telles que les pointeurs nuls (NullPointerException). Il introduit le concept de types nullable et non-nullable pour aider les développeurs à éviter les erreurs liées aux valeurs null.
 - **Interopérabilité** : Kotlin est conçu pour être totalement **compatible avec Java**, ce qui signifie que vous **pouvez utiliser du code Kotlin dans un projet Java existant et vice versa**. Vous pouvez également **utiliser des bibliothèques Java dans vos projets Kotlin** sans aucun problème.
 - **Smart Casts** : Kotlin possède des **fonctionnalités d'inférence de type intelligentes** qui permettent au compilateur de comprendre les types de données à partir desquels vous effectuez des opérations. Cela simplifie le code en évitant les conversions de types redondantes.
 - **Extension Functions** : Kotlin permet **d'ajouter de nouvelles fonctions à des classes existantes sans les modifier**, grâce à la fonctionnalité d'extension. Cela permet d'améliorer les classes standard ou des bibliothèques tierces sans modifier leur code source.

Kotlin (si vous connaissez Java)

- Kotlin s'inspire fortement de Java, ce qui signifie que si vous connaissez Java, vous serez à l'aise avec Kotlin. Cependant, Kotlin introduit plusieurs fonctionnalités et améliorations qui rendent le code plus concis, plus sûr et plus expressif.
- Voici quelques différences clés entre Kotlin et Java :
 - **Concision** : Kotlin a une **syntaxe plus concise** que Java, ce qui signifie que vous pouvez accomplir la même tâche en écrivant moins de code. Par exemple, vous n'avez pas besoin de déclarer les types de variables lorsque le compilateur peut les inférer.
 - **Nullable Safety** : Kotlin introduit le concept de **types nullable et non-nullable** pour gérer les valeurs null de manière plus sûre. Vous pouvez spécifier qu'une variable peut être null (en utilisant `String?` au lieu de `String`) et Kotlin vous oblige à traiter explicitement les valeurs null.
 - **Extension Functions** : Kotlin permet **d'ajouter des fonctions à des classes existantes** à l'aide des extension functions. Cela facilite l'ajout de fonctionnalités à des classes standard de la bibliothèque Java sans étendre directement leur code.
 - **Smart Casts** : Kotlin possède des **fonctionnalités d'inférence de type intelligentes**, ce qui signifie que vous n'avez pas besoin de faire des conversions de types redondantes. Le compilateur peut déterminer le type d'une variable après des vérifications nulles, ce qui rend le code plus expressif.
 - **Gestion des Ressources** : Kotlin a une fonctionnalité appelée `use` qui simplifie la gestion des ressources, comme la fermeture automatique des flux.
- En résumé, Kotlin est une évolution de Java avec des fonctionnalités modernes et une syntaxe plus concise, tout en restant **complètement interopérable avec le code Java existant**. Il vise à améliorer la productivité du développeur et à réduire les erreurs courantes en fournissant une expérience de programmation plus agréable et plus sûre. Si vous connaissez déjà Java, vous trouverez Kotlin facile à apprendre et vous bénéficierez des avantages supplémentaires qu'il offre

Merci ChatGPT 😊

AVANT TOUT ! Découvrons Kotlin...

- Val = variable affectée une seule fois (semblable à CONST)
- Var = variable
- Classe
 - Final par défaut (pas héritable)
 - **MaClasse(var nom:String, var prenom:String) → constructeur créé automatiquement** et donc :
 - `var personne = MaClasse("Dupont", "Marc")`
 - `println("Je m'appelle ${personne.prenom} ${personne.nom} ")`
 - Héritage : ajouter open
 - `open class MaClasse`
 - `Class Etudiant():MaClasse() {`

Question

- Dans le code suivant, y a-t-il des erreurs, et lesquelles ?

Ca fonctionne ou non ?

```
val liste1: List<Int> = listOf(1, 2, 3)
var liste2: List<Int> = listOf(1, 2, 3)
val liste3: MutableList<Int> = mutableListOf(1, 2, 3)
var liste4: MutableList<Int> = mutableListOf(1, 2, 3)
```

```
liste1.add(5)
liste1=listOf(10)
```

```
liste2.add(5)
liste2=listOf(5)
```

```
liste3.add(5)
liste3= mutableListOf(10)
```

```
liste4.add(5)
liste4=mutableListOf(10)
```

Réponses

```
val liste1: List<Int> = listOf(1, 2, 3)
var liste2: List<Int> = listOf(1, 2, 3)
val liste3: MutableList<Int> = mutableListOf(1, 2, 3)
var liste4: MutableList<Int> = mutableListOf(1, 2, 3)
```

```
liste1.add(5) // impossible de modifier la liste (pas mutable)
liste1=listOf(10) // impossible de changer la liste (val)
```

```
liste2.add(5) // impossible de modifier la liste (pas mutable)
liste2=listOf(5) // changement de liste OK
```

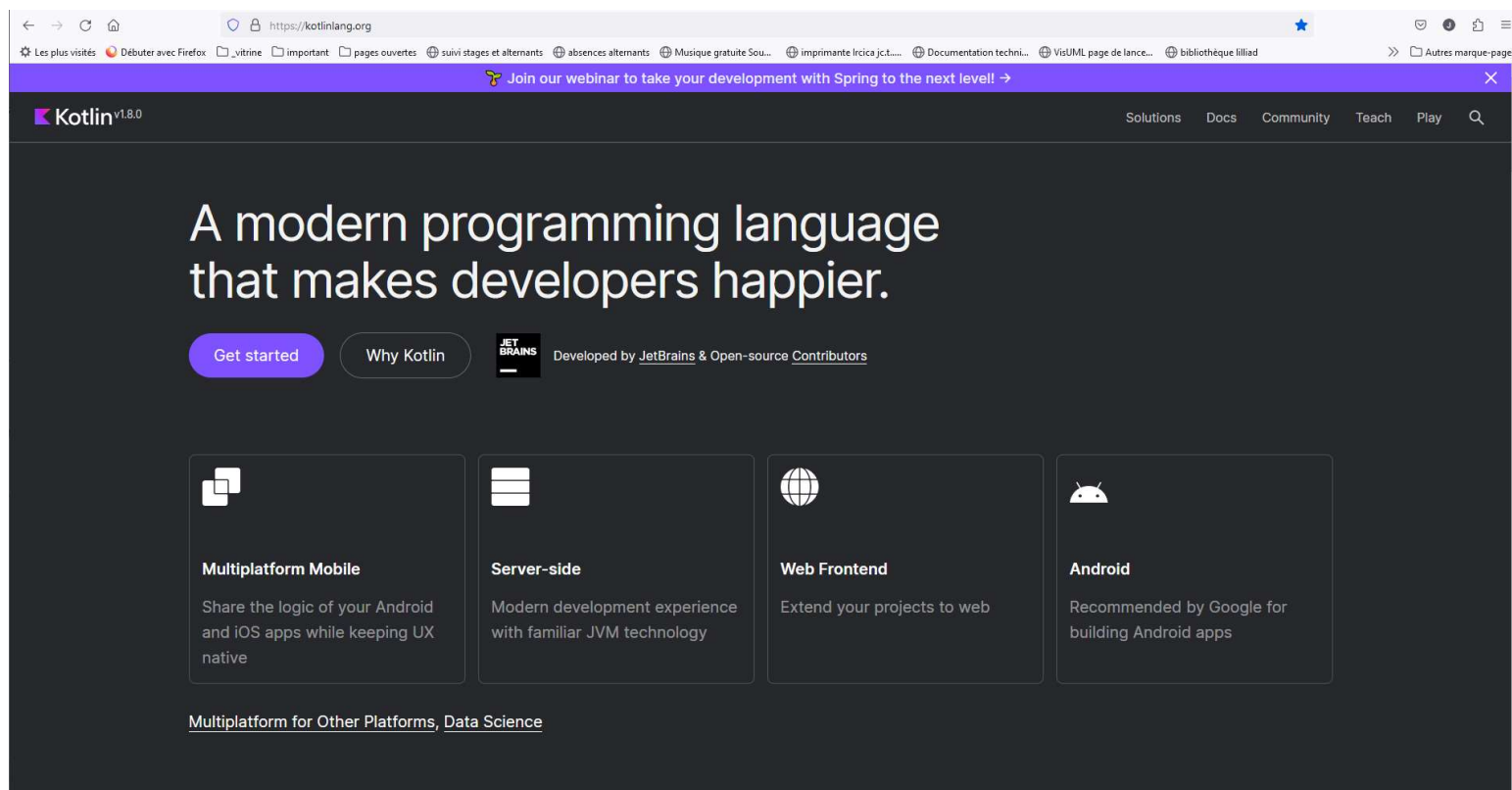
```
liste3.add(5) // modification de liste OK (mutable)
liste3= mutableListOf(10) // impossible de changer la liste (val)
```

```
liste4.add(5) // ok
liste4=mutableListOf(10) // ok
```

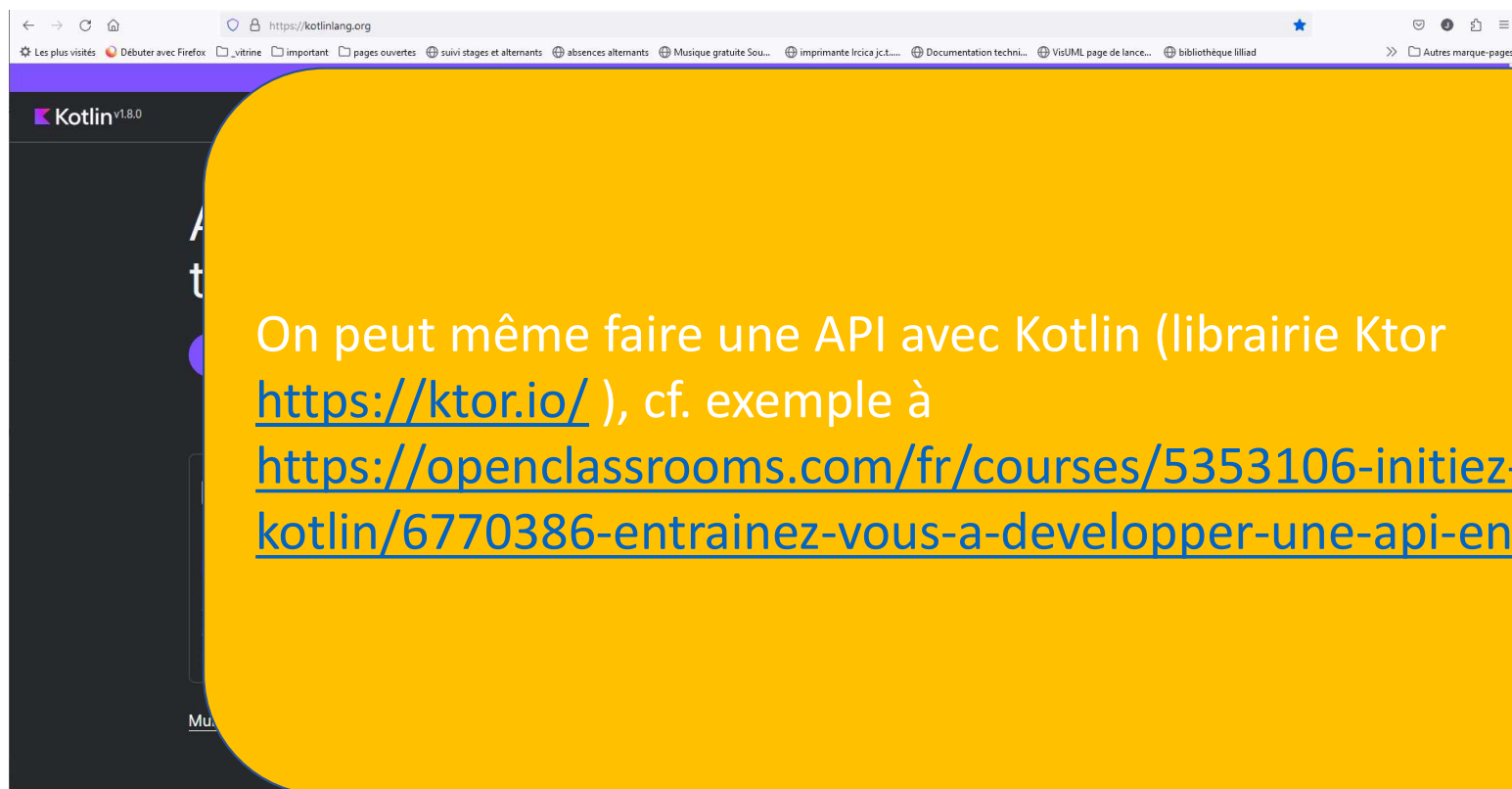
Toute petite découverte de Kotlin

- Pour rappel, on va découvrir Kotlin au cours des TDs
 - mais je vous encourage à aller voir par vous-mêmes les cours officiels (ou autres) pour vous améliorer et approfondir certaines notions.

Apprendre Kotlin (<https://kotlinlang.org/>)

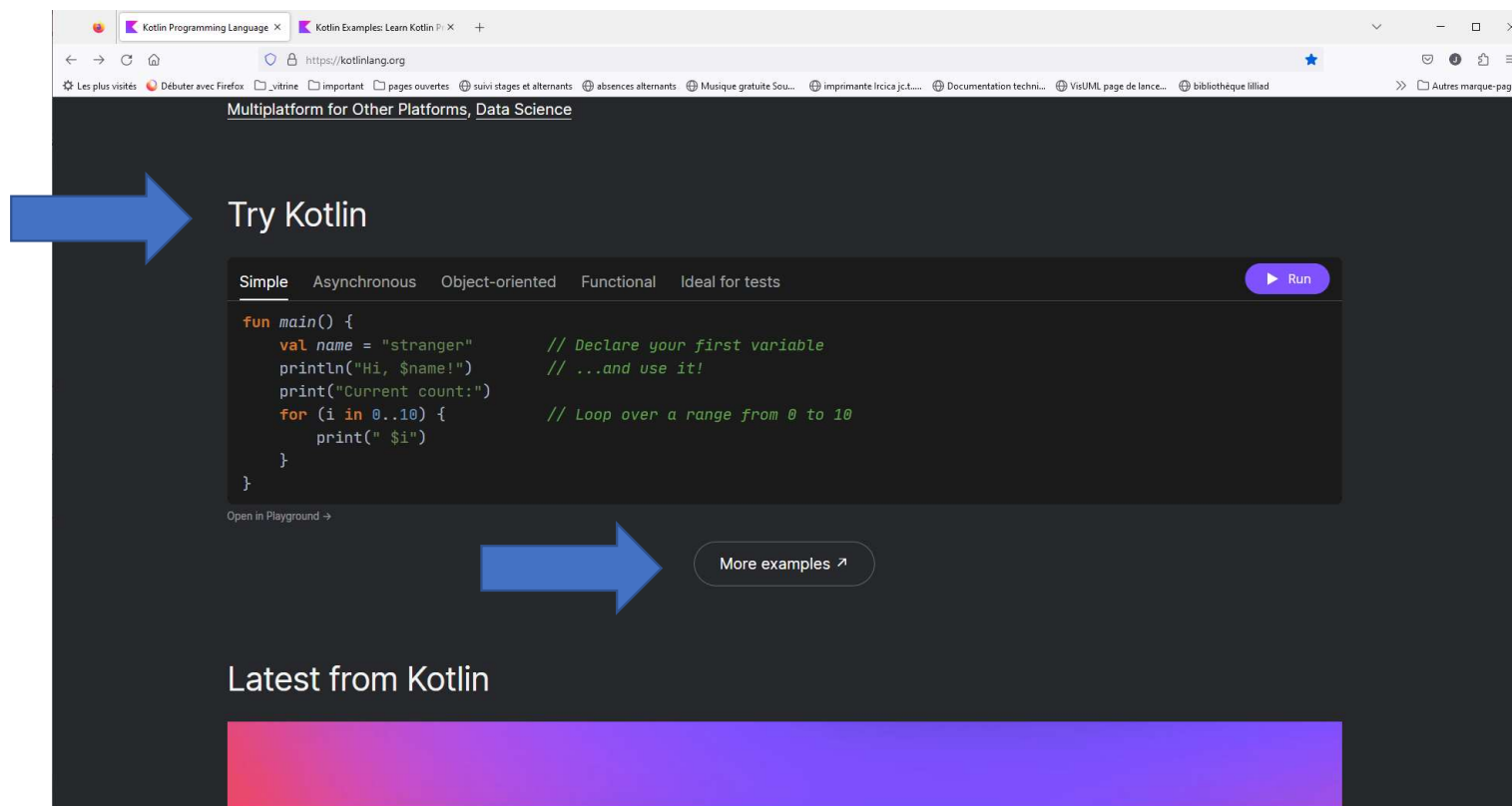


Apprendre Kotlin (<https://kotlinlang.org/>)



On peut même faire une API avec Kotlin (librairie Ktor <https://ktor.io/>), cf. exemple à <https://openclassrooms.com/fr/courses/5353106-initiez-vous-a-kotlin/6770386-entraenez-vous-a-developper-une-api-en-kotlin>

Tester Kotlin (<https://kotlinlang.org/>)



Exemple simple

```
class MainActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
  
        main()  
    }  
  
    fun main() {  
        Log.d("jc", "onCreate")  
        val name = "stranger"    // Declare your first variable  
        Log.d("jc", "Hi, $name!") // ...and use it!  
        print("Current count:")  
        for (i in 0..10) {        // Loop over a range from 0 to 10  
            Log.d("jc", "$i")  
        }  
    }  
}
```

2023-2024

JC Tarby - UE TAC - M1 E-Services

À tester dans android studio
avec empty activity (sans
utiliser compose)

- Soit print/println : sortie dans « Run », mais pensez à ajouter un println() à la fin (vidage de buffer)
- Soit un Log.d(TAG, msg) : sortie dans LogCat

2

Functionnal

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        main()
    }
}
```

```
fun main() {
    // Who sent the most messages?
    val frequentSender = messages
        .groupBy(Message::sender)
        .maxByOrNull { (_, messages) -> messages.size }
        ?.key // Get their names
    println(frequentSender) // [Ma]

    // Who are the senders?
    val senders = messages
        .asSequence() // Make operations lazy (for a long call chain)
        .filter { it.body.isNotBlank() && !it.isRead } // Use lambdas...
        .map(Message::sender) // ...or member references
        .distinct()
        .sorted()
        .toList() // Convert sequence back to a list to get a result
    println(senders) // [Adam, Ma]
}
```

Très puissant en seulement
quelques lignes...
(détails ci-après)

1

```
.....
data class Message( // Create a data class
    val sender: String,
    val body: String,
    val isRead: Boolean = false, // Provide a default value for the argument
)

val messages = listOf( // Create a list
    Message("Ma", "Hey! Where are you?"),
    Message("Adam", "Everything going according to plan today?"),
    Message("Ma", "Please reply. I've lost you!"),
)
```

Functionnal

```
fun main() {
```

```
    val frequentSender = messages  
        .groupBy(Message::sender)  
        .maxByOrNull { (_, messages) -> messages.size }  
        ?.key  
    println(frequentSender)
```

`groupBy(Message::sender)` regroupe les messages en fonction de l'expéditeur (`sender`) → crée une Map où chaque clé est un expéditeur (`sender`) et chaque valeur est une liste de messages envoyés par cet expéditeur.

`maxByOrNull { (_, messages) -> messages.size }`:

- `maxByOrNull` trouve l'entrée dans la Map qui a la plus grande taille de liste de messages. Cette fonction prend une **lambda function** avec un argument représentant chaque entrée de la Map,
- l'underscore ``_`` est utilisé comme nom de variable pour ignorer la clé (l'expéditeur) car pas utilisé ici.
- `messages.size` accède à la taille de la liste de messages de chaque entrée (c'est-à-dire le nombre de messages envoyés par cet expéditeur).

`?.key` pour obtenir la clé de l'entrée ayant la plus grande taille de liste de messages. Le résultat de `maxByOrNull` peut être `null` s'il n'y a pas de messages dans la liste `messages`, donc nous utilisons l'opérateur de navigation sécurisée (`?.`) pour accéder à la clé seulement si le résultat n'est pas `null`. Cette clé est le nom de l'expéditeur qui a envoyé le plus grand nombre de messages.

Le résultat final est stocké dans la variable `frequentSender`, qui contiendra le nom de l'expéditeur qui a envoyé le plus grand nombre de messages, ou `null` si la liste `messages` est vide.

Functionnal

```
fun main() {  
  
    val frequentSender = messages  
        .groupBy(Message::sender)  
        .maxByOrNull { (_, messages) -> messages.size }  
        ?.key  
    println(frequentSender)
```

Code équivalent

```
val senderMap = mutableMapOf<String, List<Message>>()  
  
// Regrouper les messages par expéditeur dans une Map  
for (message in messages) {  
    val sender = message.sender  
    if (senderMap.containsKey(sender)) {  
        senderMap[sender]?.add(message)  
    } else {  
        senderMap[sender] = mutableListOf(message)  
    }  
}  
  
var maxMessages = 0  
var frequentSender: String? = null  
  
// Trouver l'expéditeur avec le plus grand nombre de messages  
for ((sender, messages) in senderMap) {  
    if (messages.size > maxMessages) {  
        maxMessages = messages.size  
        frequentSender = sender  
    }  
}
```

Lequel est le plus lisible pour vous ?

Functionnal

```
val senders = messages
    .asSequence()
    .filter { it.body.isNotBlank() && !it.isRead }
    .map(Message::sender)
    .distinct()
    .sorted()
    .toList()
println(senders)
}
```

`.asSequence()` convertit la liste `messages` en une séquence .

Une séquence est un flux d'éléments, similaire à une liste, mais les opérations sur une séquence sont traitées de manière paresseuse, ce qui signifie que les éléments ne sont évalués que lorsque nécessaire. Cela peut être utile pour les longues chaînes d'opérations, car cela peut économiser des ressources en évitant de traiter tous les éléments en une seule fois.

`.filter { it.body.isNotBlank() && !it.isRead }` filtre la séquence et ne garde que les messages qui ont un `body` non vide (`it.body.isNotBlank()`) et qui ne sont pas encore lus (`!it.isRead`). Cela permet de ne garder que les messages ayant un contenu non vide et non lus.

`.map(Message::sender)` transforme chaque message en utilisant une référence de membre (`Message::sender`). Cela crée une séquence de noms d'expéditeurs.

`.distinct()` élimine les doublons des noms d'expéditeurs dans la séquence. Cela garantit que chaque nom d'expéditeur n'apparaît qu'une seule fois dans la liste finale.

`.sorted()` trie les noms d'expéditeurs en ordre alphabétique.

`.toList()` convertit la séquence résultante en une liste.

Le résultat final est stocké dans la variable `senders`, qui contiendra la liste triée et sans doublon des noms d'expéditeurs dont les messages ont un contenu non vide et ne sont pas encore lus.

Functionnal

Code équivalent

```
val senders = mutableListOf<String>()

for (message in messages) {
    if (message.body.isNotBlank() && !message.isRead) {
        val sender = message.sender
        if (!senders.contains(sender)) {
            senders.add(sender)
        }
    }
}

senders.sort()
```

```
val senders = messages
    .asSequence()
    .filter { it.body.isNotBlank() && !it.isRead }
    .map(Message::sender)
    .distinct()
    .sorted()
    .toList()
println(senders)
}
```

Lequel est le plus lisible pour vous ?

Et pour finir, les lambda functions

- Format d'une lambda function

```
val nomDeLaFonction: (paramètres) -> typeDeRetour = { arguments -> corpsDeLaFonction }
```

- Exemple de déclaration

```
val addition: (Int, Int) -> Int = { a, b -> a + b }
```

On a vu (sans le
savoir) un exemple
avec .filter
précédemment

Fonction
classique
équivalente

```
fun addition(a: Int, b: Int): Int {  
    return a + b  
}
```

- Appel

```
val resultat = addition(5, 3)  
println(resultat) // Affiche : 8
```

Et pour finir, les lambda functions

```
val somme: (Int, Int) -> Int = { a, b -> a + b }
```

```
val somme = { a: Int, b: Int -> a + b }
```

← Déclaration + concise

Passage de fonction en paramètre

← Très puissant, et beaucoup utilisé dans Compose !

```
fun appliquerOperation(a: Int, b: Int, operation: (Int, Int) -> Int): Int {  
    return operation(a, b)  
}  
  
fun main() {  
    val resultatMultiplication = appliquerOperation(5, 3) { x, y -> x * y }  
    println(resultatMultiplication) // Affiche : 15  
  
    val resultatAddition = appliquerOperation(5, 3) { x, y -> x + y }  
    println(resultatAddition) // Affiche : 8  
}
```

Et pour finir, les lambda functions

```
val somme: (Int, Int) -> Int = { a, b -> a + b }
```

```
val somme = { a: Int, b: Int -> a + b }
```

← Déclaration + concise

Passage de fonction en paramètre

← Très puissant, et beaucoup utilisé dans Compose !

```
fun appliquerOperation(a: Int, b: Int, operation: (Int, Int) -> Int): Int {  
    return operation(a, b)  
}  
  
fun main() {  
    val resultatMultiplication = appliquerOperation(5, 3) { x, y -> x * y }  
    println(resultatMultiplication) // Affiche : 15  
  
    val resultatAddition = appliquerOperation(5, 3) { x, y -> x + y }  
    println(resultatAddition) // Affiche : 8  
}
```

Et pour finir, les lambda functions

```
data class Personne(val nom: String, val age: Int)

val maLambda: (Personne) -> Unit = { (nom, age) -> println("$nom a $age ans") }
```



Déstructuration automatique faite par Kotlin. Les paramètres sont automatiquement associés au type en entrée.

```
fun main() {
    val personnel = Personne("Alice", 30)
    val personne2 = Personne("Bob", 25)

    maLambda(personnel) // Affiche : Alice a 30 ans
    maLambda(personne2) // Affiche : Bob a 25 ans
}
```

Et pour finir, les lambda functions

```
data class Personne(val nom: String, val age: Int)

val maLambda: (Personne) -> Unit = { (nom, age) -> println("$nom a $age ans") }
```

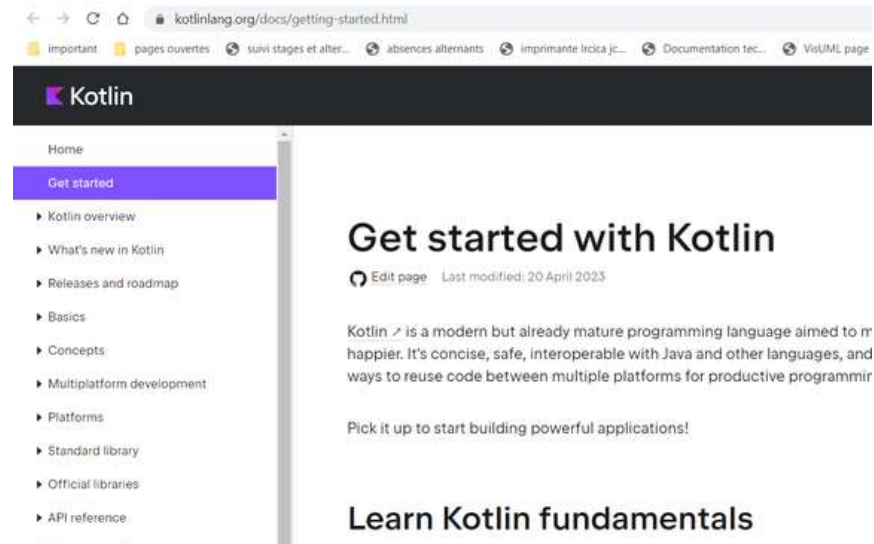
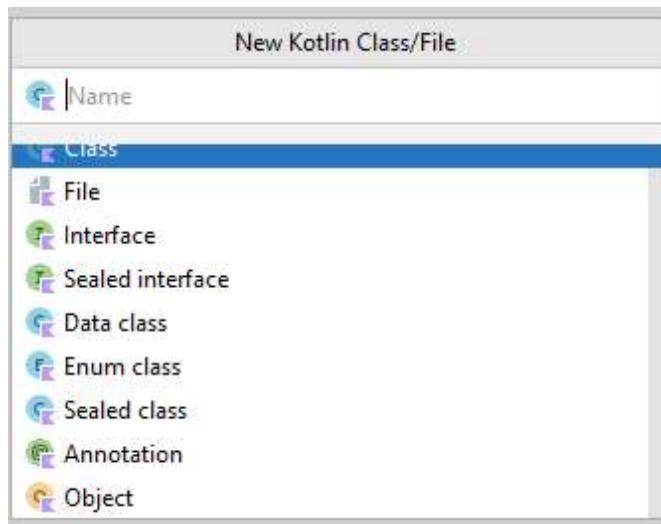


```
val maLambda: (Personne) -> Unit = { personne -> println("${personne.nom} a ${personne.age} ans") }
```

Écriture équivalente (+ verbeuse)

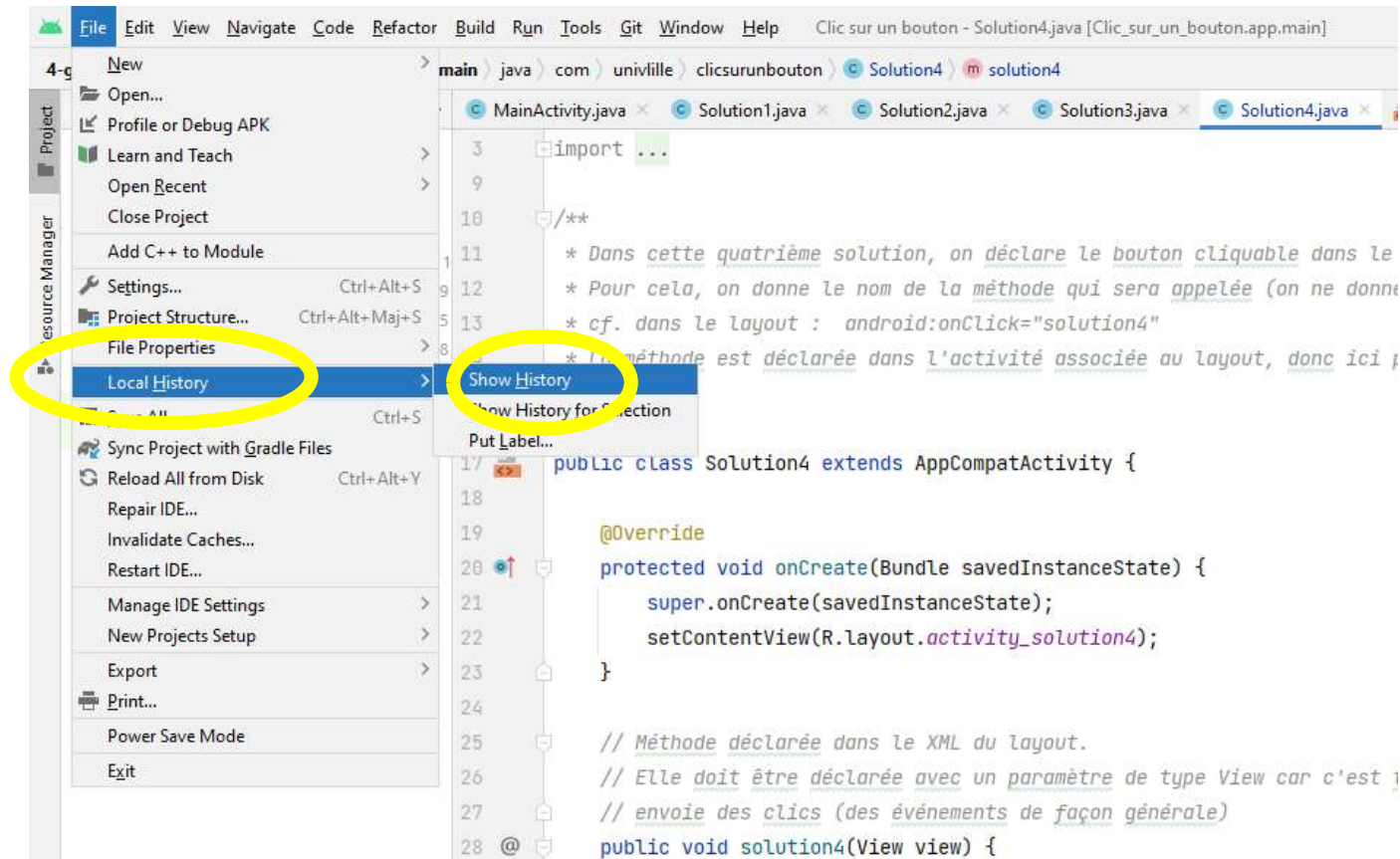
Pour continuer à vous former sur Kotlin

- Voir aussi le Get started
 - <https://kotlinlang.org/docs/getting-started.html> par exemple pour comprendre les différents type de déclaration comme ci-dessous (Object n'est pas que pour déclarer un objet...)



Un petit détour par VCS et GIT avant de poursuivre...

Android Studio et VCS



Android Studio et VCS

C:\Users\Jean-Claude\AndroidStudioProjects\2021-2022\4-gestions-de-clic-sur-un-bouton\app\src\main\java\com\univille\clicsurunbouton\Solution4.java

20/09/2022 12:28 - Solution4.java

* La methode est declaree dans l'activite associee au layout, donc ici pour cet exemple

```
public class Solution4 extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_solution4);
    }

    // Methode declaree dans le XML du layout.
    // Elle doit être déclarée avec un paramètre de type View car c'est toujours une
    // envoie des clics (des événements de façon générale)
    public void solution4(View view) {
        // on récupère l'id de l'objet cliqué et on exécute l'action associée à cet ob

        switch (view.getId()) {
            // si c'est le bouton 1
            case R.id.btn1_solution4:
                Toast.makeText(view.getContext(), "Bouton 1 cliqué", Toast.LENGTH_SH
                break;
            // si c'est le bouton 2
            case R.id.btn2_solution4:
                Toast.makeText(view.getContext(), "Bouton 2 cliqué", Toast.LENGTH_SH
                break;
            // si c'est le bouton 3
            case R.id.btn_back_solution4:
                Toast.makeText(view.getContext(), "Bouton back cliqué", Toast.LENGTH
                finish();
                break;
            default:
                throw new IllegalStateException("Unexpected value: " + view.getId())
        }
    }
}
```

Current

* La methode est declaree dans l'activite associee au layout, donc ici pour cet exemp

```
public class Solution4 extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_solution4);
    }

    // Methode declaree dans le XML du layout.
    // Elle doit être déclarée avec un paramètre de type View car c'est toujours une V
    // envoie des clics (des événements de façon générale)
    public void solution4(View view) {
        // on récupère l'id de l'objet cliqué et on exécute l'action associée à cet ob

        // plus tard, avec Gradle 8, il faudra changer la déclaration des ID ci-dessou
        switch (view.getId()) {
            // si c'est le bouton 1
            case R.id.btn1_solution4:
                Toast.makeText(view.getContext(), "Bouton 1 cliqué", Toast.LENGTH_SHOR
                break;
            // si c'est le bouton 2
            case R.id.btn2_solution4:
                Toast.makeText(view.getContext(), "Bouton 2 cliqué", Toast.LENGTH_SHOR
                break;
            default:
                throw new IllegalStateException("Unexpected value: " + view.getId());
        }
    }
}
```

2 differences

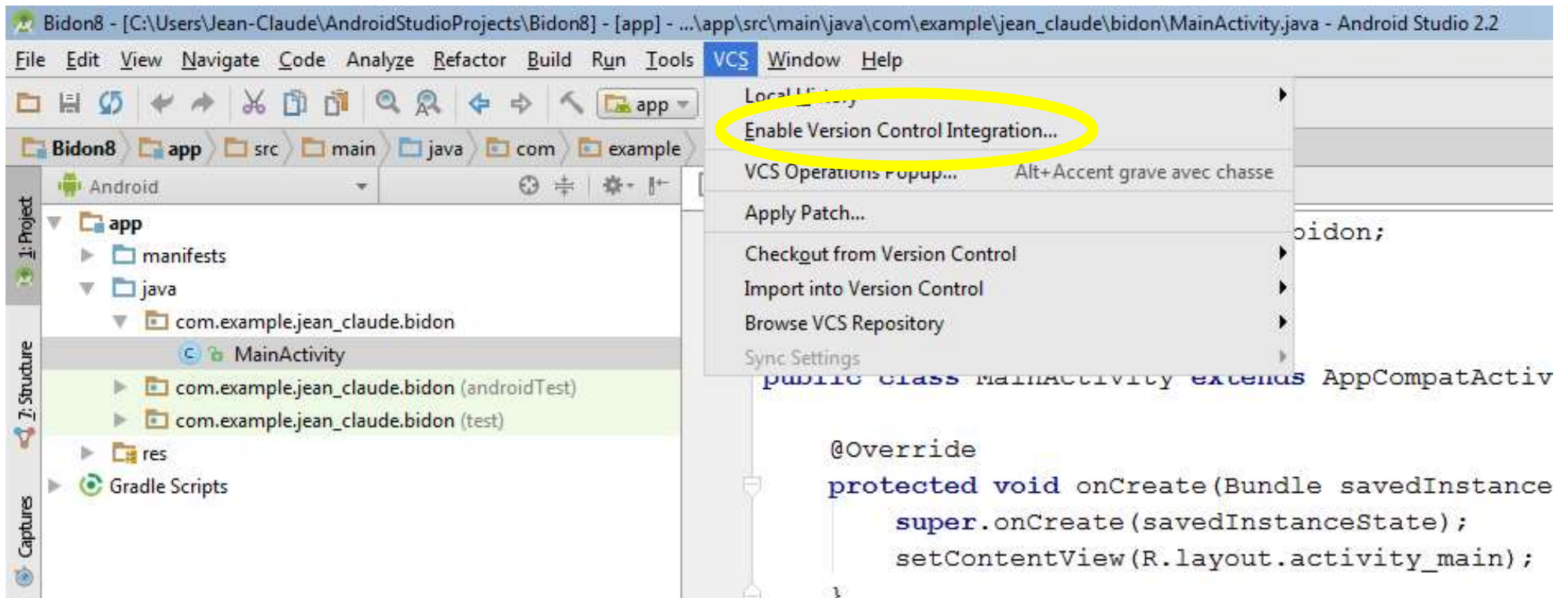
Android Studio et GIT

Pas un cours de GIT, juste son usage dans Android Studio (sans la console)...

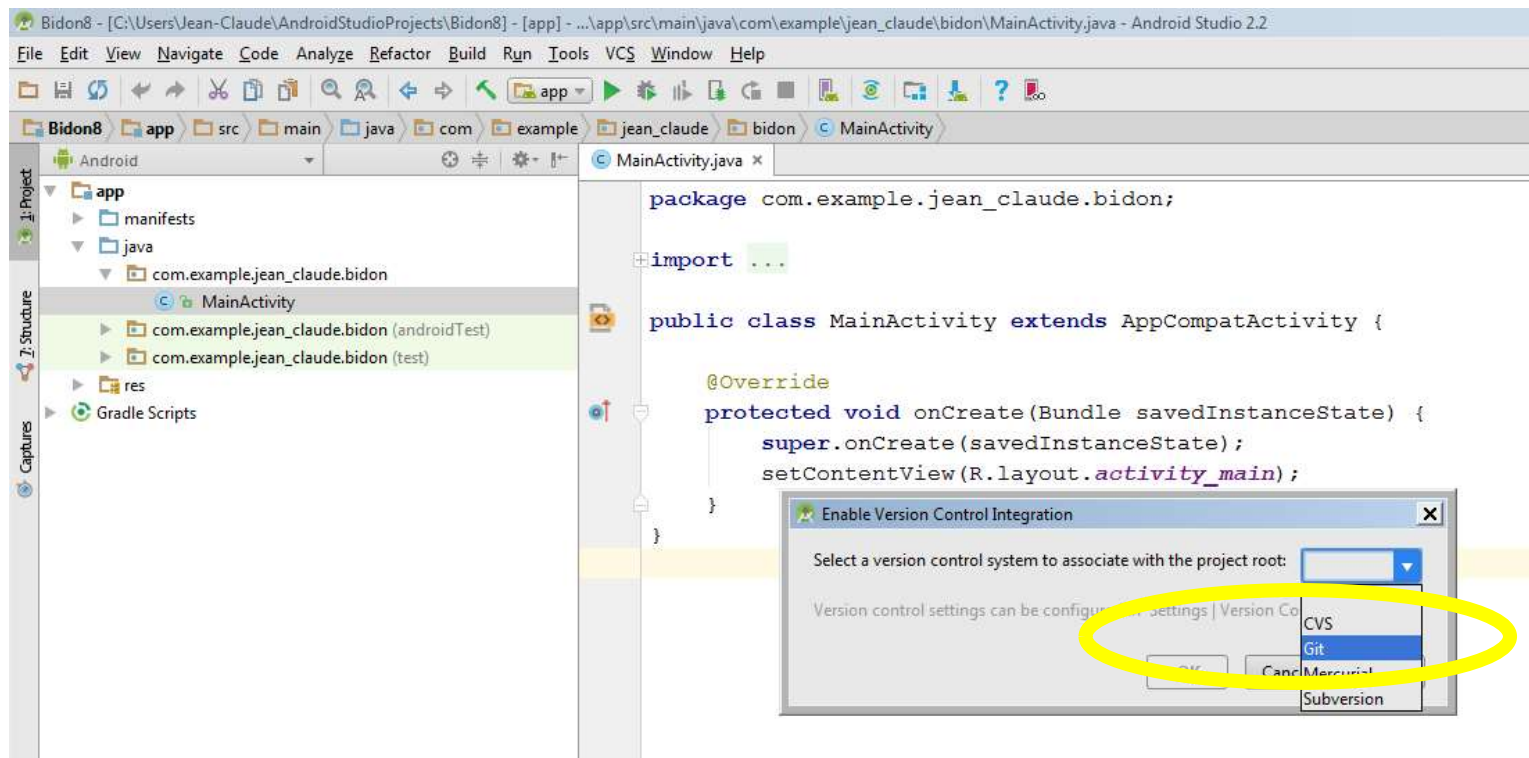
- Démo...
 - Création d'un dépôt GIT (Gitlab)
 - Création d'une appli Android
 - Connexion au git
 - Modification du code, commit, push
 - Branches, checkout...
- Récupération d'un projet Git

Quelques écrans pour mémoire

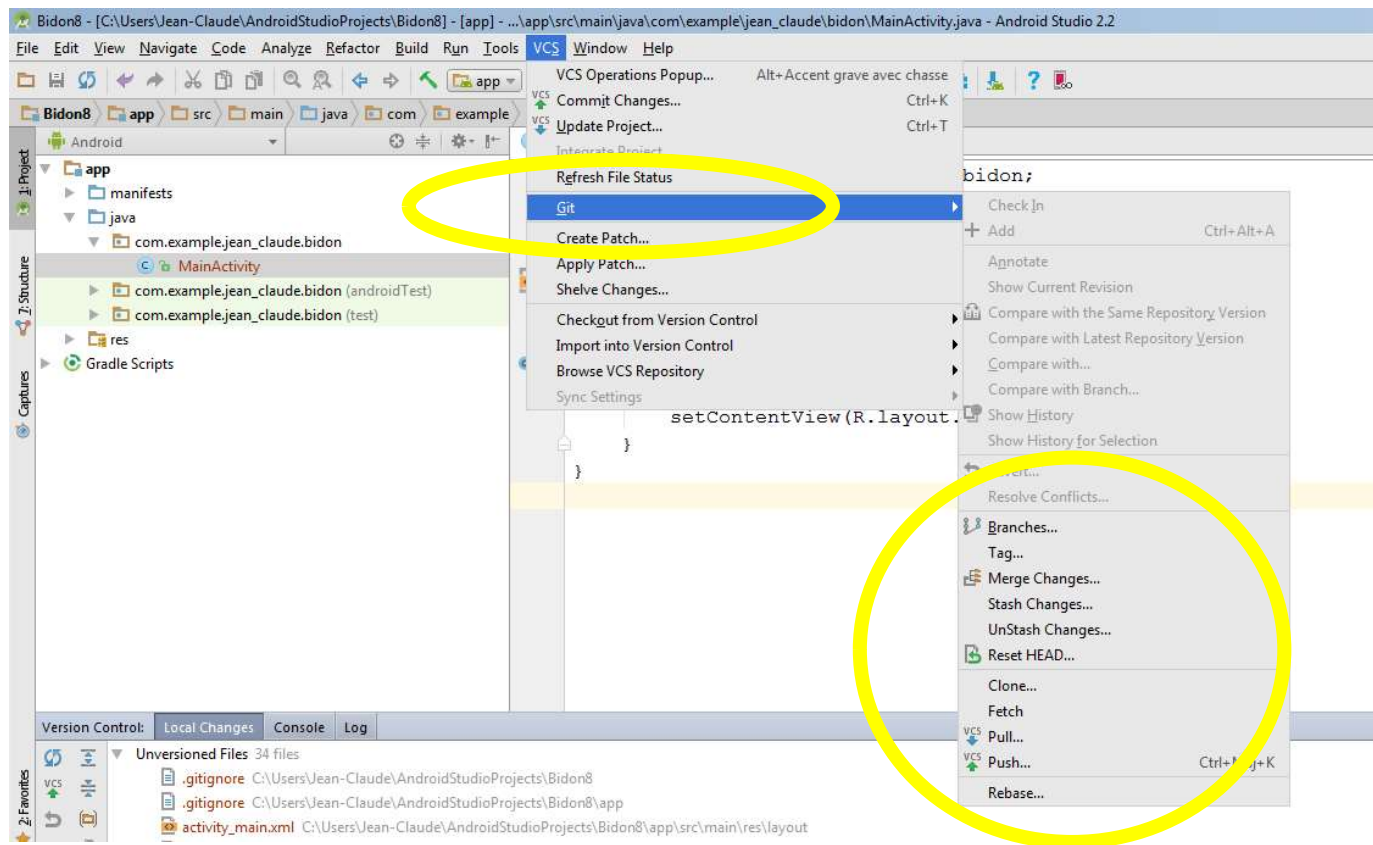
Android Studio : Git...



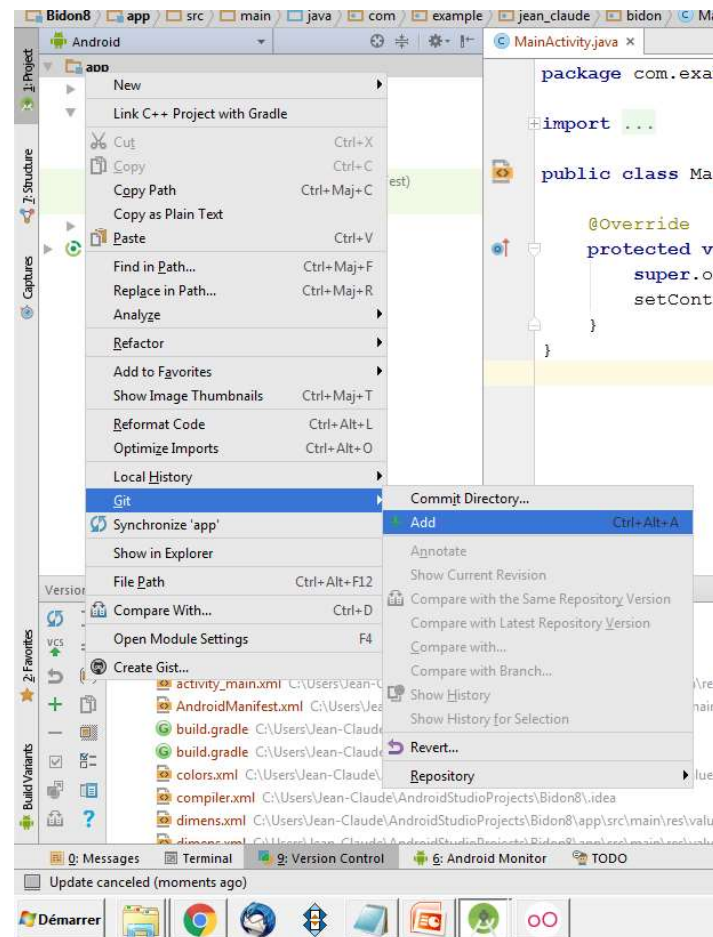
Android Studio : Git...



Android Studio : Git...

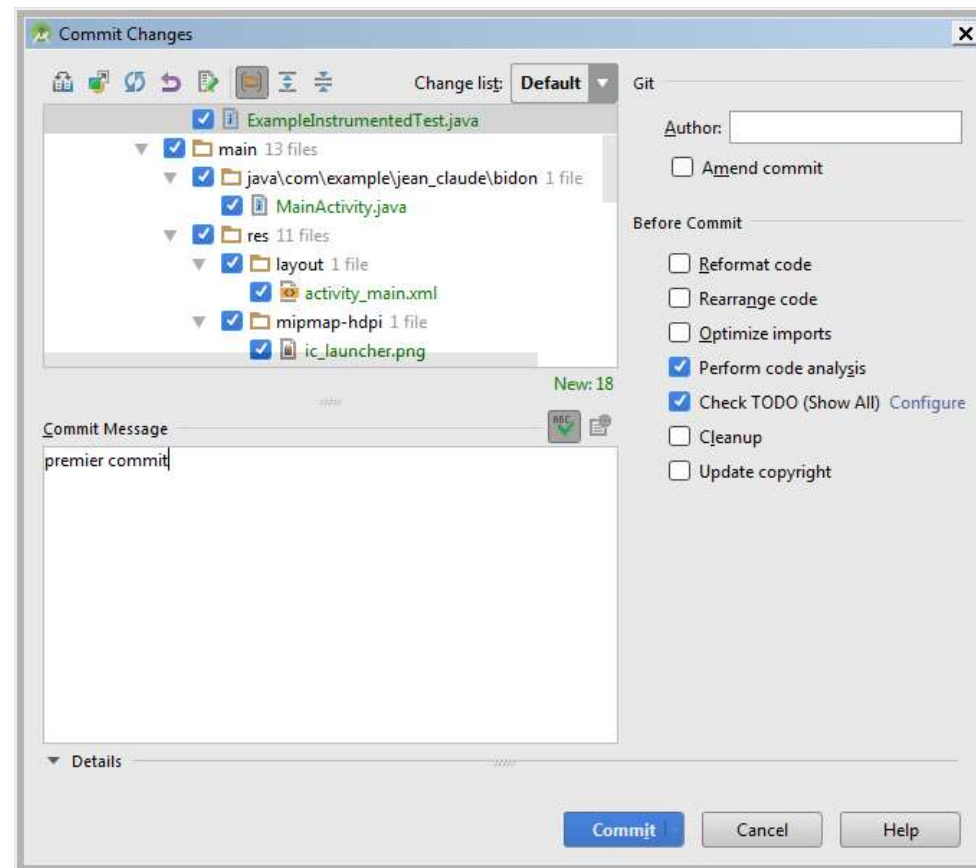


Android Studio : Git...

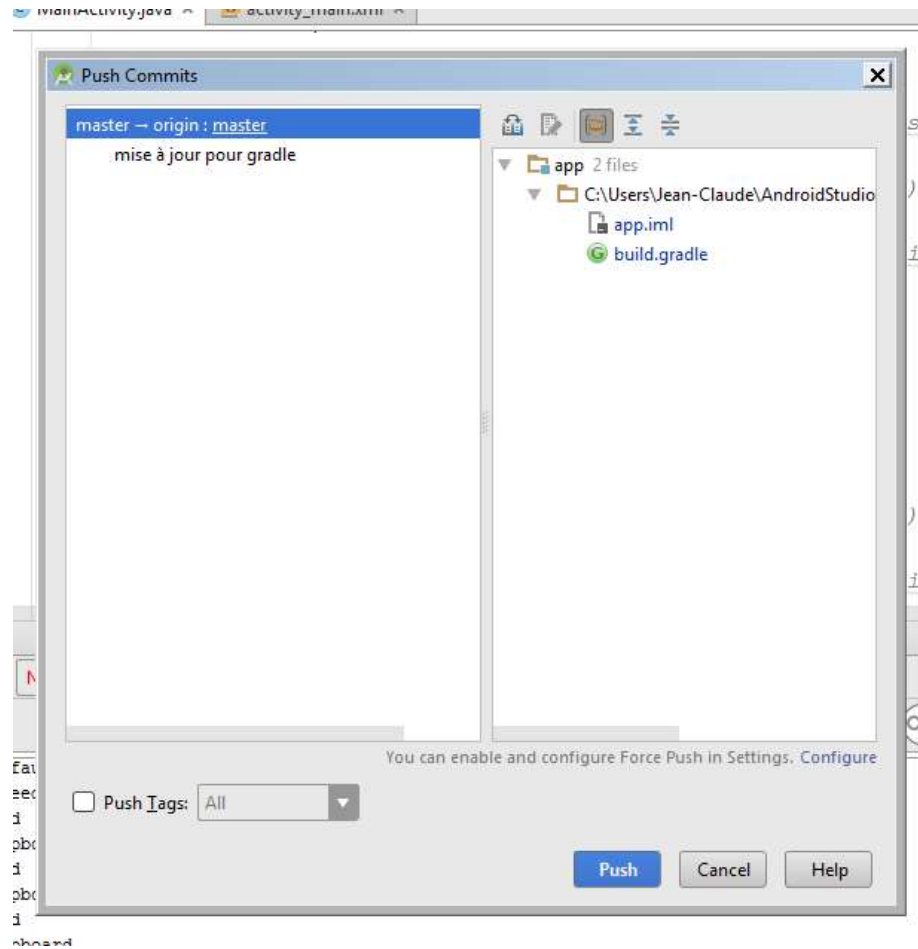


Android Studio : Git...

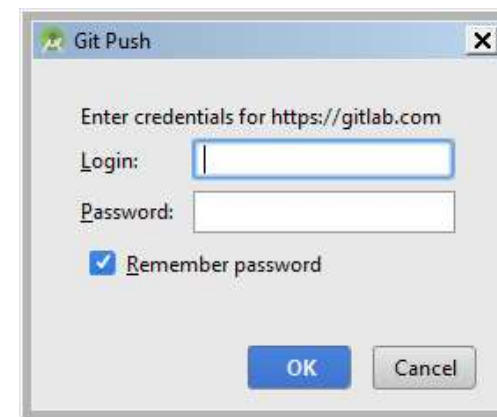
Commit...
Ctrl K



Android Studio : Git...

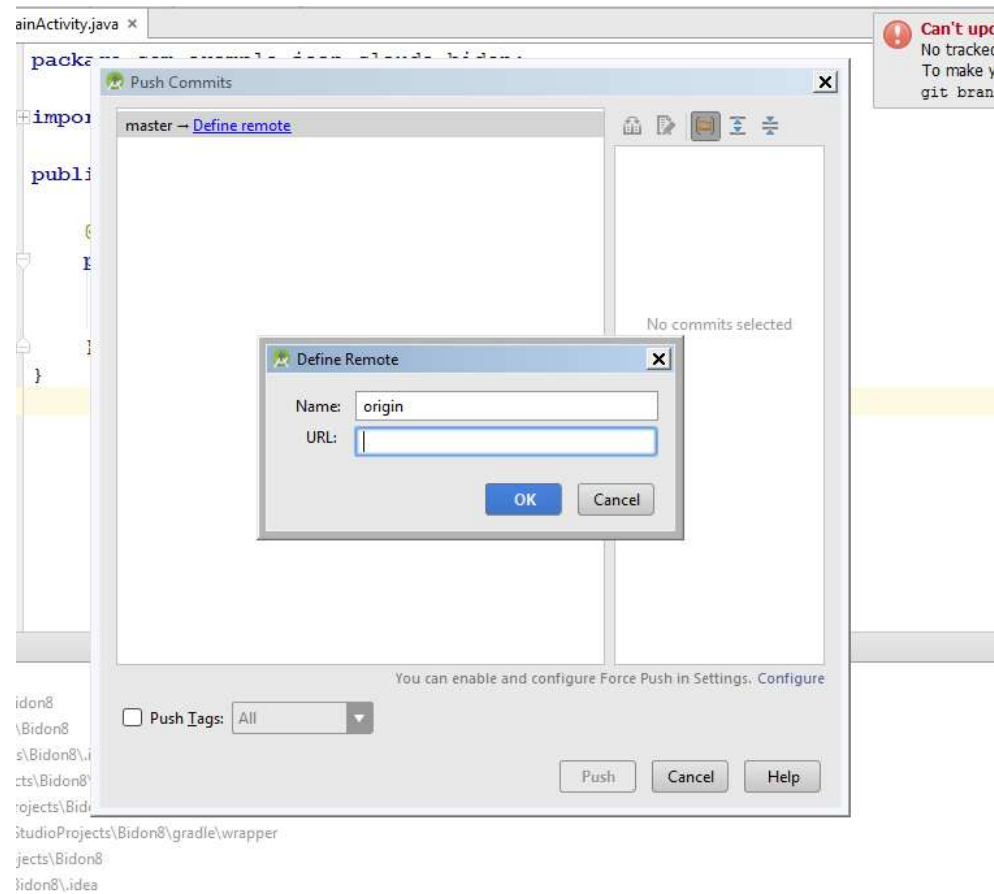


Push...
Ctrl MAJ K



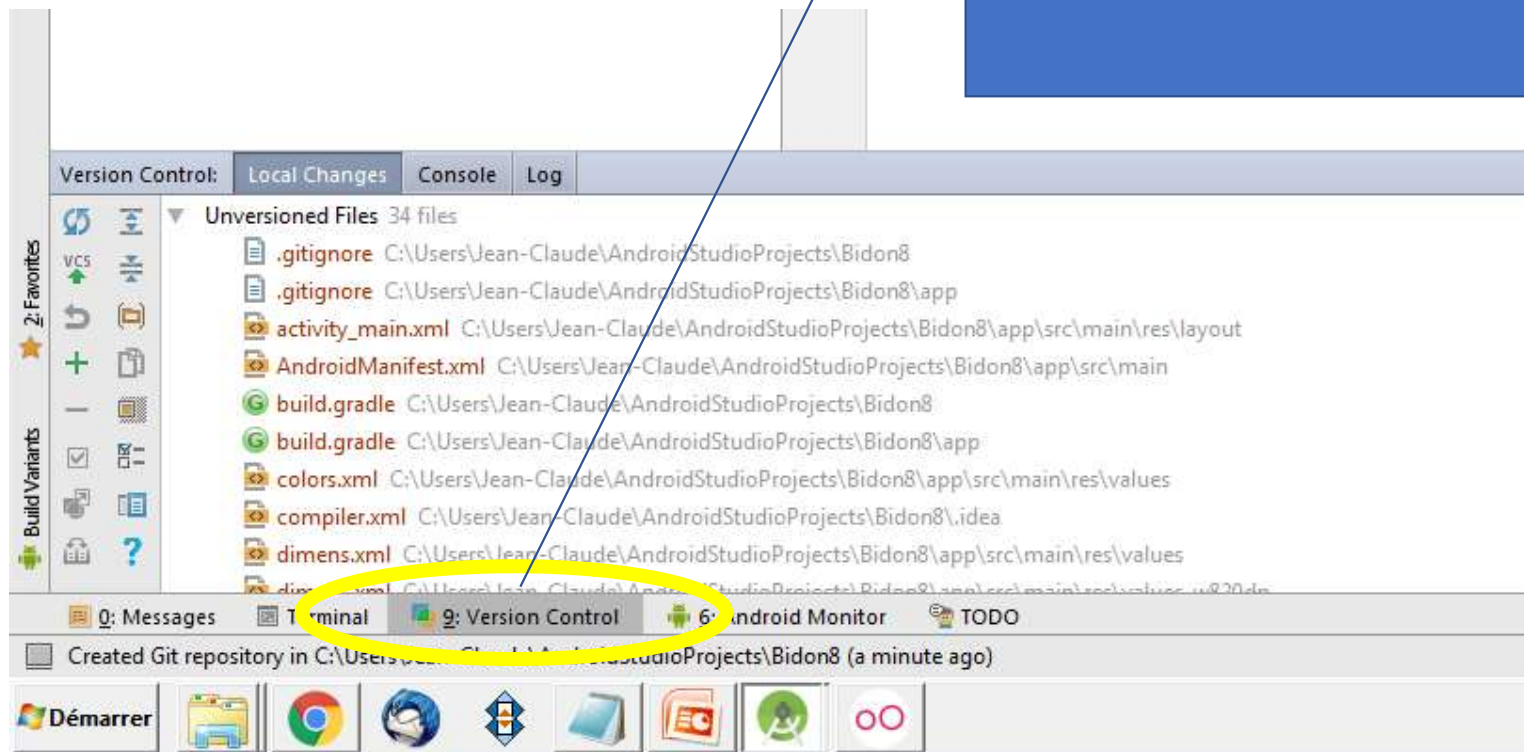
Android Studio : Git...

Push...
Ctrl MAJ K

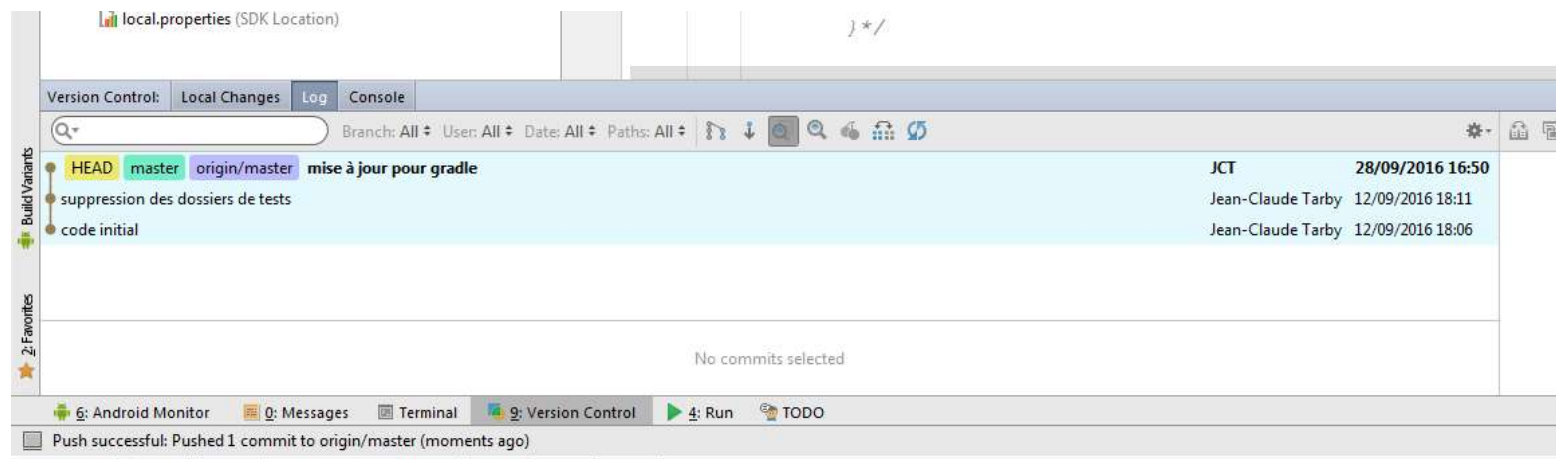


Android Studio : Git...

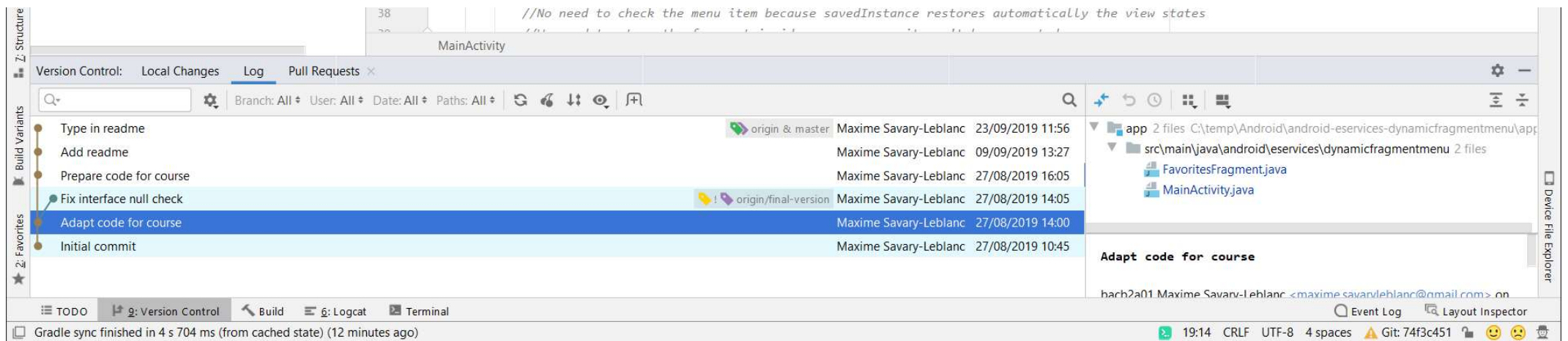
« GIT » dans Android Studio 4



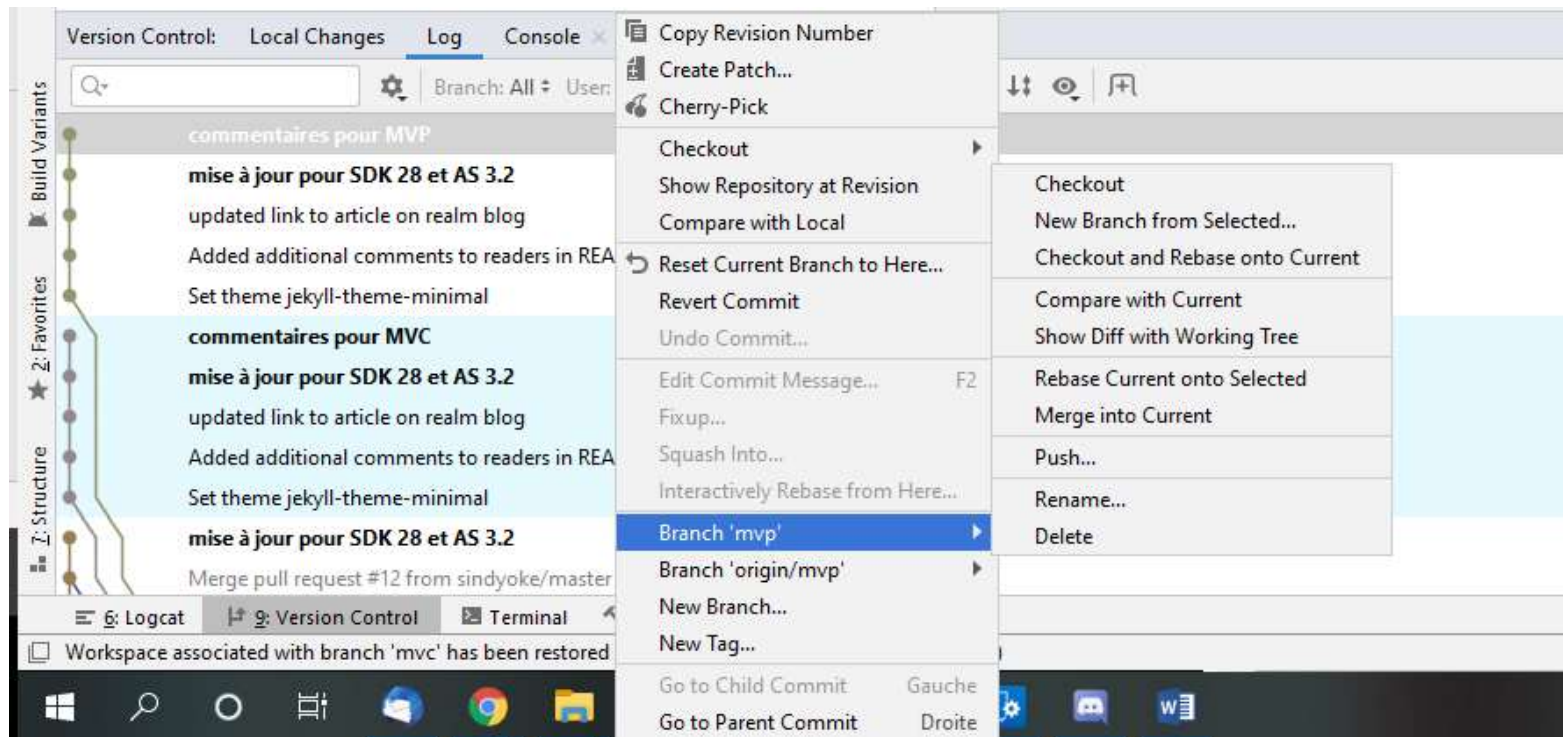
Android Studio : Git...



Android Studio : Git...



Android Studio : Git...



TD 1 : Introduction à Android

et Kotlin et Jetpack Compose...

Avant que je n'oublie...

- Pour travailler avec Compose, dorénavant il faut être en JDK 17 !
 - Pensez à vérifier cela dans vos projets/IDE.

C'est parti !

- L'IDE, le seul, l'unique : **Android Studio**



<https://developer.android.com/studio>

Proxy ?

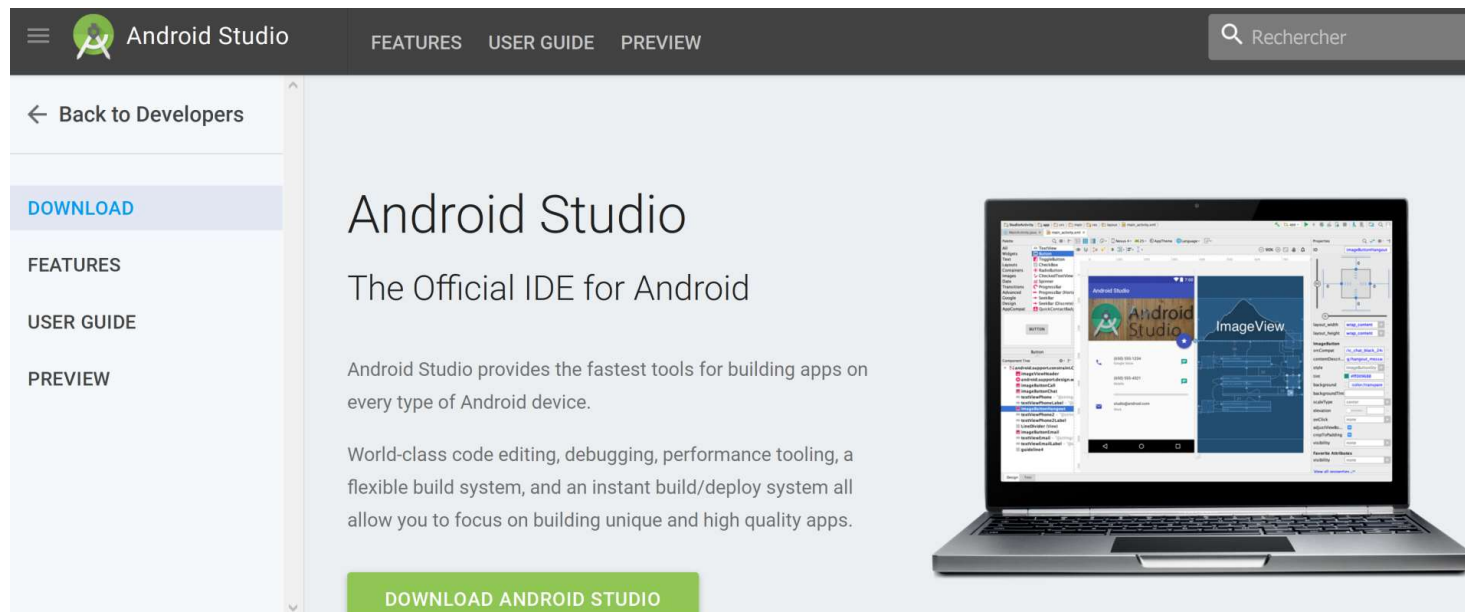
- Pour ceux qui souhaiteraient utiliser leur machine perso en Android sur le réseau de la fac, bien penser à ajouter les paramètres de proxy suivants :
- Dans "Gradle Scripts" (en mode de navigation Android) dans le fichier gradle.properties :
 - `systemProp.http.proxyHost=cache.univ-lille.fr`
 - `systemProp.http.proxyPort=3128`
 - `systemProp.https.proxyHost=cache.univ-lille.fr`
 - `systemProp.https.proxyPort=3128`
- Détails à <https://developer.android.com/studio/intro/studio-config.html>
- Faites attention à le retirer si vous ne passez plus par le proxy (si les packages sont téléchargés, ça compilera peut-être mais si vous modifiez le gradle, ça ne marchera pas)

Logiciels nécessaires

- AndroidStudio
 - IDE de développement basé sur IntelliJ
 - Dernière version (ou celle de la salle de cours)
- Android SDK
 - Standard Development Kit

Downloads

- <https://developer.android.com/studio/index.html>



Les logiciels

- AndroidStudio
 - ➔ /opt/android-studio/bin
 - ./studio.sh
- SDK
 - ➔ /local/Android

Démarrer Android Studio

- Chercher sur votre ordinateur → android
 - Démarrer Android Studio
- Ou :
 - Ouvrir une console
 - `cd /opt/android-studio/bin`
 - `./studio.sh`

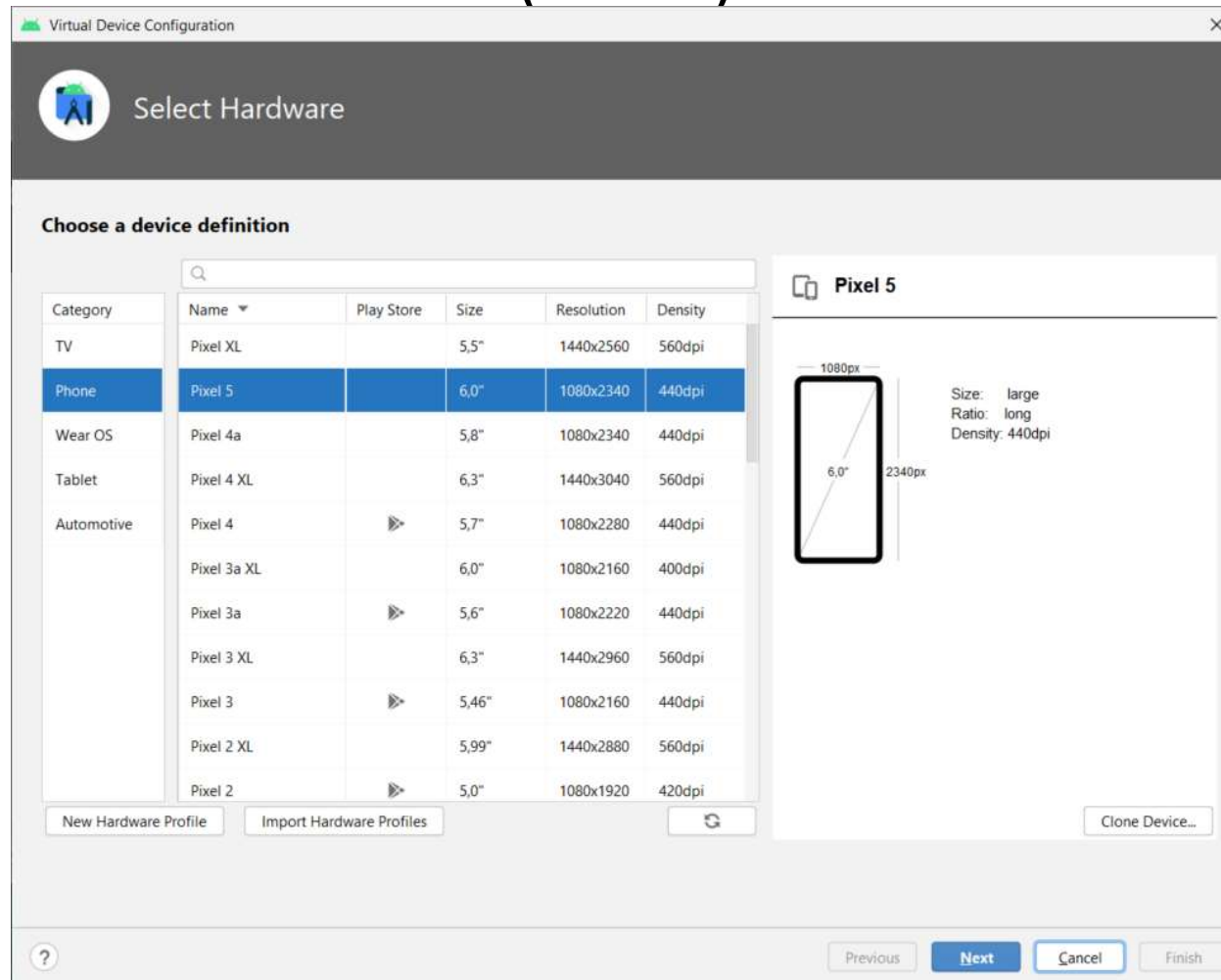
Premier Démarrage

- AndroidStudio doit être configuré !
 - Install Type
 - Choisir Standard
 - L'emplacement du SDK est
 - /local/AndroidSDK
 - Vous pouvez installer différentes version du SDK
 - Les fichiers correspondants sont installés dans /local/AndroidSDK
 - Mais attention à votre quota disque !

Créer un émulateur (AVD)

- Attention, les émulateurs occupent beaucoup d'espace disque (de 1 Go à 8 Go)
- Télécharger l'image système (cf. slides suivants...)
 - « R - API level 30 »
 - ne pas prendre l'API 31 (bug) ?!
- Prendre 5.1_WVGA
 - Internal Storage : 2048 Mb
 - No SDCard

Créer un émulateur (AVD)



Créer un émulateur (AVD)



Virtual Device Configuration

System Image

Select a system image

Recommended x86 Images Other Images

Release Name	API Level	ABI	Target
R	30	x86	Android 11.0 (Google APIs)
Q Download	29	x86	Android 10.0 (Google APIs)
Pie	28	x86	Android 9.0 (Google APIs)
Oreo Download	27	x86	Android 8.1 (Google APIs)
Oreo Download	26	x86	Android 8.0 (Google APIs)
Nougat Download	25	x86	Android 7.1.1 (Google APIs)
Nougat Download	24	x86	Android 7.0 (Google APIs)
Marshmallow Download	23	x86	Android 6.0 (Google APIs)
Lollipop Download	22	x86	Android 5.1 (Google APIs)

R



API Level
30

Android
11.0

Google Inc.

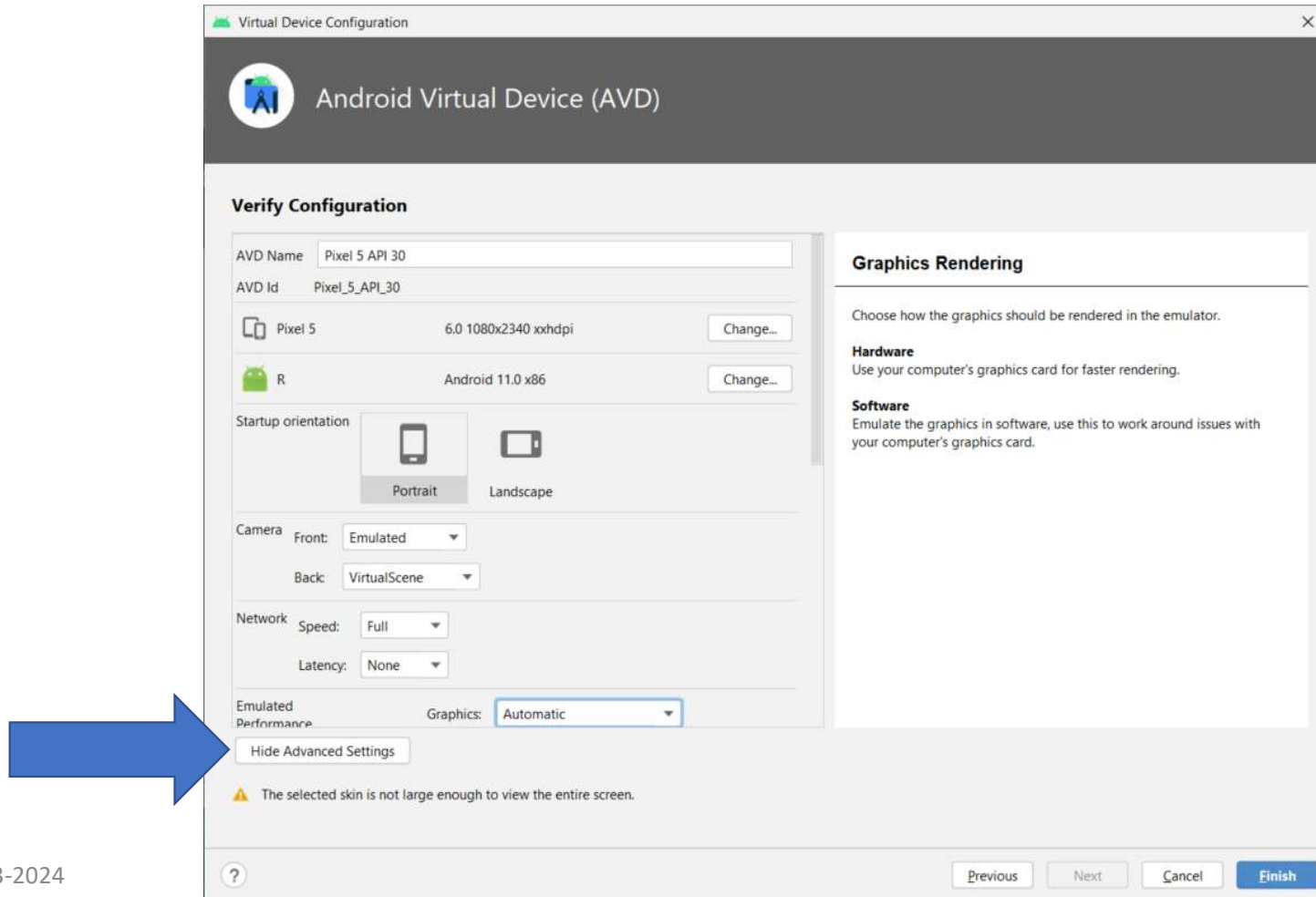
System Image
x86

We recommend these images because they run the fastest and support Google APIs.

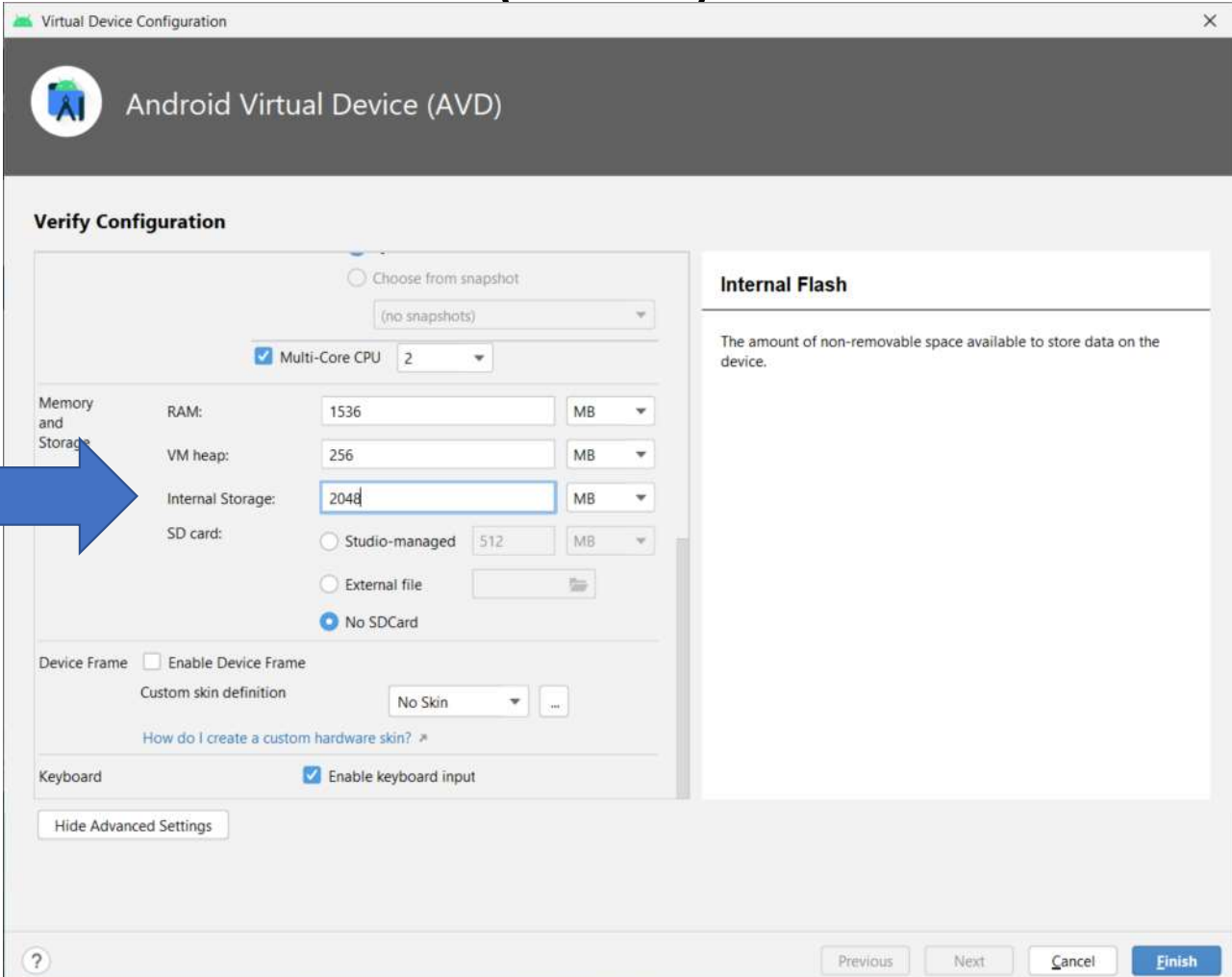
Questions on API level?
[See the API level distribution chart](#)

Previous Next Cancel Finish

Créer un émulateur (AVD)



Créer un émulateur (AVD)



Virtual Device Configuration

Android Virtual Device (AVD)

Verify Configuration

☐ Choose from snapshot
(no snapshots)

☒ Multi-Core CPU 2

Memory and Storage

RAM: 1536 MB

VM heap: 256 MB

Internal Storage: 2048 MB

SD card:

☐ Studio-managed 512 MB

☐ External file

☒ No SDCard

Device Frame

☐ Enable Device Frame

Custom skin definition: No Skin

[How do I create a custom hardware skin? *](#)

Keyboard

☒ Enable keyboard input

[Hide Advanced Settings](#)

Internal Flash

The amount of non-removable space available to store data on the device.

Previous Next Cancel Finish

« J'ai raté le premier démarrage ... »

- Vous pouvez réessayer le paramétrage après avoir nettoyé vos répertoires :
 - Allez à votre racine
 - Affichez les fichiers cachés
 - Supprimez les répertoires contenant 'android'
 - `.android`
 - `~/ .config/Google/.AndroidStudioXxx`
- Attention :
 - Cela supprime toute votre configuration Android !!
 - Mais pas vos projets 😊

Si l'installation de Android est instable...

- Essayer de supprimer le contenu de :
 - /local/AndroidSDK
- Cela supprime les maj et les installations du SDK.
 - Android demandera à les réinstaller

Activer le mode développeur !

- On Android 4.2 and newer, Developer options is hidden by default. To make it available, go to Settings > About phone and tap Build number (ou n° de version en français) seven times.
 - Affichage nouveaux paramètres généraux (options développeurs) ⓘ débogage USB, autoriser les sources inconnues...

Découvrons un peu Android Studio et Android

Hello World!

- Prise en main d'Android Studio (son IHM, ses menus...)
- Principales fonctionnalités
- Création de projets « vides » Hello World! en **Java et Kotlin**
- **Comparaison code : Java / Kotlin (XML) / Kotlin (Compose)**

Juste pour voir les différences
(et prendre un peu l'outil en main),
explications Kotlin et Android + tard

Kotlin (xml)

vs.

Java (xml)

```
class MainActivity : AppCompatActivity() {
```

```
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
    }  
}
```

```
public class MainActivity extends AppCompatActivity {
```

```
    @Override
```

```
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_main);  
    }  
}
```

Semble identique, mais Kotlin fait généralement
gagner beaucoup de lignes de code...
Nous verrons des exemples dans TAC

Kotlin (Compose)

vs.

Java (xml)

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            HelloWorldKotlinComposeTheme {
                // A surface container using the 'background' color from the theme
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colors.background
                ) {
                    Greeting("Android")
                }
            }
        }
    }
}

@Composable
fun Greeting(name: String) {
    Text(text = "Hello $name!")
}

@Preview(showBackground = true)
@Composable
fun DefaultPreview() {
    HelloWorldKotlinComposeTheme {
        Greeting("Android")
    }
}
```

```
public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}

<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello World!"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

Déclarative

vs.

Programmatic

```
class MainActivity : ComponentActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContent {  
            HelloWorldKoltinComposeTheme {  
                // A surface container using the 'background' color from the theme  
                Surface(  
                    modifier = Modifier.fillMaxSize(),  
                    color = MaterialTheme.colors.background  
                ) {  
                    Greeting("Android")  
                }  
            }  
        }  
    }  
}  
  
@Composable  
fun Greeting(name: String) {  
    Text(text = "Hello $name!")  
}  
  
@Preview(showBackground = true)  
@Composable  
fun DefaultPreview() {  
    HelloWorldKoltinComposeTheme {  
        Greeting("Android")  
    }  
}
```

@Composable
vu + loin

```
public class MainActivity extends AppCompatActivity {  
  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_main);  
    }  
}  
  
-----  
  
<?xml version="1.0" encoding="utf-8"?>  
<androidx.constraintlayout.widget.ConstraintLayout  
    xmlns:android="http://schemas.android.com/apk/res/android"  
    xmlns:app="http://schemas.android.com/apk/res-auto"  
    xmlns:tools="http://schemas.android.com/tools"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent"  
    tools:context=".MainActivity">  
  
    <TextView  
        android:layout_width="wrap_content"  
        android:layout_height="wrap_content"  
        android:text="Hello World!"  
        app:layout_constraintBottom_toBottomOf="parent"  
        app:layout_constraintEnd_toEndOf="parent"  
        app:layout_constraintStart_toStartOf="parent"  
        app:layout_constraintTop_toTopOf="parent" />  
  
</androidx.constraintlayout.widget.ConstraintLayout>
```

Architecture du code

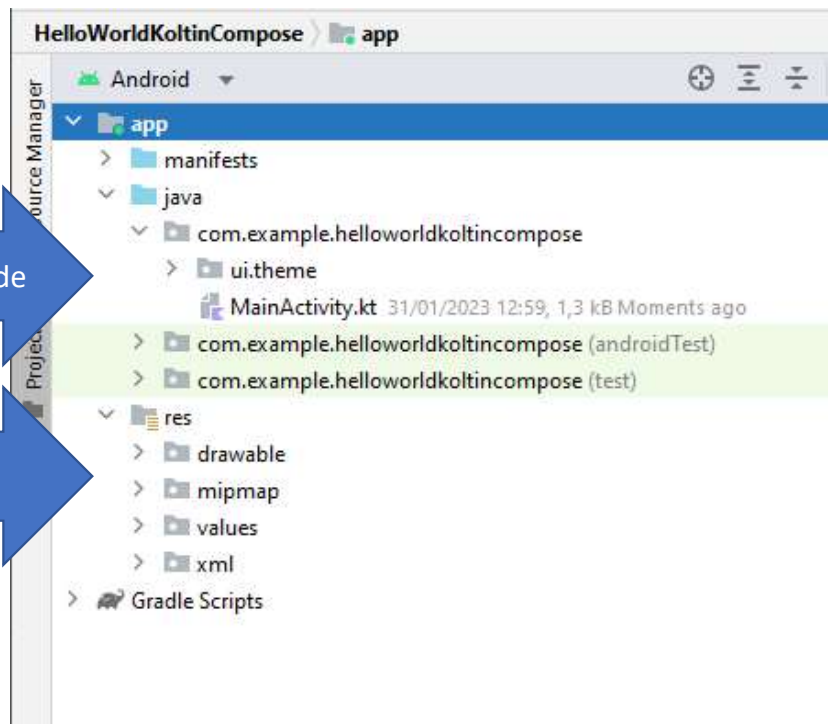
<https://developer.android.com/training/basics/firstapp/creating-project>

Doc > Guide > Create and Android Project

Attention à la vue d'Android Studio (project, android,...).

Architectures différentes

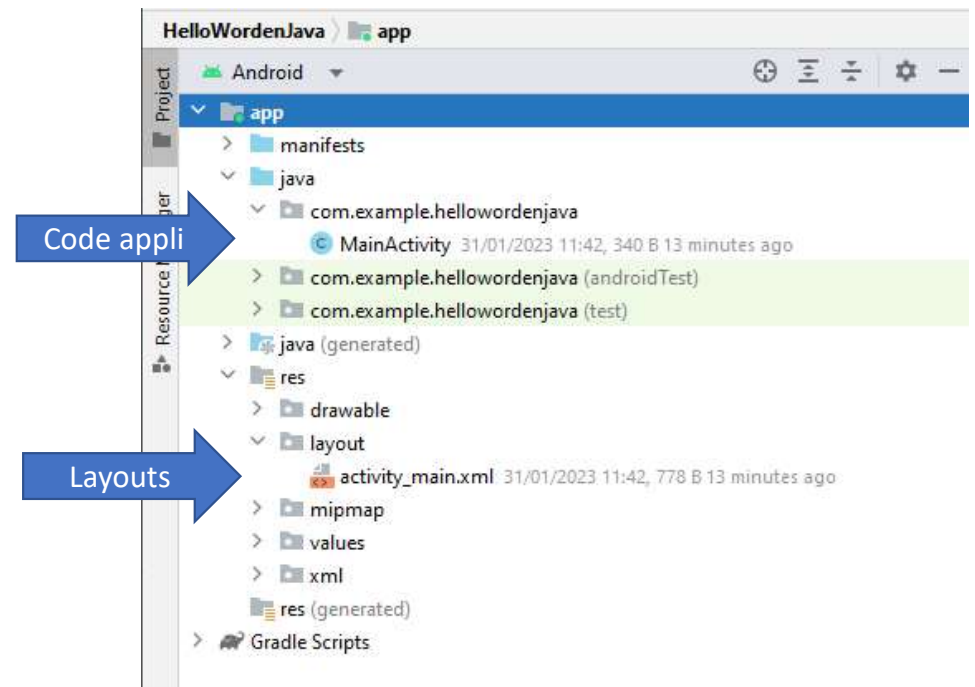
Kotlin + Compose



Thème dans le code

Pas de Layout

Java



Code appli

Layouts

Les ressources

<https://developer.android.com/guide/topics/resources/providing-resources>

Les différents types de ressources : quoi et où les ranger ?

→ **rappel : les layouts XML n'existe plus en Compose**

Comment accéder aux ressources depuis le code ?

R... (ex. R.string.app_name)

Modification du projet Hello World ! Changer de couleur et de texte.

Le Manifest

<https://developer.android.com/guide/topics/manifest/manifest-intro>

App manifest overview.

A quoi ça sert ?

Quoi mettre dedans ?

Le lien avec le PlayStore

Gradle

<https://developer.android.com/studio/build>

Lecture de cette page Android Developers.

Qu'est-ce que /main, /debug, /release, /test ?

Jetpack Compose...?

- Kotlin, ok
- mais Compose ?

Compose 3

- Version 3 : Cf <https://developer.android.com/jetpack/androidx/releases/compose-material3?hl=fr>
 - Et <https://m3.material.io/>
- Les bonnes dépendances
 - <https://developer.android.com/jetpack/androidx/releases/compose?hl=fr>

Compose ➔ @Composable

- Fini les layouts en XML
- Fini les fragments (Yee Haw!)

Mais encore valable quelques « temps » pour assurer la compatibilité avec l'existant
Cf. <https://developer.android.com/jetpack/compose/interop> par exemple

Déclarative

vs.

Programmatic

```
class MainActivity : ComponentActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContent {  
            HelloWorldKoltinComposeTheme {  
                // A surface container using the 'background' color from the theme  
                Surface(  
                    modifier = Modifier.fillMaxSize(),  
                    color = MaterialTheme.colors.background  
                ) {  
                    Greeting("Android")  
                }  
            }  
        }  
    }  
}  
  
@Composable  
fun Greeting(name: String) {  
    Text(text = "Hello $name!")  
}  
  
@Preview(showBackground = true)  
@Composable  
fun DefaultPreview() {  
    HelloWorldKoltinComposeTheme {  
        Greeting("Android")  
    }  
}
```

2023-2024

```
public class MainActivity extends AppCompatActivity {  
  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_main);  
    }  
}  
  
-----  
  
<?xml version="1.0" encoding="utf-8"?>  
<androidx.constraintlayout.widget.ConstraintLayout  
    xmlns:android="http://schemas.android.com/apk/res/android"  
    xmlns:app="http://schemas.android.com/apk/res-auto"  
    xmlns:tools="http://schemas.android.com/tools"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent"  
    tools:context=".MainActivity">  
  
    <TextView  
        android:layout_width="wrap_content"  
        android:layout_height="wrap_content"  
        android:text="Hello World!"  
        app:layout_constraintBottom_toBottomOf="parent"  
        app:layout_constraintEnd_toEndOf="parent"  
        app:layout_constraintStart_toStartOf="parent"  
        app:layout_constraintTop_toTopOf="parent" />  
  
</androidx.constraintlayout.widget.ConstraintLayout>
```

JC Tarby - UE TAC - M1 ES

86

Déclarative

vs.

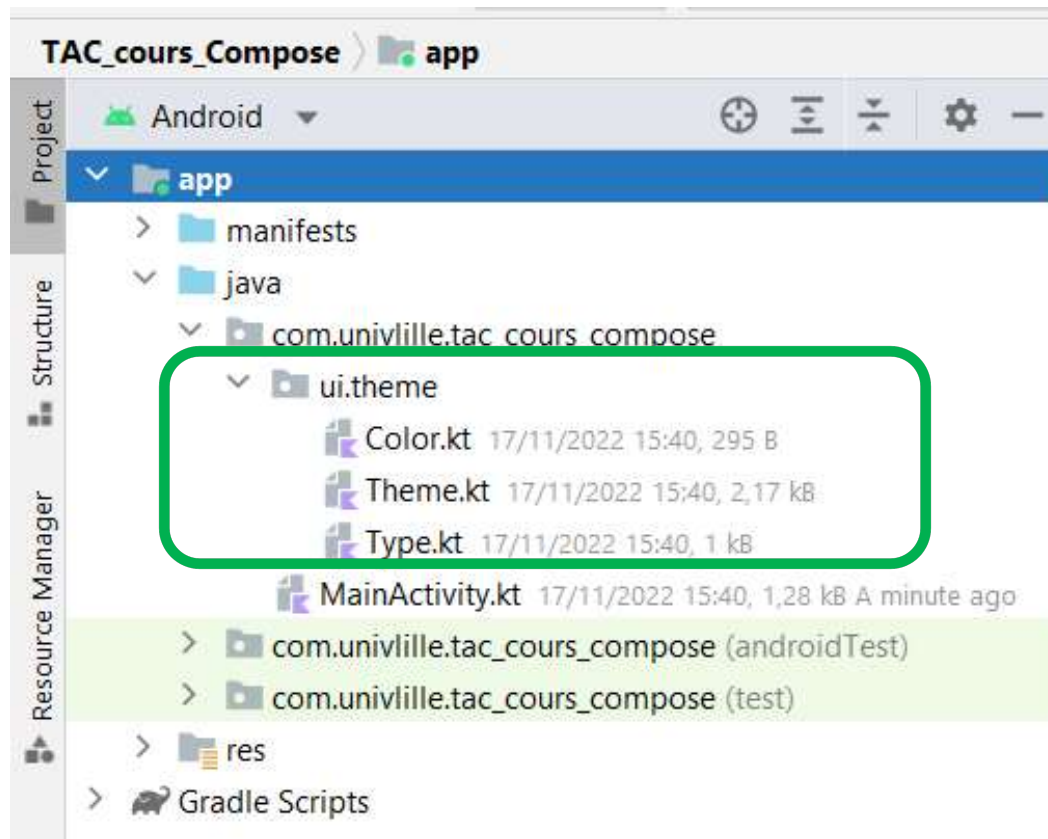
Programmatic

```
class MainActivity : ComponentActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContent {  
            HelloWorldKotlinComposeTheme {  
                // A surface container using the 'background' color from the theme  
                Surface(  
                    modifier = Modifier.fillMaxSize(),  
                    color = MaterialTheme.colors.background  
                ) {  
                    Greeting("Android")  
                }  
            }  
        }  
    }  
}  
  
@Composable  
fun Greeting(name: String) {  
    Text(text = "Hello $name!")  
}  
  
@Preview(showBackground = true)  
@Composable  
fun DefaultPreview() {  
    HelloWorldKotlinComposeTheme {  
        Greeting("Android")  
    }  
}
```

Les noms des
@Composables
commencent toujours par
des majuscules

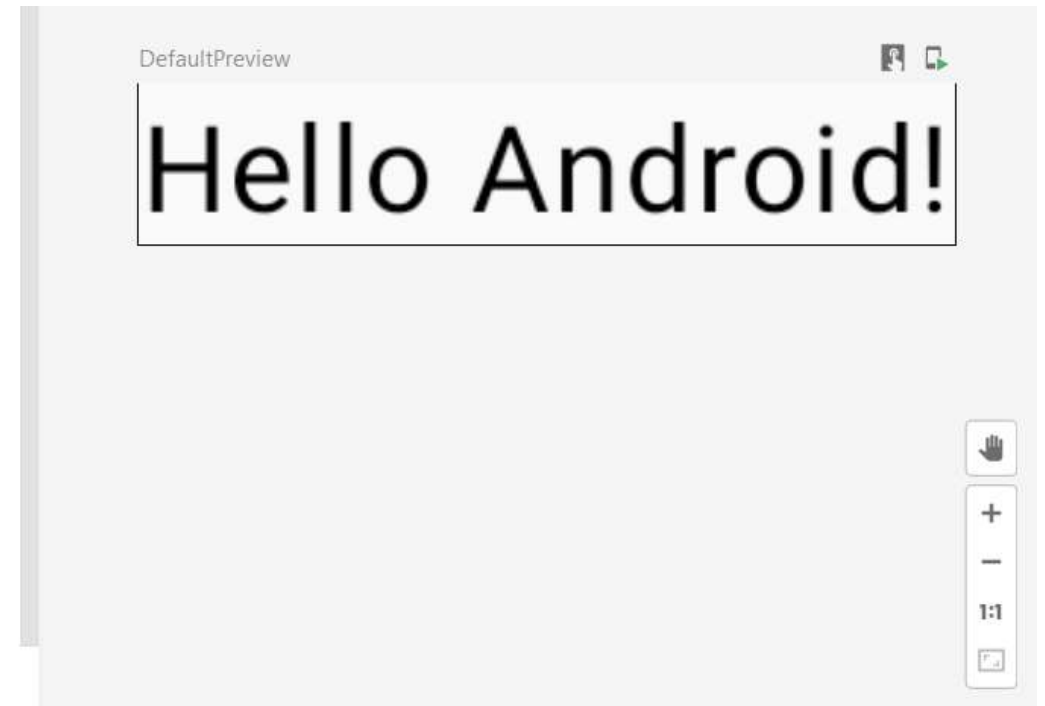
```
public class MainActivity extends AppCompatActivity {  
  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_main);  
    }  
}  
  
-----  
  
<?xml version="1.0" encoding="utf-8"?>  
<androidx.constraintlayout.widget.ConstraintLayout  
    xmlns:android="http://schemas.android.com/apk/res/android"  
    xmlns:app="http://schemas.android.com/apk/res-auto"  
    xmlns:tools="http://schemas.android.com/tools"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent"  
    tools:context=".MainActivity">  
  
    <TextView  
        android:layout_width="wrap_content"  
        android:layout_height="wrap_content"  
        android:text="Hello World!"  
        app:layout_constraintBottom_toBottomOf="parent"  
        app:layout_constraintEnd_toEndOf="parent"  
        app:layout_constraintStart_toStartOf="parent"  
        app:layout_constraintTop_toTopOf="parent" />  
  
</androidx.constraintlayout.widget.ConstraintLayout>
```

Style Material inclus de base



Preview

```
31
32  @Composable
33  fun Greeting(name: String) {
34      Text(text = "Hello $name!")
35  }
36
37  @Preview(showBackground = true)
38  @Composable
39  fun DefaultPreview() {
40      TAC_cours_ComposeTheme {
41          Greeting(name: "Android")
42      }
43  }
```



Preview

Preview : même principe avec Flutter, mais Flutter rafraîchit BEAUCOUP + vite la preview !

```
31  
32 @Comp  
33 fun G  
34 T  
35 }  
36  
37 @Preview(showBackground = true)  
38 @Composable  
39 fun DefaultPreview() {  
40     TAC_cours_Compose  
41     Greeting( nam  
42 }  
43 }
```

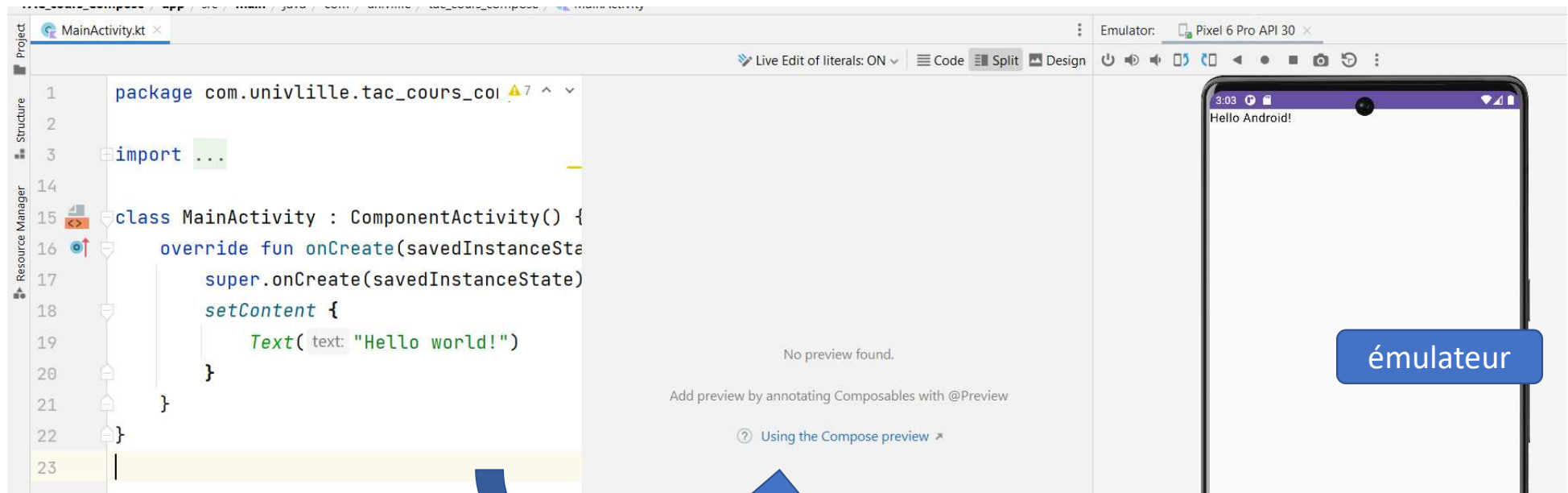
Par contre, en mode run sur téléphone, si vous ne modifiez que des valeurs numériques (taille, padding...), la MAJ se fait en temps réel 😊

Parenthèse sur Flutter

- Les E-Services le découvriront en M2
- Flutter et Compose sont semblables sur pas mal de points (états → mise à jour auto de l'IHM, liste scrollable créée à partir de liste d'éléments...)
 - Si vous comprenez les concepts en Compose, cela sera d'autant + facile avec Flutter
 - Si vous voulez « jeter un œil », demandez à ChatGPT :
 - *écris-moi un exemple de lazycolumn en compose, puis le même exemple en Flutter, et explique moi les ressemblances et les différences,*
 - *donne-moi un exemple de gestion d'état identique entre compose et flutter, de façon à avoir un composant qui possède un état qui permettra de mettre à jour automatique l'ihm si l'état change de valeur*

Réponses dans les
notes du slide 😊

Commençons tout simple... (sans @Composable)

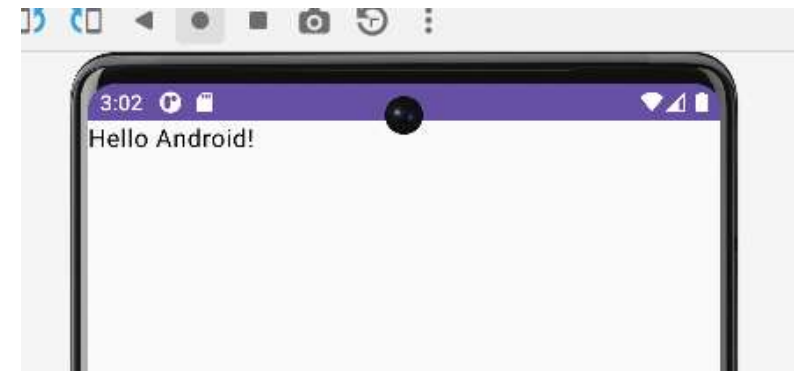


Pas de @Preview donc rien à voir...

Exemple Preview 1

Commençons tout simple...

```
class MainActivity : ComponentActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContent {  
            Text("Hello world!")  
        }  
    }  
}
```



Un premier @Composable et @Preview

The screenshot shows the Android Studio IDE with the following code in MainActivity.kt:

```
package com.univlille.tac_cours_compos

import ...

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            MessageCard(name = "Android")
        }
    }
}

@Composable
fun MessageCard(name: String) {
    Text(text = "Hello $name!")
}

@Preview
@Composable
fun PreviewMessageCard() {
    MessageCard(name = "Android")
}
```

The IDE interface includes a Project view on the left showing the file structure, a central editor for MainActivity.kt, a Design view showing a preview of the MessageCard component with the text "Hello Android!", and an Android emulator on the right displaying "Hello Android!". The bottom status bar shows the date 2023-02-24, the project name "TAC - UE TAC - M1 E-Services", and the commit message "1 file committed: une première fonction @Composable et sa @Preview (moments ago)".

Une première @Composable et @Preview

```
class MainActivity : ComponentActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContent {  
            MessageCard("Android")  
        }  
    }  
}
```

```
@Composable  
fun MessageCard(name: String) {  
    Text(text = "Hello $name!")  
}
```

```
@Preview  
@Composable  
fun PreviewMessageCard() {  
    MessageCard("Android")  
}
```



En « dehors » de la classe, donc fichiers de @Composable possible



Plein de paramètres possibles pour Preview

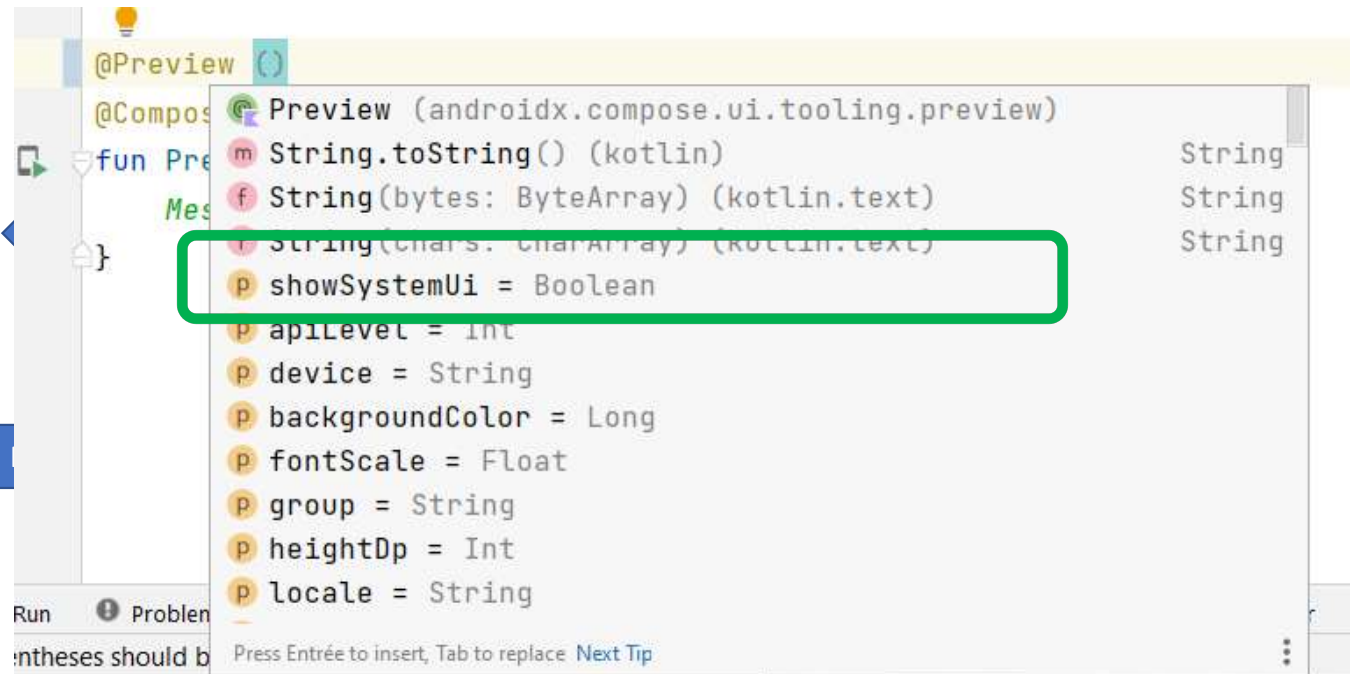
Une première @Composable et @Preview

```
class MainActivity : ComponentActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContent {  
            MessageCard("Android")  
        }  
    }  
}
```

```
@Composable  
fun MessageCard(name: String) {  
    Text(text = "Hello $name!")  
}
```

```
@Preview  
@Composable  
fun PreviewMessageCard() {  
    MessageCard("Android")  
}
```

2023-2024



Live Edit of literals: ON | Code | Split | Design

```
1 package com.univlille.tac_cours_compose
2
3 import ...
4
14
15 class MainActivity : ComponentActivity() {
16     override fun onCreate(savedInstanceState: Bundle?) {
17         super.onCreate(savedInstanceState)
18         setContent {
19             MessageCard( name: "Android")
20         }
21     }
22 }
23
24 @Composable
25 fun MessageCard(name: String) {
26     Text(text = "Hello $name!")
27 }
28
29 @Preview (showSystemUi = true)
30 @Composable
31 fun PreviewMessageCard() {
32     MessageCard( name: "Android")
33 }
```

PreviewMessageCard

12:00
Hello Android!

Start Interactive Mode
Live preview to test gestures

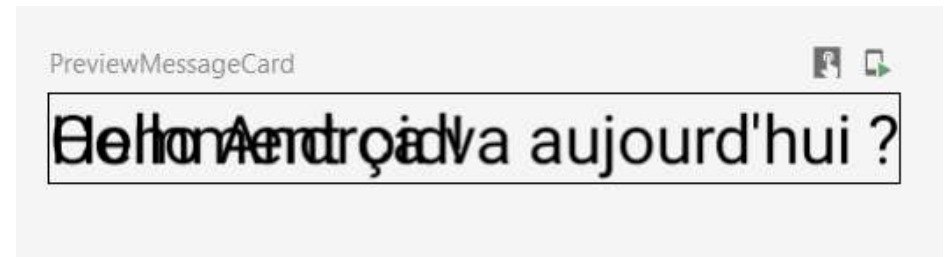
Téléphone affiché

202

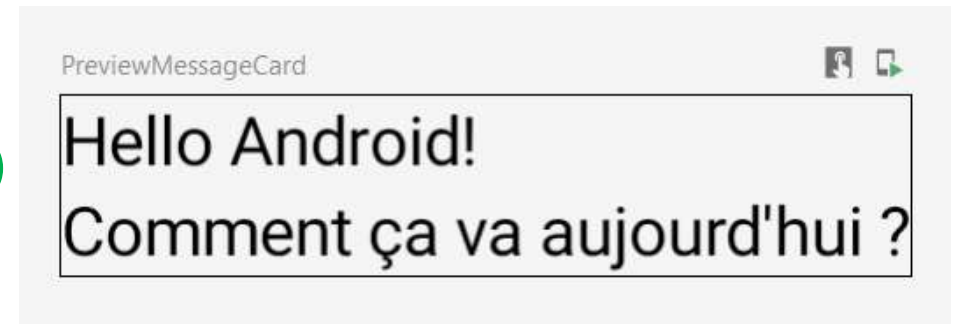
97

Mise en page (layout)

```
@Composable
fun MessageCard(name: String) {
    Text(text = "Hello $name!")
    Text(text = "Comment ça va aujourd'hui ?")
}
```



```
@Composable
fun MessageCard(name: String) {
    Column() {
        Text(text = "Hello $name!")
        Text(text = "Comment ça va aujourd'hui ?")
    }
}
```



Mise en page (layout)

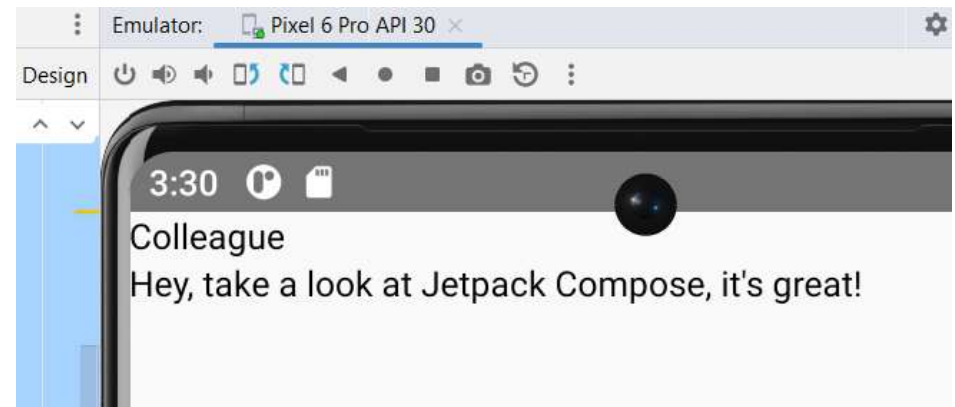
```
setContent {  
    MessageCard(Message("Android", "Jetpack Compose"))  
}
```

Classe de données (Kotlin)

```
data class Message(val author: String, val body: String)
```

```
@Composable  
fun MessageCard(msg: Message) {  
    Column {  
        Text(text = msg.author)  
        Text(text = msg.body)  
    }  
}
```

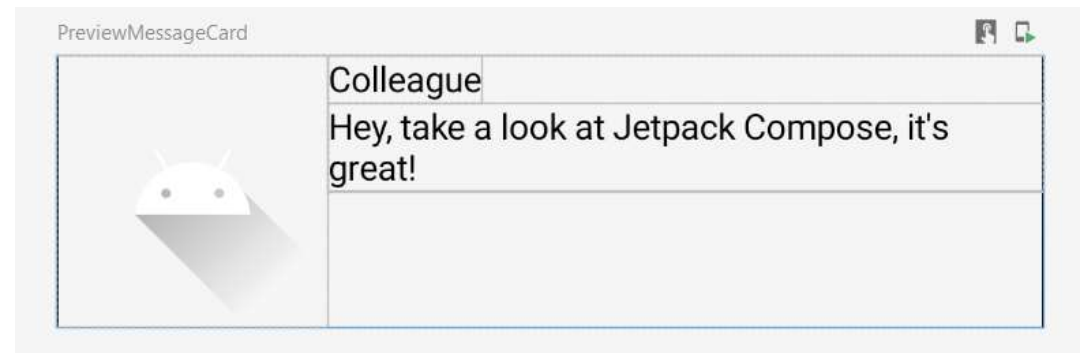
```
@Preview  
@Composable  
fun PreviewMessageCard() {  
    MessageCard(  
        msg = Message("Colleague", "Hey, take a look at Jetpack Compose, it's great!")  
    )  
}
```



Mise en page (layout)

@Composable

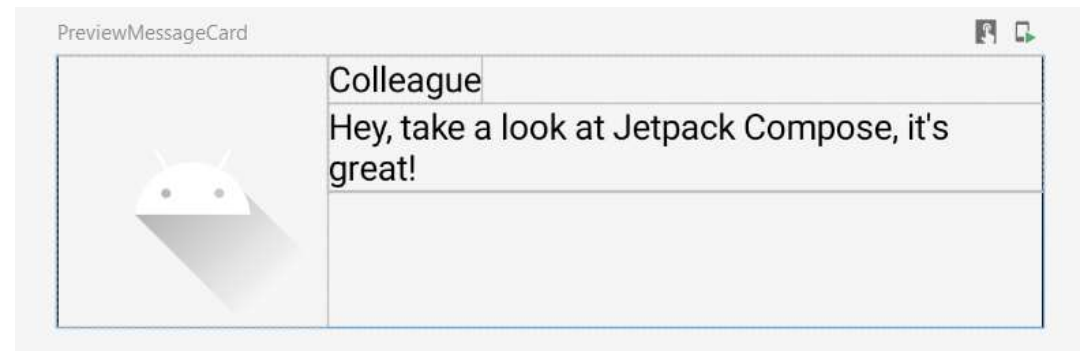
```
fun MessageCard(msg: Message) {  
    Row {  
        Image(  
            painter = painterResource(R.drawable.ic_launcher_foreground),  
            contentDescription = "Contact profile picture",  
        )  
  
        Column {  
            Text(text = msg.author)  
            Text(text = msg.body)  
        }  
    }  
}
```



Mise en page (layout)

```
@Composable
fun MessageCard(msg: Message) {
    Row {
        Image(
            painter = painterResource(R.drawable.ic_launcher_foreground),
            contentDescription = "Contact profile picture",
        )
        Column {
            Text(text = msg.author)
            Text(text = msg.body)
        }
    }
}
```

Si souci pour l'import d'image :
`import androidx.compose.foundation.Image`

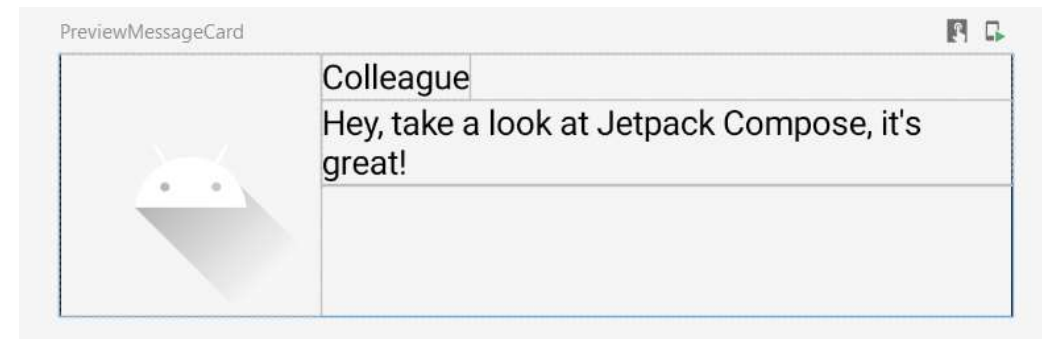


Mise en page (Modifier)

```
@Composable
fun MessageCard(msg: Message) {
    // Add padding around our message
    Row(modifier = Modifier.padding(all = 8.dp)) {
        Image(
            painter = painterResource(R.drawable.ic_launcher_foreground),
            contentDescription = "Contact profile picture",
            modifier = Modifier
                // Set image size to 40 dp
                .size(40.dp)
                // Clip image to be shaped as a circle
                .clip(CircleShape)

            // Add a horizontal space between the image and the column
            Spacer(modifier = Modifier.width(8.dp))

            Column {
                Text(text = msg.author)
                // Add a vertical space between the author and message texts
                Spacer(modifier = Modifier.height(4.dp))
                Text(text = msg.body)
            }
        }
    }
}
```

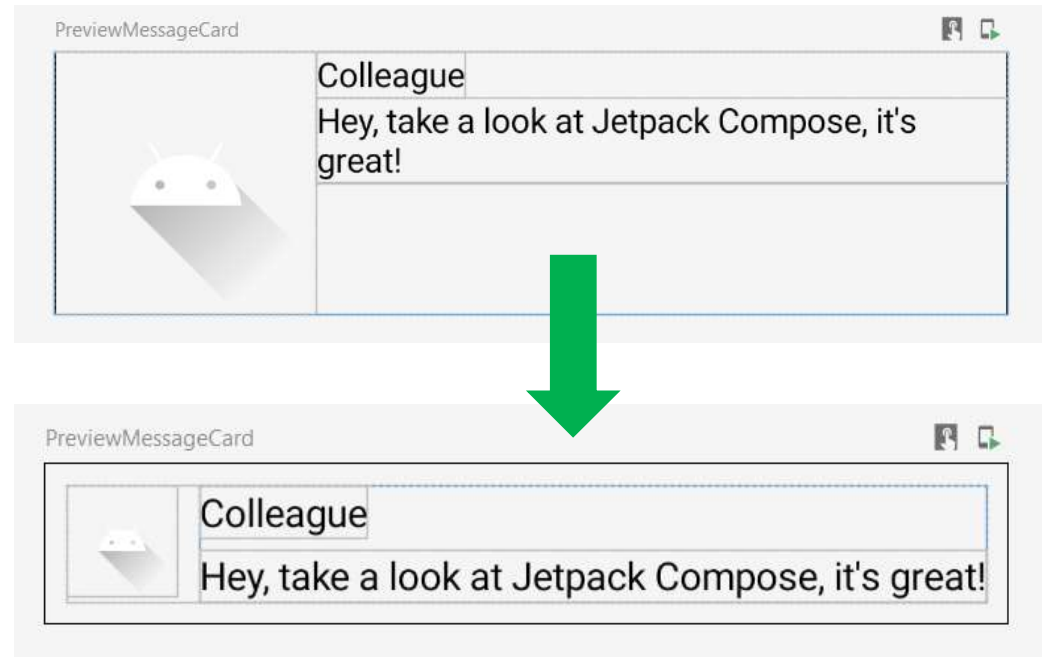


Mise en page (Modifier)

```
@Composable
fun MessageCard(msg: Message) {
    // Add padding around our message
    Row(modifier = Modifier.padding(all = 8.dp)) {
        Image(
            painter = painterResource(R.drawable.ic_launcher_foreground),
            contentDescription = "Contact profile picture",
            modifier = Modifier
                // Set image size to 40 dp
                .size(40.dp)
                // Clip image to be shaped as a circle
                .clip(CircleShape)
        )

        // Add a horizontal space between the image and the column
        Spacer(modifier = Modifier.width(8.dp))

        Column {
            Text(text = msg.author)
            // Add a vertical space between the author and message texts
            Spacer(modifier = Modifier.height(4.dp))
            Text(text = msg.body)
        }
    }
}
```



Material Design (Theme)

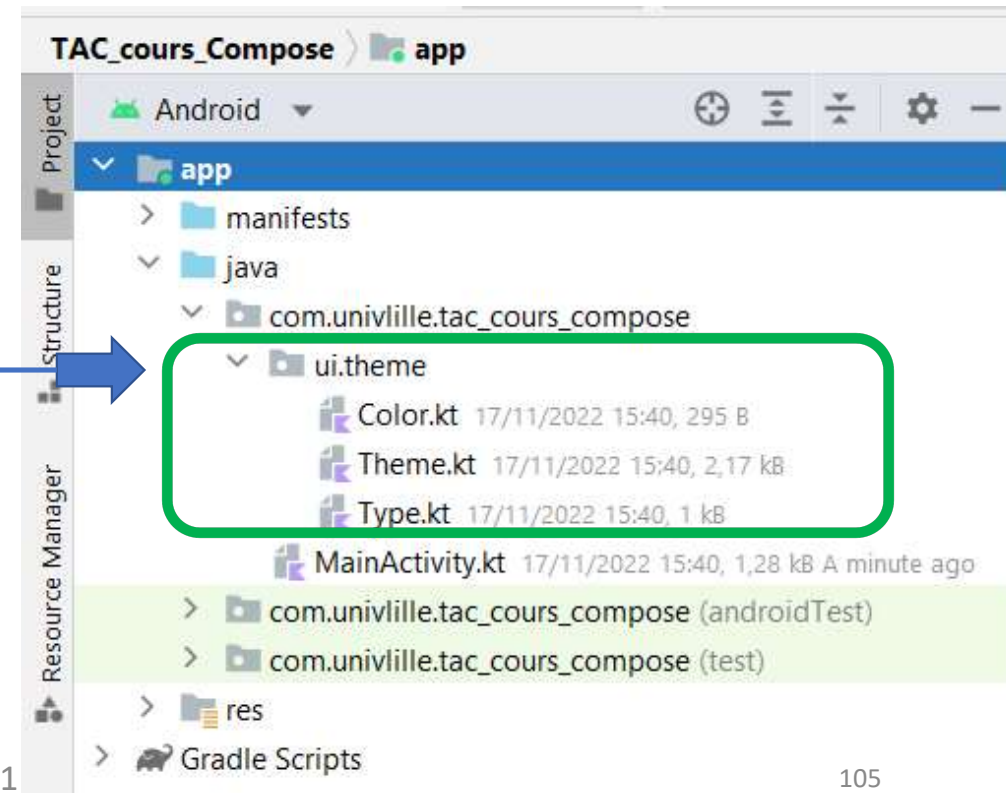
The screenshot shows an IDE interface with the 'Structure' and 'Resource Manager' toolbars on the left. The 'Structure' pane displays the package hierarchy: `manifests`, `java`, `com.univlille.tac_cours_compose` (highlighted with a green box), and `ui.theme`. A blue arrow points from the `ui.theme` package to the `Theme.kt` file, which is also highlighted with a green box. The 'Resource Manager' pane shows the file `Theme.kt` (17/11/2022 15:40, 2,17 kB) selected. The main editor displays the content of `Theme.kt`, which is also labeled in a separate box at the top right. The code defines a `TAC_cours_ComposeTheme` function, highlighted with a green box, which is annotated with `@Composable`. The function signature is `fun TAC_cours_ComposeTheme(`. The code includes parameters for `darkTheme`, `dynamicColor`, and `content`. The `dynamicColor` parameter is set to `true` and has a comment: `// Dynamic color is available on Android 12+`. The `content` parameter is of type `@Composable () -> Unit`. The function body starts with `val colorScheme = when {`.

```
38 )
39
40 @Composable
41 fun TAC_cours_ComposeTheme(
42     darkTheme: Boolean = isSystemInDarkTheme(),
43     // Dynamic color is available on Android 12+
44     dynamicColor: Boolean = true,
45     content: @Composable () -> Unit
46 ) {
47     val colorScheme = when {
```

Material Design

Dans activité

```
class MainActivity : ComponentActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContent {  
            TAC_cours_ComposeTheme {  
                Surface(modifier = Modifier.fillMaxSize()) {  
                    MessageCard(Message("Android", "Jetpack Compose"))  
                }  
            }  
        }  
    }  
}
```



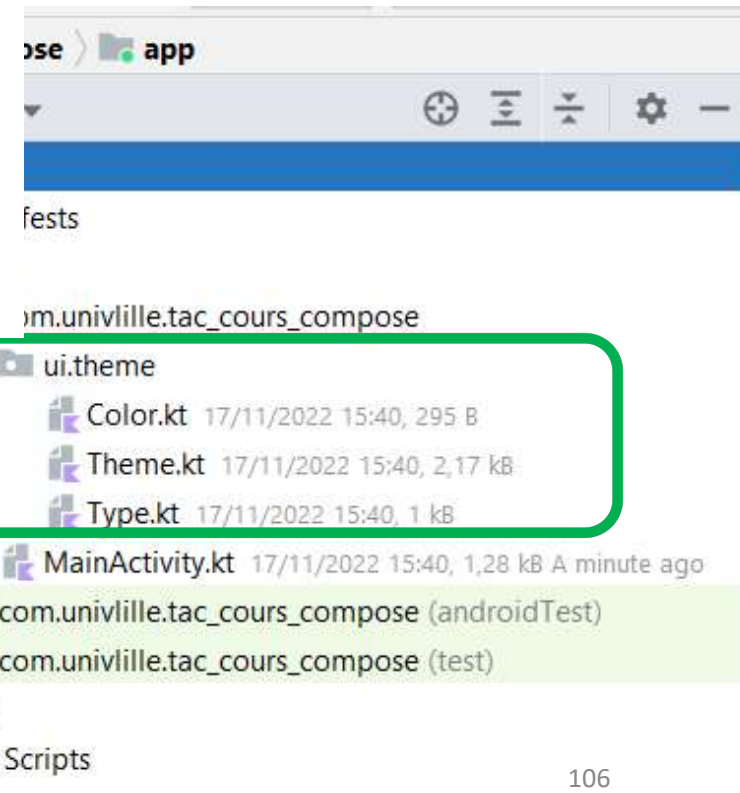
Material Design

@Preview

@Composable

```
fun PreviewMessageCard() {  
    TAC_cours_ComposeTheme {  
        Surface {  
            MessageCard(  
                msg = Message("Colleague", "Hey, take a look at Jetpack Compose, it's great!")  
            )  
        }  
    }  
}
```

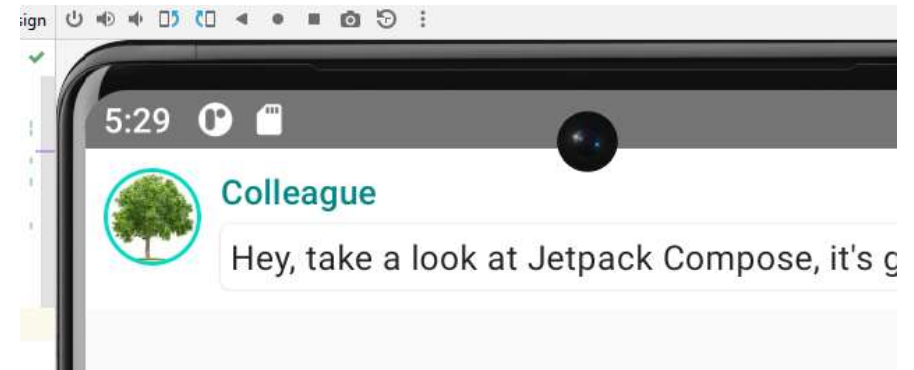
Dans Preview



@Composable

```
fun MessageCard(msg: Message) {  
    Row(modifier = Modifier.padding(all = 8.dp)) {  
        Image(  
            painter = painterResource(R.drawable.arbre),  
            contentDescription = null,  
            modifier = Modifier  
                .size(40.dp)  
                .clip(CircleShape)  
                .border(1.5.dp, MaterialTheme.colors.secondary, CircleShape)  
        )  
        Spacer(modifier = Modifier.width(8.dp))  
  
        Column {  
            Text(  
                text = msg.author,  
                color = MaterialTheme.colors.secondaryVariant,  
                style = MaterialTheme.typography.subtitle2  
            )  
  
            Spacer(modifier = Modifier.height(4.dp))  
  
            Surface(shape = MaterialTheme.shapes.medium, elevation = 1.dp) {  
                Text(  
                    text = msg.body,  
                    modifier = Modifier.padding(all = 4.dp),  
                    style = MaterialTheme.typography.body2  
                )  
            }  
        }  
    }  
}
```

Material Design

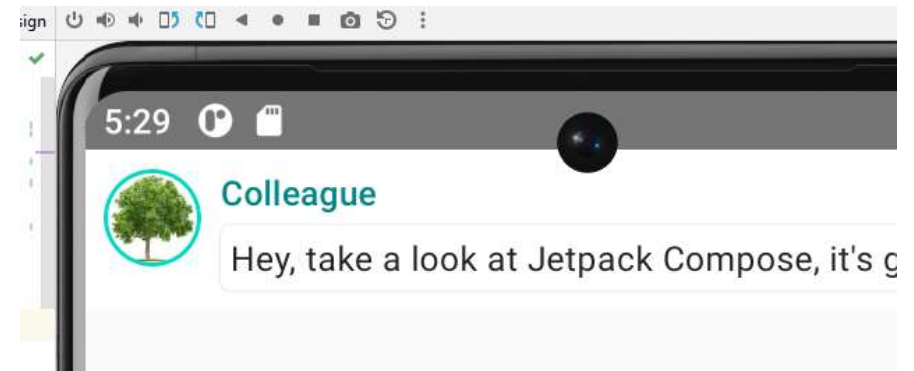


Material Design

@Composable

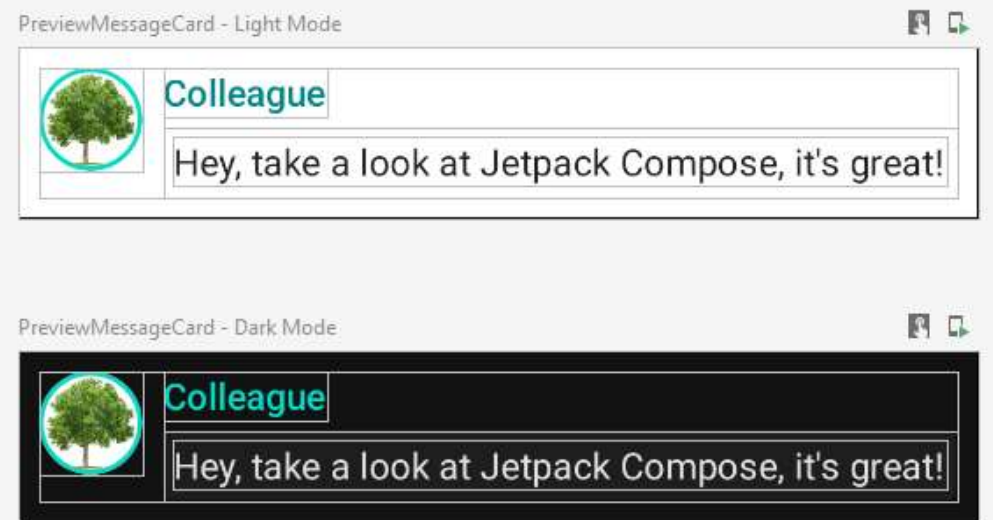
```
fun MessageCard(msg: Message) {  
    Row(modifier = Modifier.padding(all = 8.dp)) {  
        Image(  
            painter = painterResource(R.drawable.arbre),  
            contentDescription = null,  
            modifier = Modifier  
                .size(40.dp)  
                .clip(CircleShape)  
                .border(1.5.dp, MaterialTheme.colors.secondary, CircleShape)  
        )  
        Spacer(modifier = Modifier.width(8.dp))  
  
        Column {  
            Text(  
                text = msg.author,  
                color = MaterialTheme.colors.secondaryVariant,  
                style = MaterialTheme.typography.subtitle2  
            )  
  
            Spacer(modifier = Modifier.height(4.dp))  
  
            Surface(shape = MaterialTheme.shapes.medium, elevation = 1.dp) {  
                Text(  
                    text = msg.body,  
                    modifier = Modifier.padding(all = 4.dp),  
                    style = MaterialTheme.typography.body2  
                )  
            }  
        }  
    }  
}
```

Déjà des changements depuis...
colorScheme à présent



Multiple preview (dark mode géré de base)

```
@Preview(name = "Light Mode")
@Preview(
    uiMode = Configuration.UI_MODE_NIGHT_YES,
    showBackground = true,
    name = "Dark Mode"
)
@Composable
fun PreviewMessageCard() {
    TAC_cours_ComposeTheme {
        Surface {
            MessageCard(
                msg = Message("Colleague", "Hey, take a look at Jetpack Compose, it's
            )
        }
    }
}
```



Multiple preview (dark mode géré de base)

```
@Preview(name = "Light Mode")
@Preview(
    uiMode = UI_MODE_NIGHT_YES,
    showBackground = true
)
```

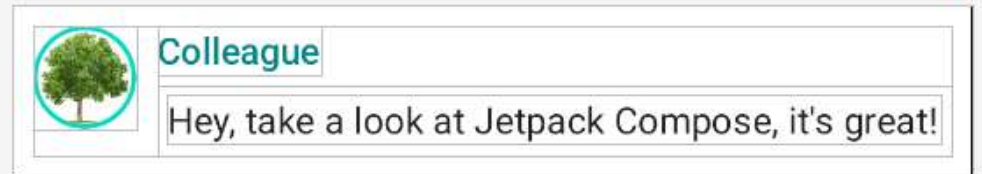
dans Theme.kt

```
private val LightColorPalette = lightColors(
    primary = Purple500,
    primaryVariant = Purple700,
    secondary = Teal200
)
```

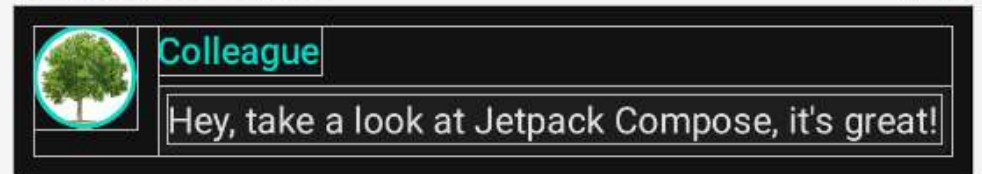
```
private val DarkColorPalette = darkColors(
    primary = Purple200,
    primaryVariant = Purple700,
    secondary = Teal200
)
```

Avec des
« Scheme »
maintenant

PreviewMessageCard - Light Mode



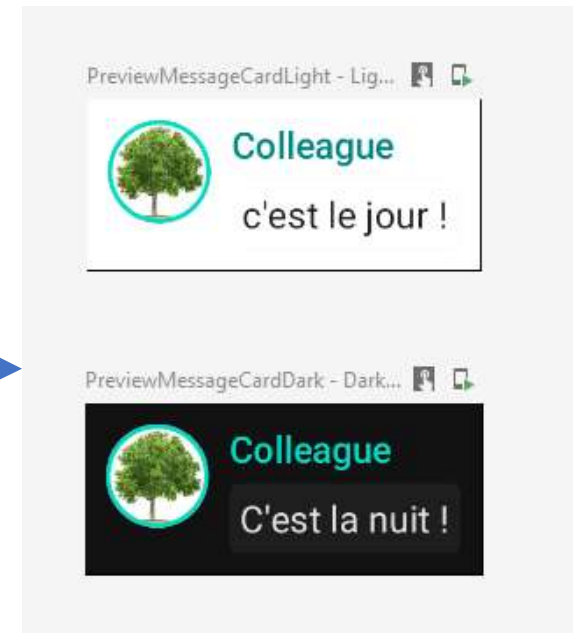
PreviewMessageCard - Dark Mode



Multiple preview séparés

```
@Preview(name = "Light Mode")
@Composable
fun PreviewMessageCardLight() {
    TAC_cours_ComposeTheme {
        Surface {
            MessageCard(
                msg = Message("Colleague", "c'est le jour !")
            )
        }
    }
}
```

```
@Preview(
    uiMode = Configuration.UI_MODE_NIGHT_YES,
    showBackground = true,
    name = "Dark Mode"
)
@Composable
fun PreviewMessageCardDark() {
    TAC_cours_ComposeTheme {
        Surface {
            MessageCard(
                msg = Message("Colleague", "C'est la nuit !")
            )
        }
    }
}
```



RecyclerView...

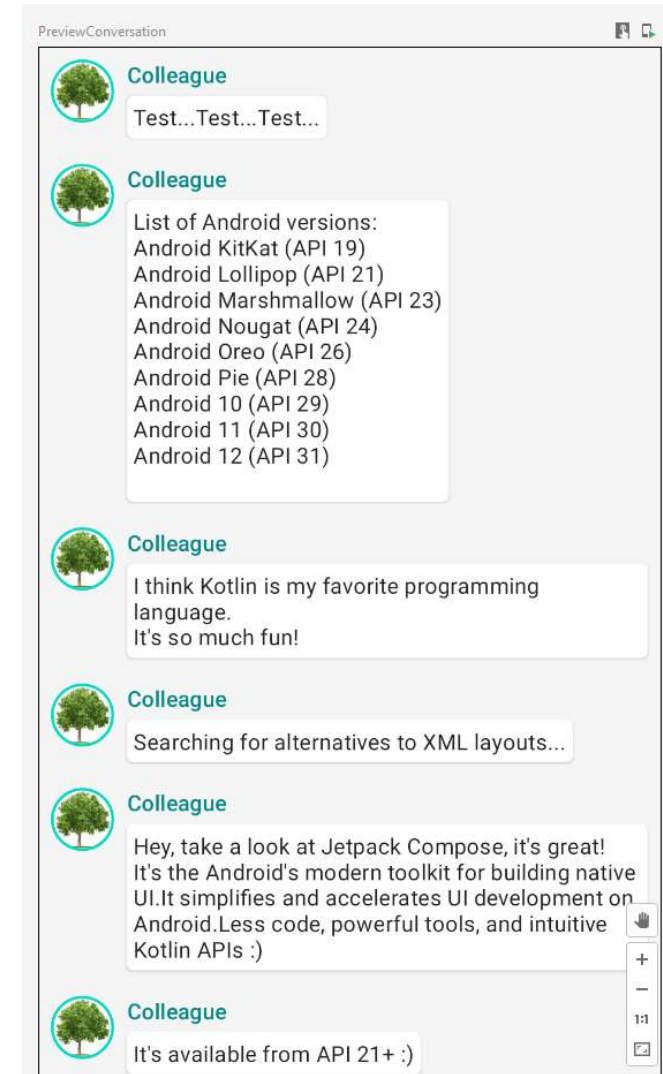
- Fini les mauvais souvenirs avec Java et XML, ViewHolder, Adapter et autres !
Yee Haw! 😊

LazyColumn (ex-RecyclerView)

```
class MainActivity : ComponentActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContent {  
            TAC_cours_ComposeTheme {  
                Surface(modifier = Modifier.fillMaxSize()) {  
                    Conversation(SampleData.conversationSample)  
                }  
            }  
        }  
    }  
}  
  
@Composable  
fun Conversation(messages: List<Message>) {  
    LazyColumn {  
        messages.map { item { MessageCard(it) } }  
    }  
}
```

2023-2024

JC Tarby - UE TAC - M1 E-Services

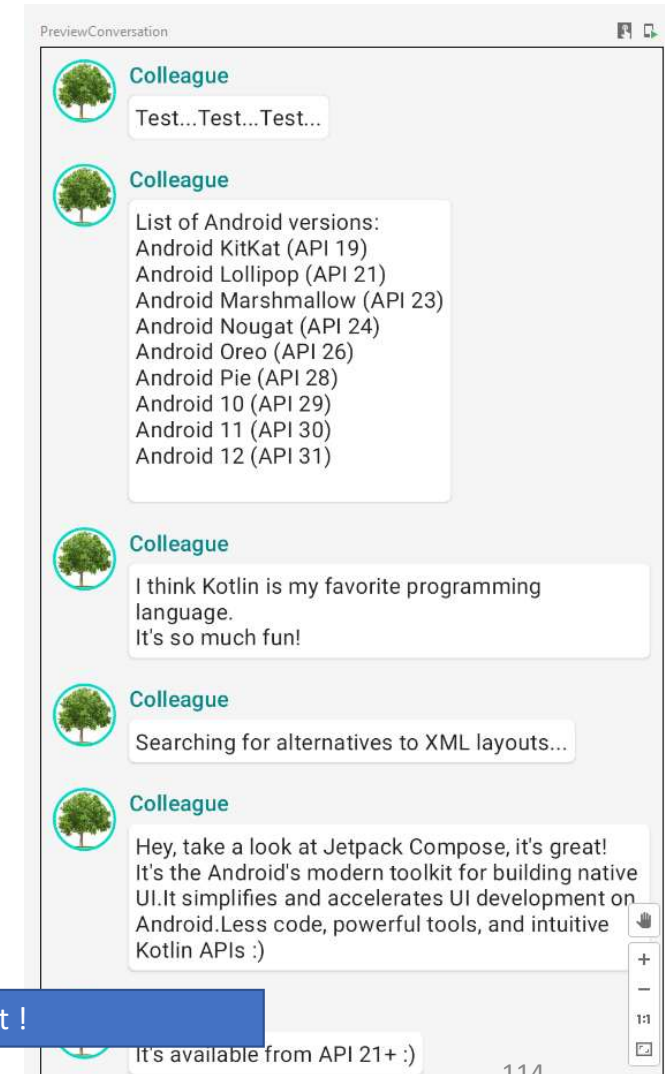


LazyColumn (ex-RecyclerView)

```
class MainActivity : ComponentActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContent {  
            TAC_cours_ComposeTheme {  
                Surface(modifier = Modifier.fillMaxSize()) {  
                    Conversation(SampleData.conversationSample)  
                }  
            }  
        }  
    }  
}
```

```
@Composable  
fun Conversation(messages: List<Message>) {  
    LazyColumn {  
        messages.map { item { MessageCard(it) } }  
    }  
}
```

Pour chaque élément de la liste, on crée un **item** de la liste avec le Composable indiqué, et c'est tout !



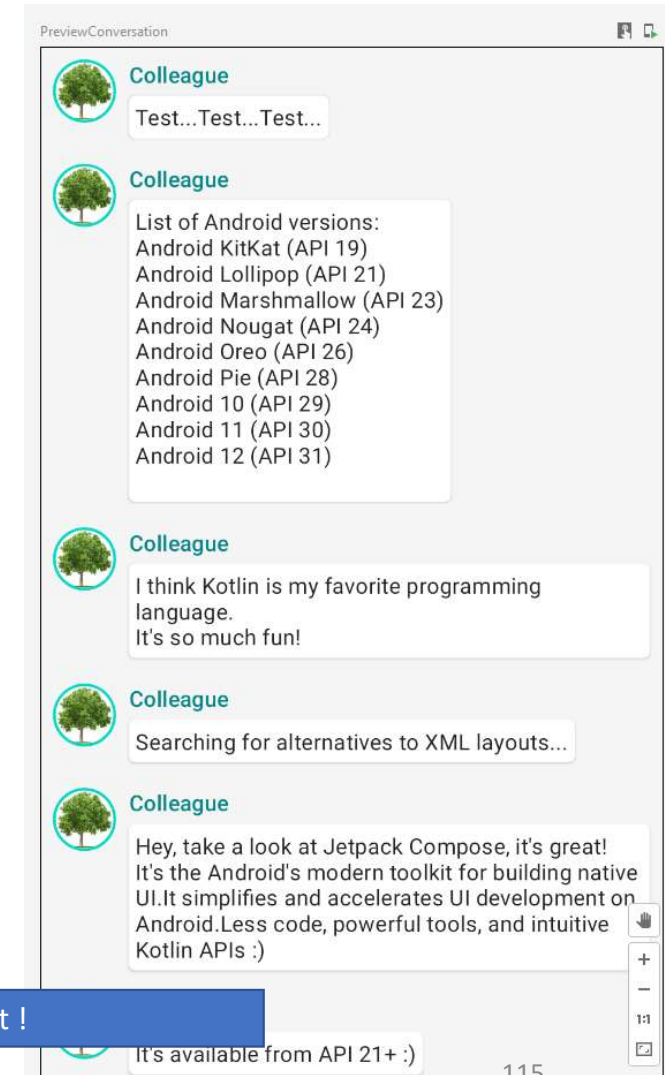
LazyColumn (ex-RecyclerView)

Code équivalent :

```
@Composable
fun Conversation(messages: List<Message>) {
    LazyColumn {
        messages.map { message ->
            item {
                MessageCard(message)
            }
        }
    }
}
```

```
@Composable
fun Conversation(messages: List<Message>) {
    LazyColumn {
        messages.map { item { MessageCard(it) } }
    }
}
```

Pour chaque élément de la liste, on crée un **item** de la liste avec le Composable indiqué, et c'est tout !



Extrait de MessageCard

```
// We keep track if the message is expanded or not in this
// variable
var isExpanded by remember { mutableStateOf(false) }
// surfaceColor will be updated gradually from one color to the other
val surfaceColor by animateColorAsState(
    if (isExpanded) MaterialTheme.colors.primary else MaterialTheme.colors.surface,
)

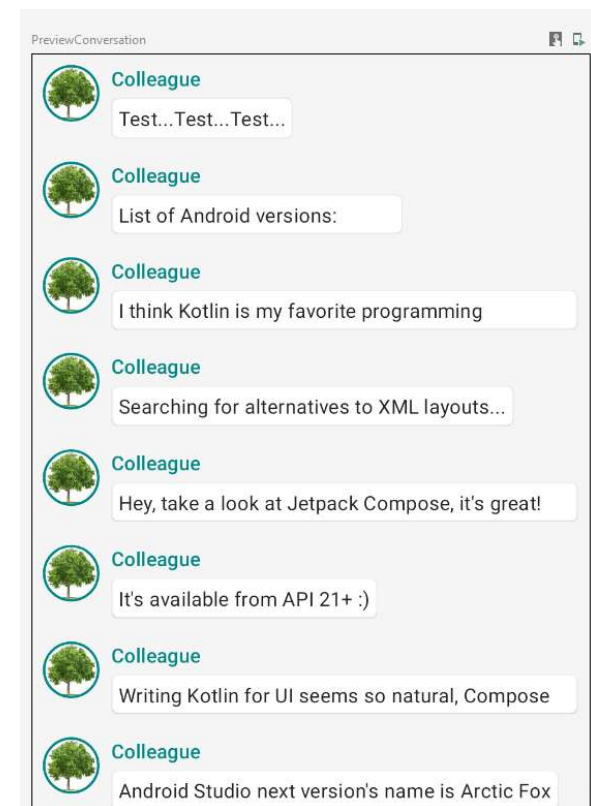
// We toggle the isExpanded variable when we click on this Column
Column(modifier = Modifier.clickable { isExpanded = !isExpanded }) {
    Text(
        text = msg.author,
        color = MaterialTheme.colors.secondaryVariant,
        style = MaterialTheme.typography.subtitle2
    )

    Spacer(modifier = Modifier.height(4.dp))

    Surface(
        shape = MaterialTheme.shapes.medium,
        elevation = 1.dp,
        // surfaceColor color will be changing gradually from primary to surface
        color = surfaceColor,
        // animateContentSize will change the Surface size gradually
        modifier = Modifier.animateContentSize().padding(1.dp)
    ) {
        Text(
            text = msg.body,
            modifier = Modifier.padding(all = 4.dp),
            // If the message is expanded, we display all its content
            // otherwise we only display the first line
            maxLines = if (isExpanded) Int.MAX_VALUE else 1,

```

Gestion des interactions (ex. de la liste)



Extrait de MessageCard

```
// We keep track if the message is expanded or not in this
// variable
var isExpanded by remember { mutableStateOf(false) }
// surfaceColor will be updated gradually from one color to the other
val surfaceColor by animateColorAsState(
    if (isExpanded) MaterialTheme.colors.primary else MaterialTheme.colors.secondary,
)

// We toggle the isExpanded variable when we click on this Column
Column(modifier = Modifier.clickable { isExpanded = !isExpanded }) {
    Text(
        text = msg.author,
        color = MaterialTheme.colors.secondaryVariant,
        style = MaterialTheme.typography.subtitle2
    )

    Spacer(modifier = Modifier.height(4.dp))

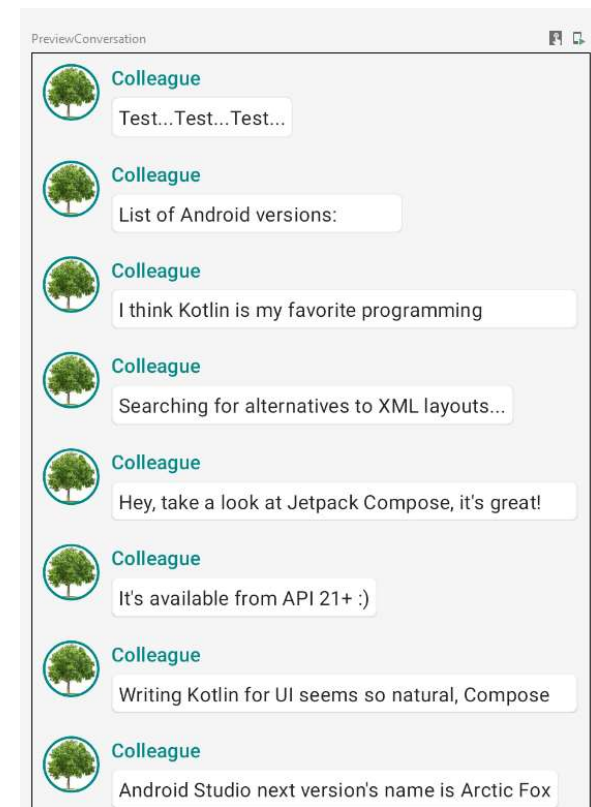
    Surface(
        shape = MaterialTheme.shapes.medium,
        elevation = 1.dp,
        // surfaceColor color will be changing gradually from primary to surface
        color = surfaceColor,
        // animateContentSize will change the Surface size gradually
        modifier = Modifier.animateContentSize().padding(1.dp)
    ) {
        Text(
            text = msg.body,
            modifier = Modifier.padding(all = 4.dp),
            // If the message is expanded, we display all its content
            // otherwise we only display the first line
            maxLines = if (isExpanded) Int.MAX_VALUE else 1,
        )
    }
}
```

Booléen
ouvert/fermé

Bascule si on clique

Si ouvert, on affiche tout le texte
de la Card, sinon juste une ligne

Gestion des interactions (ex. de la liste)



Extrait de MessageCard

```
// We keep track if the message is expanded or not in this
// variable
var isExpanded by remember { mutableStateOf(false) }
// surfaceColor will be updated gradually from one color to the other
val surfaceColor by animateColorAsState(
    if (isExpanded) MaterialTheme.colors.primary else MaterialTheme.colors.surface,
)

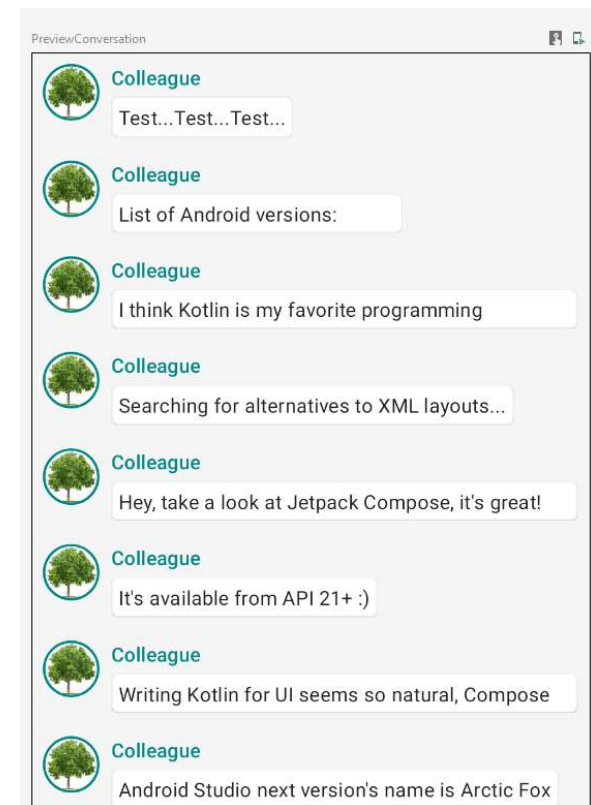
// We toggle the isExpanded variable when we click on this Column
Column(modifier = Modifier.clickable { isExpanded = !isExpanded }) {
    Text(
        text = msg.author,
        color = MaterialTheme.colors.secondaryVariant,
        style = MaterialTheme.typography.subtitle2
    )

    Spacer(modifier = Modifier.height(4.dp))

    Surface(
        shape = MaterialTheme.shapes.medium,
        elevation = 1.dp,
        // surfaceColor color will be changing gradually from primary to surface
        color = surfaceColor,
        // animateContentSize will change the Surface size gradually
        modifier = Modifier.animateContentSize().padding(1.dp)
    ) {
        Text(
            text = msg.body,
            modifier = Modifier.padding(all = 4.dp),
            // If the message is expanded, we display all its content
            // otherwise we only display the first line
            maxLines = if (isExpanded) Int.MAX_VALUE else 1,
        )
    }
}
```

Gestion des interactions (ex. de la liste)

State → Mécanisme de
« Recomposition »



Extrait de MessageCard

```
// We keep track if the message is expanded or not in this
// variable
var isExpanded by remember { mutableStateOf(false) }
// surfaceColor will be updated gradually from one color to the other
val surfaceColor by animateColorAsState(
    if (isExpanded) MaterialTheme.colors.primary else MaterialTheme.colors.surface,
)

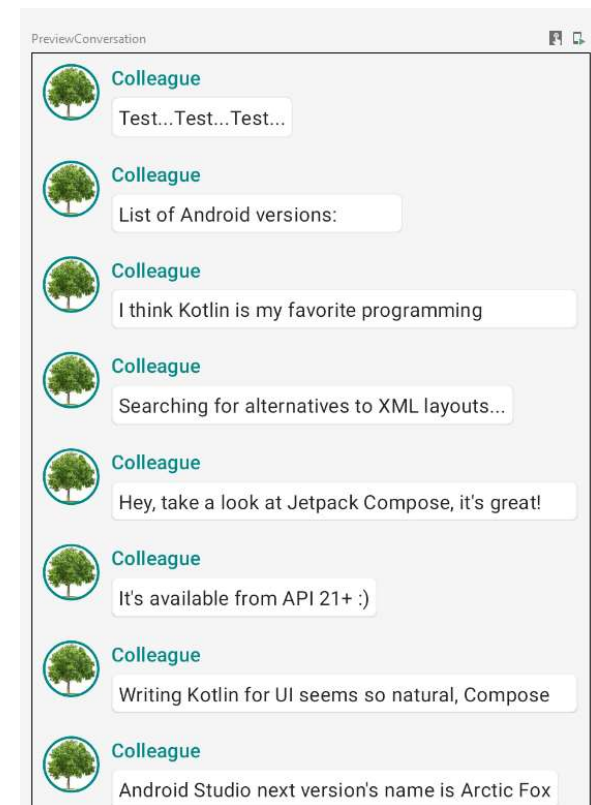
// We toggle the isExpanded variable when we click on this Column
Column(modifier = Modifier.clickable { isExpanded = !isExpanded }) {
    Text(
        text = msg.author,
        color = MaterialTheme.colors.secondaryVariant,
        style = MaterialTheme.typography.subtitle2
    )

    Spacer(modifier = Modifier.height(4.dp))

    Surface(
        shape = MaterialTheme.shapes.medium,
        elevation = 1.dp,
        // surfaceColor color will be changing gradually from primary to surface
        color = surfaceColor,
        // animateContentSize will change the Surface size gradually
        modifier = Modifier.animateContentSize().padding(1.dp)
    ) {
        Text(
            text = msg.body,
            modifier = Modifier.padding(all = 4.dp),
            // If the message is expanded, we display all its content
            // otherwise we only display the first line
            maxLines = if (isExpanded) Int.MAX_VALUE else 1,
        )
    }
}
```

Gestion des interactions (ex. de la liste)

State → Mécanisme de
« Recomposition »



Extrait de MessageCard

```
// We keep track if the message is expanded or not in this
// variable
var isExpanded by remember { mutableStateOf(false) }
// surfaceColor will be updated gradually from one color to the other
val surfaceColor by animateColorAsState(
    if (isExpanded) MaterialTheme.colors.primary else MaterialTheme.colors.surface,
)

// We toggle the isExpanded variable when we click on this Column
Column(modifier = Modifier.clickable { isExpanded = !isExpanded }) {
    Text(
        text = msg.author,
        color = MaterialTheme.colors.secondaryVariant,
        style = MaterialTheme.typography.subtitle2
    )

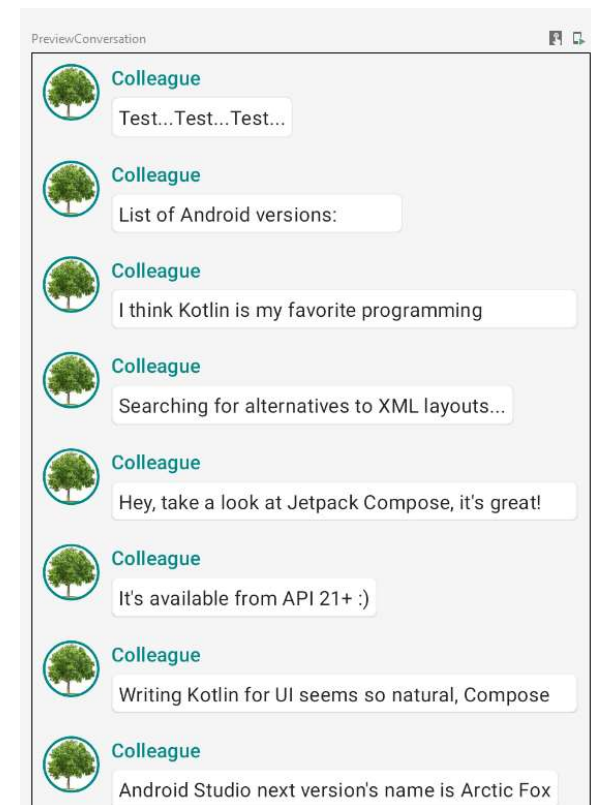
    Spacer(modifier = Modifier.height(4.dp))

    Surface(
        shape = MaterialTheme.shapes.medium,
        elevation = 1.dp,
        // surfaceColor color will be changing gradually from primary to surface
        color = surfaceColor,
        // animateContentSize will change the Surface size gradually
        modifier = Modifier.animateContentSize().padding(1.dp)
    ) {
        Text(
            text = msg.body,
            modifier = Modifier.padding(all = 4.dp),
            // If the message is expanded, we display all its content
            // otherwise we only display the first line
            maxLines = if (isExpanded) Int.MAX_VALUE else 1,
        )
    }
}
```

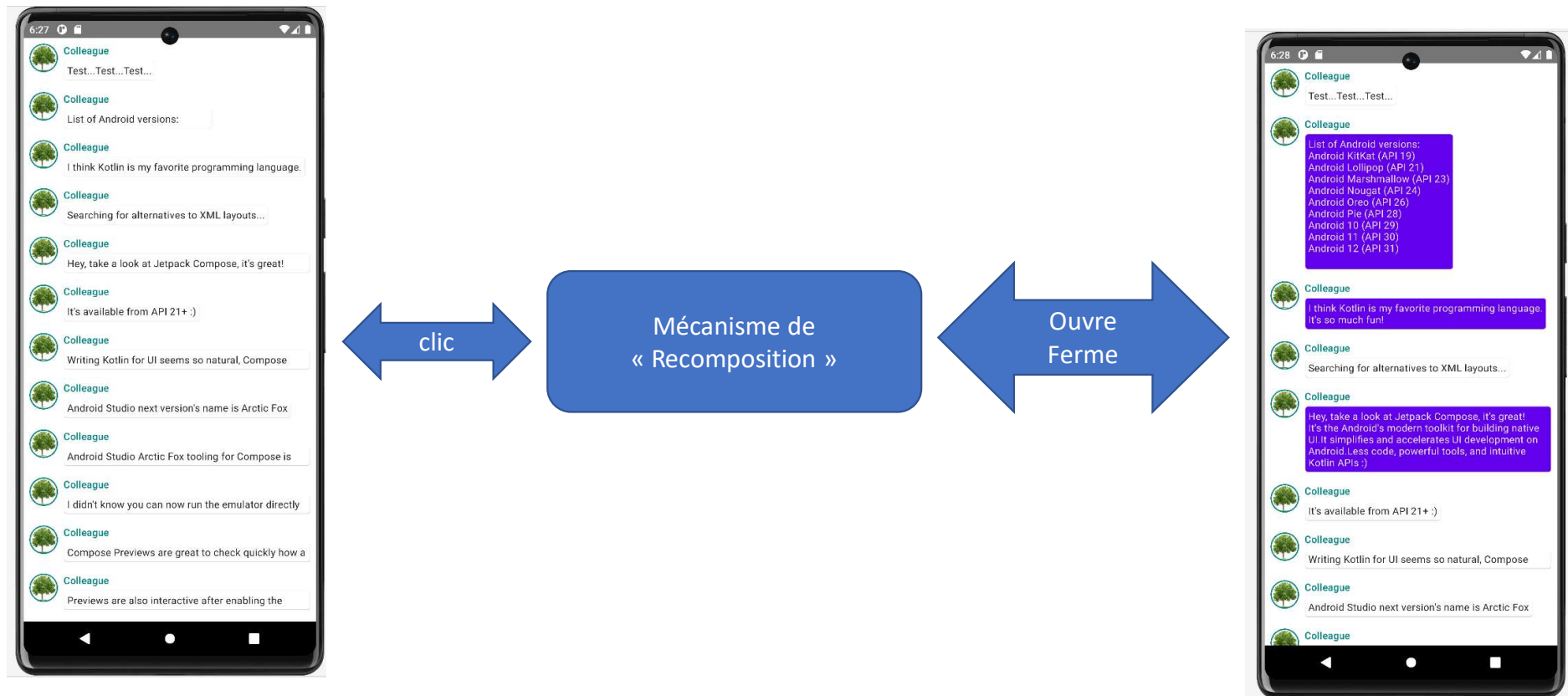
Gestion des interactions (ex. de la liste)

State → Mécanisme de
« Recomposition »

Détails sur remember et
autres, + loin



Gestion des interactions (ex. de la liste)



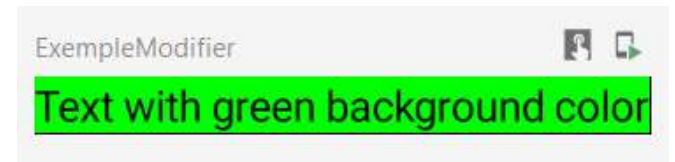
Modifier...

Modifier...

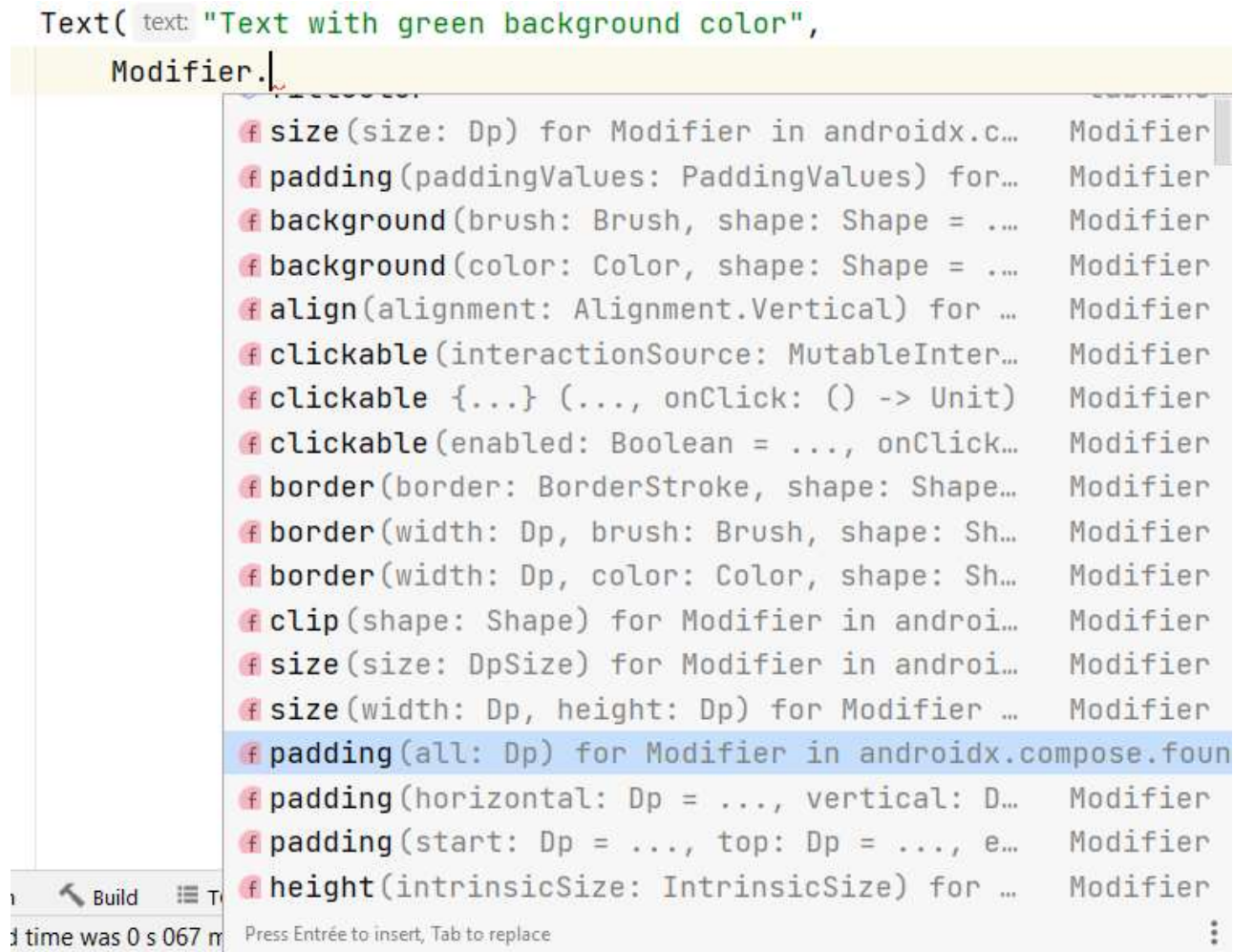
```
@Preview
@Composable
fun ExempleModifier() {
    Text("Je suis un texte")
}
```



```
@Preview
@Composable
fun ExempleModifier() {
    Text("Text with green background color",
        Modifier.background(color = Color.Green))
}
```



Beaucoup de propriétés de Modifier



Padding, Marging... (pas de Marging ☹)

@Composable

```
fun TextWidthPadding() {  
    Text(  
        "Je suis un texte avec un padding et une marge",  
        Modifier  
            .padding(32.dp) // Outer padding (margin)  
            .background(color = Color.Green) //background color  
            .padding(16.dp) // Inner padding  
    )  
}
```

Avant background, donc
« outer »

ExempleModifier

Text with green background color

Je suis un texte avec un padding et une marge

@Composable

```
fun TextWidthPadding() {  
    Text(  
        "Je suis un texte avec un padding et une marge",  
        Modifier  
            .background(color = Color.Green) //background color  
            .padding(16.dp) // Inner padding  
    )  
}
```

Après background, donc
« inner »

ExempleModifier

Text with green background color

Je suis un texte avec un padding et une marge

Padding, Weight...

@Composable

```
fun TextWidthPadding() {
```

```
    Text(
```

"Je suis un long texte avec un padding, qui essaie de prendre toute la largeur de l'écran. Pour le moment, je n'occupe que 50% de la largeur..."

Modifier

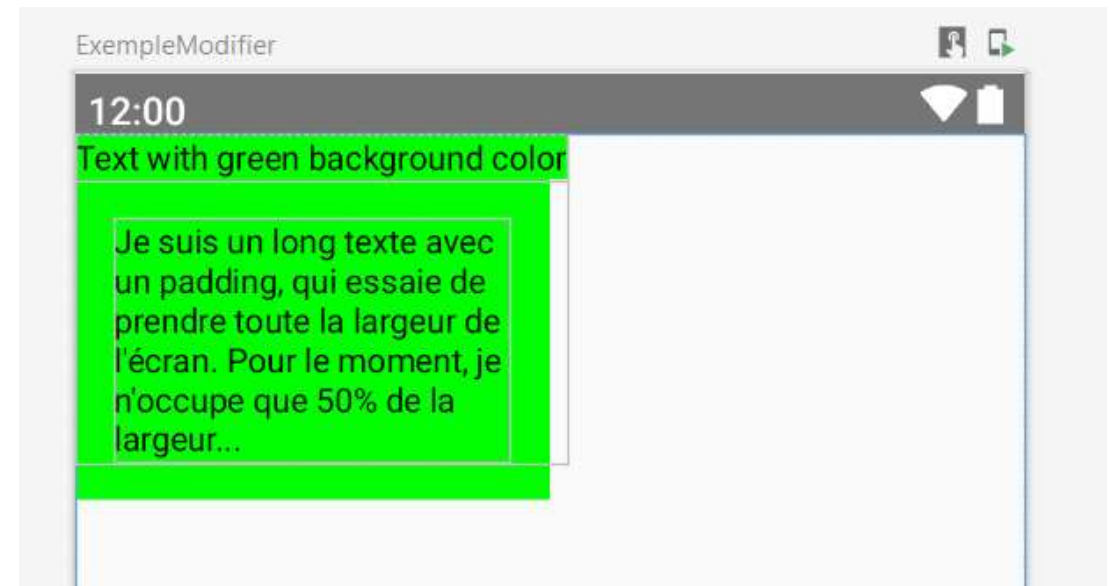
```
        .fillMaxWidth(0.5f)
```

```
        .background(color = Color.Green) //background color
```

```
        .padding(16.dp) // Inner padding
```

```
    )
```

```
}
```



Padding, Weight...

@Composable

```
fun TextWidthPadding() {
```

```
    Text(
```

"Je suis un long texte avec un padding, qui essaie de prendre toute la largeur de l'écran. Maintenant, j'occupe 100% de la largeur..."

Modifier

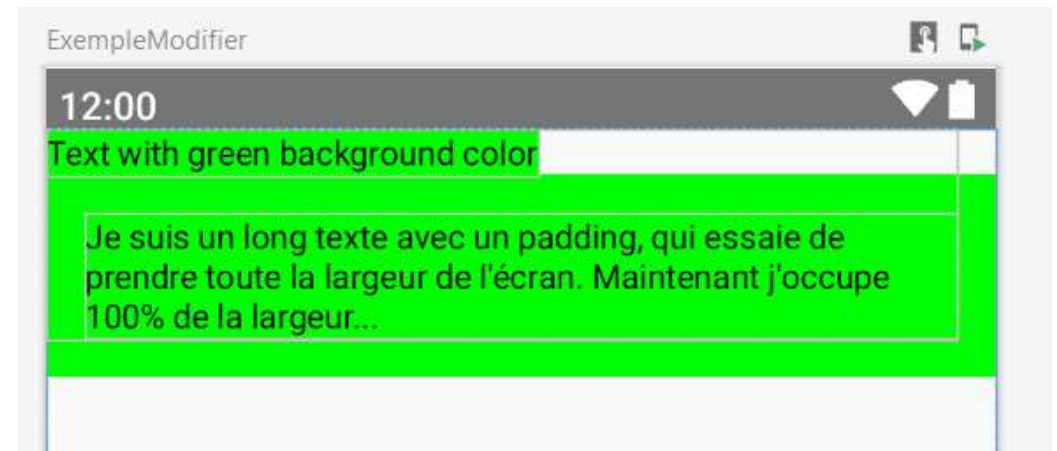
```
        .fillMaxWidth(1f)
```

```
        .background(color = Color.Green) //background color
```

```
        .padding(16.dp) // Inner padding
```

```
    )
```

```
}
```



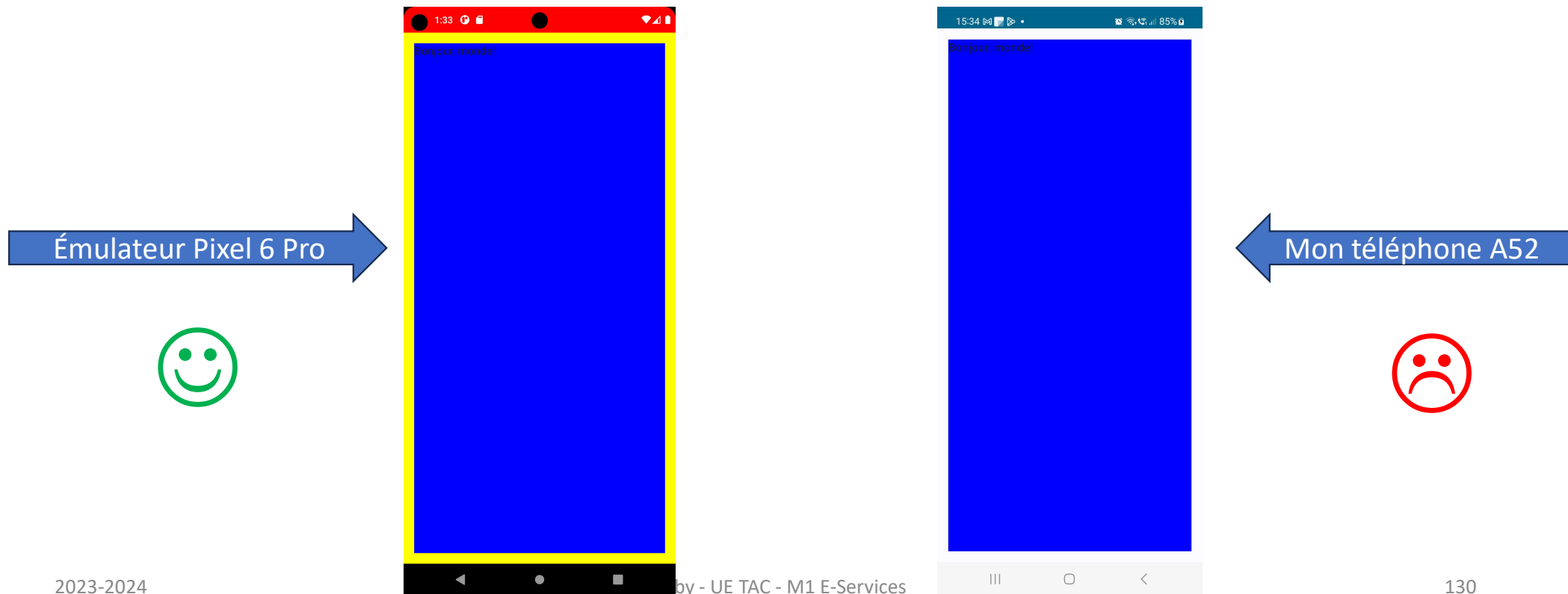
Petite pause pour pratiquer un peu...

TP Git, couleurs, et Text

- Récupérer mon projet de départ (avec GIT) à <https://gitlab.com/m1-ue-tac/compose/tp-de-demarrage.git>
- Testez-le sur votre téléphone (pb avec le mien, cf. slide d'après !)
- Puis créez votre branche (mettez votre nom par exemple)
- Puis modifier le code en ajoutant 3 Text en colonne, et changez les couleurs que j'ai moi-même modifiées
- Puis « poussez » votre code

TP Git, couleurs, et Text

- Le même code, mais pas le même résultat !



TP Git, couleurs, et Text

- Le même code, mais pas le même résultat !



On repart sur le TD...

Boutons

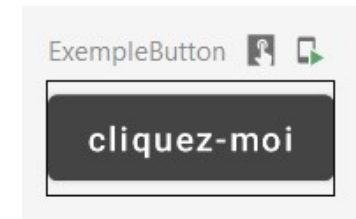
```
@Composable
fun ExempleButton() {
    Button(
        onClick =
            {
                //code de la gestion du clic
            }
    )
    { //code pour le contenu du bouton
        Text(text = "cliquez-moi")
    }
}
```



Boutons

@Composable

```
fun ExempleButton() {  
    Button(  
        onClick =  
        {  
            //code de la gestion du clic  
        },  
        colors = ButtonDefaults.buttonColors(backgroundColor = Color.DarkGray)  
    )  
    { //code pour le contenu du bouton  
        Text(text = "cliquez-moi", color = Color.White)  
    }  
}
```



Boutons

@Composable

```
fun ExempleButton() {  
    Button(  
        onClick =  
        {  
            //code de la gestion du clic  
        },  
        colors = ButtonDefaults.buttonColors(backgroundColor = Color.DarkGray)  
    ) {  
        Image(  
            painterResource(id = R.drawable.caddy),  
            contentDescription = "icône du panier",  
            modifier = Modifier.size(20.dp)  
        )  
  
        Text(text = "Ajouter au panier", Modifier.padding(start = 10.dp), color = Color.White)  
    }  
}
```



Boutons

@Composable

```
fun ExempleButton() {
```

```
    Button(
```

```
        onClick =
```

```
        {
```

```
            //code de la gestion du clic
```

```
        },
```

```
        colors = ButtonDefaults.buttonColors(backgroundColor = Color.DarkGray),
```

```
        shape = RoundedCornerShape(20.dp)
```

```
    ) {
```

```
        Image(
```

```
            painterResource(id = R.drawable.caddy),
```

```
            contentDescription = "icône du panier",
```

```
            modifier = Modifier.size(20.dp)
```

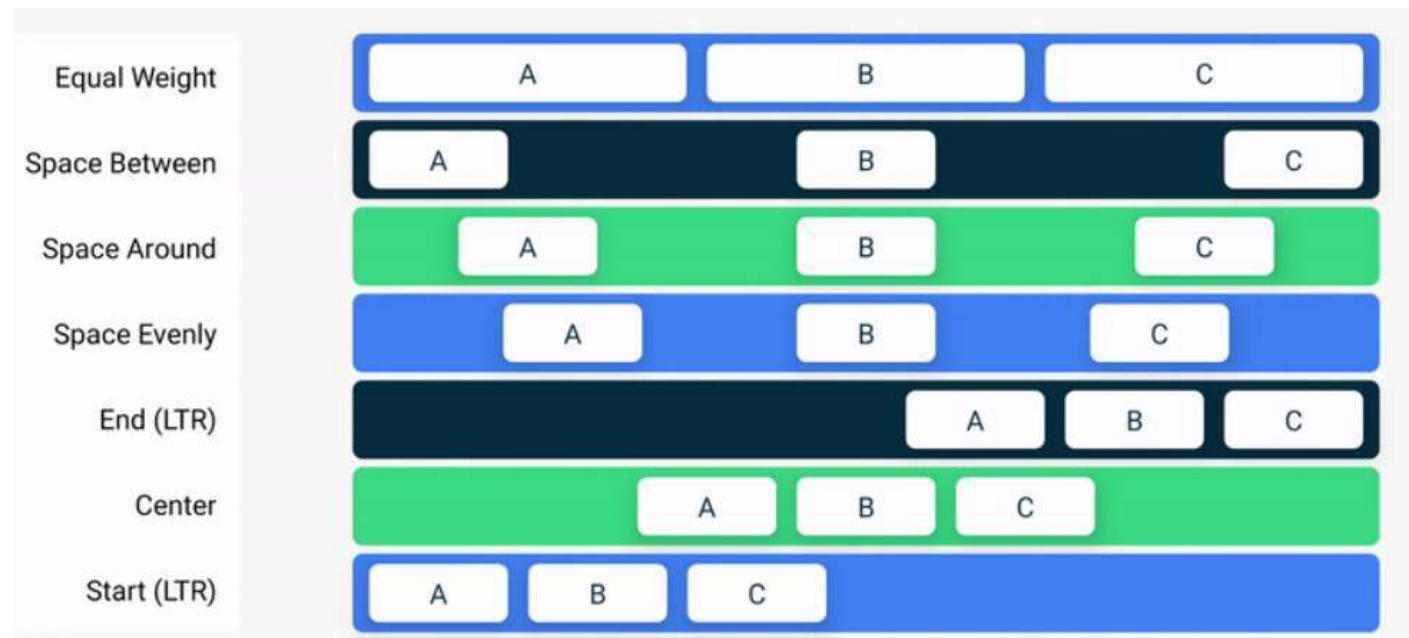
```
        )
```

```
        Text(text = "Ajouter au panier", Modifier.padding(start = 10.dp), color = Color.White)
```



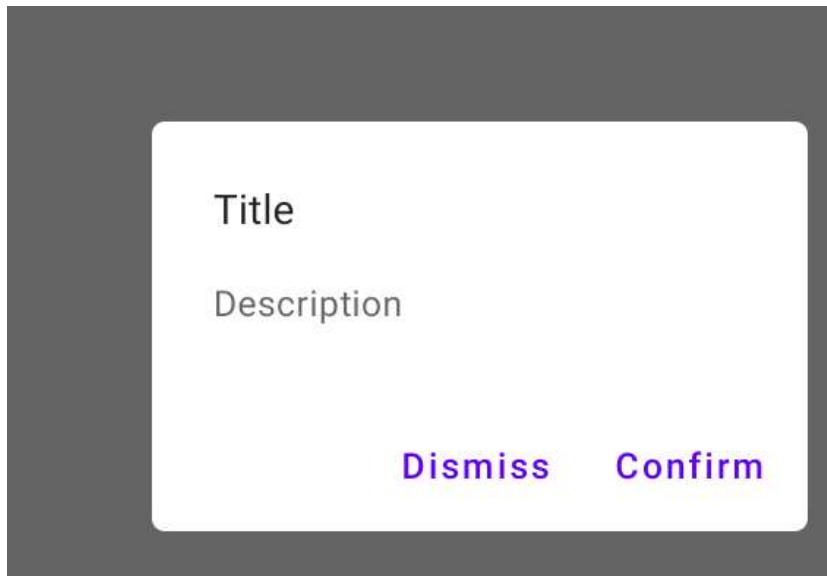
Aligner des objets (par exemple des boutons)

- TB explications (animées !) à <https://developer.android.com/reference/kotlin/androidx/compose/foundation/layout/Arrangement>



Autre explication à <https://semicolonspace.com/jetpack-compose-alignment-arrangement/>

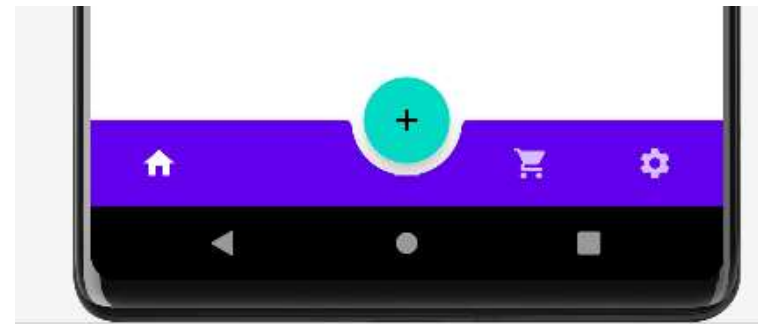
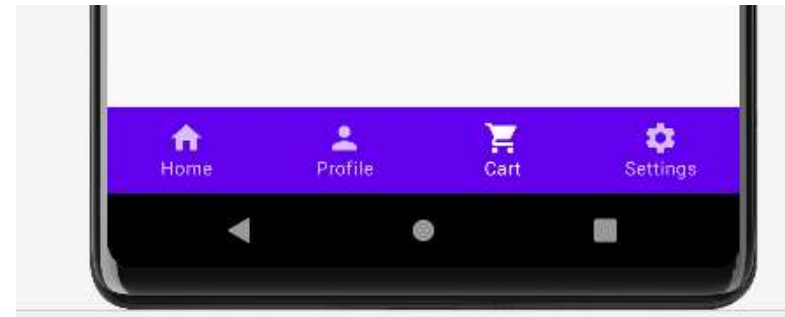
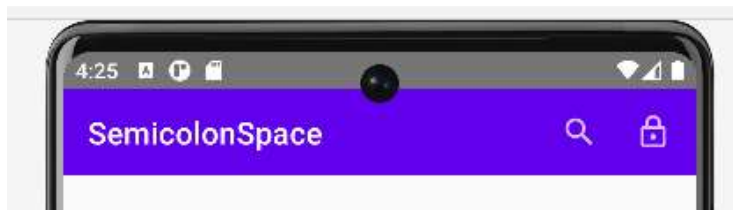
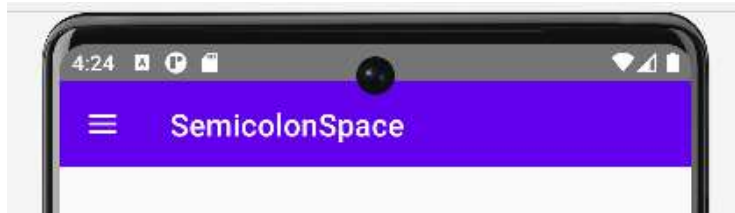
Boite de dialogue



```
if (dialogOpen) {  
    AlertDialog(  
        onDismissRequest: {  
            // @Composable  
            // @ComposableInferredTarget  
            public fun AlertDialog(  
                onDismissRequest: () -> Unit,  
                confirmButton: @Composable () -> Unit,  
                modifier: Modifier,  
                dismissButton: @Composable() (() -> Unit)?,  
                title: @Composable() (() -> Unit)?,  
                text: @Composable() (() -> Unit)?,  
                shape: Shape,  
                backgroundColor: Color,  
                contentColor: Color,  
                properties: DialogProperties  
            ): Unit  
        },  
        confirmButton: {  
            // Material Design alert dialog.  
            // Alert dialogs interrupt users with urgent information, details, or actions.  
            !Dialogs image  
            The dialog will position its buttons based on the available space. By default it will  
            try to place them horizontally next to each other and fallback to horizontal  
            placement if not enough space is available. There is also another version of this  
            composable that has a slot for buttons to provide custom buttons layout.  
            Sample of dialog:  
        },  
        dismissButton: {  
            //  
            T  
        }  
    )  
}
```

TopBar, NavBar...

- Cf. mon exemple sur Gitlab (+ loin)



TopBar, NavBar...Navigation... *Scaffold*

Composant Scaffold

@Composable

fun Scaffold(

modifier:

...

topBar:

bottomBar:

snackbarHost:

floatingActionButton:

...

drawerContent:

...

Tous des
@Composables

TopBar, NavBar...Navigation... *Scaffold*

Composant Scaffold

@Composable

fun Scaffold(

modifier:

...

topBar:

bottomBar:

snackbarHost:

floatingActionButton:

...

~~drawerContent:~~

...

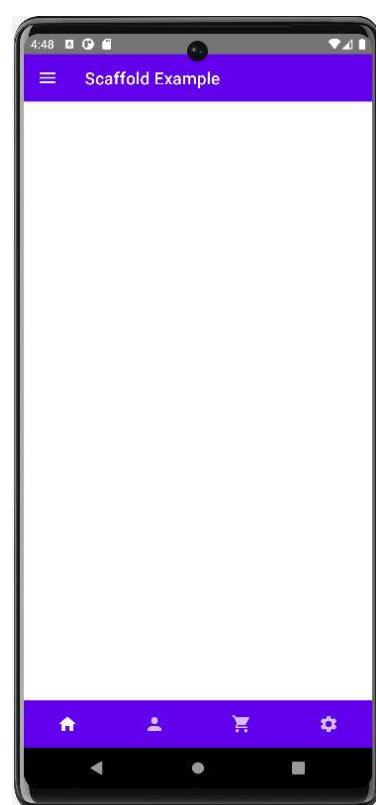
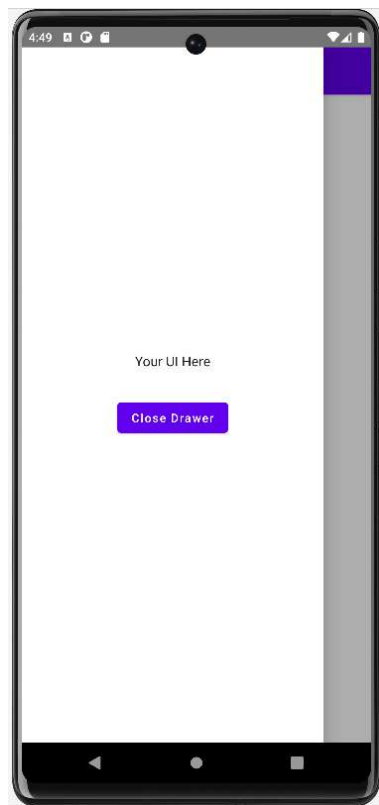
Tous des
@Composables

Encore des changements !
Maintenant en dehors du
scaffold...

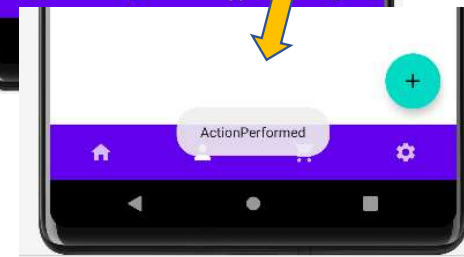
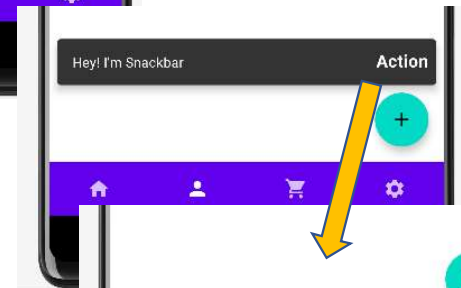
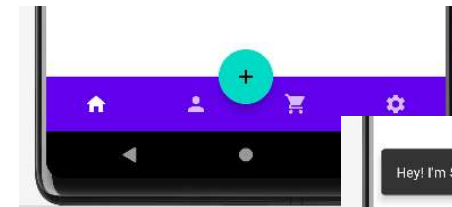
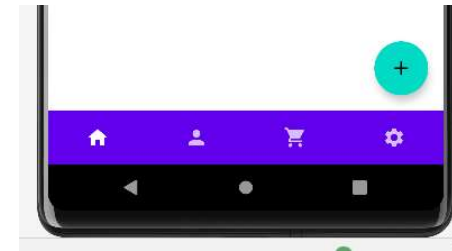
TopBar, NavBar...Navigation... *Scaffold*



2023-2024



JC Tarby - UE TAC - M1 E-Services



142

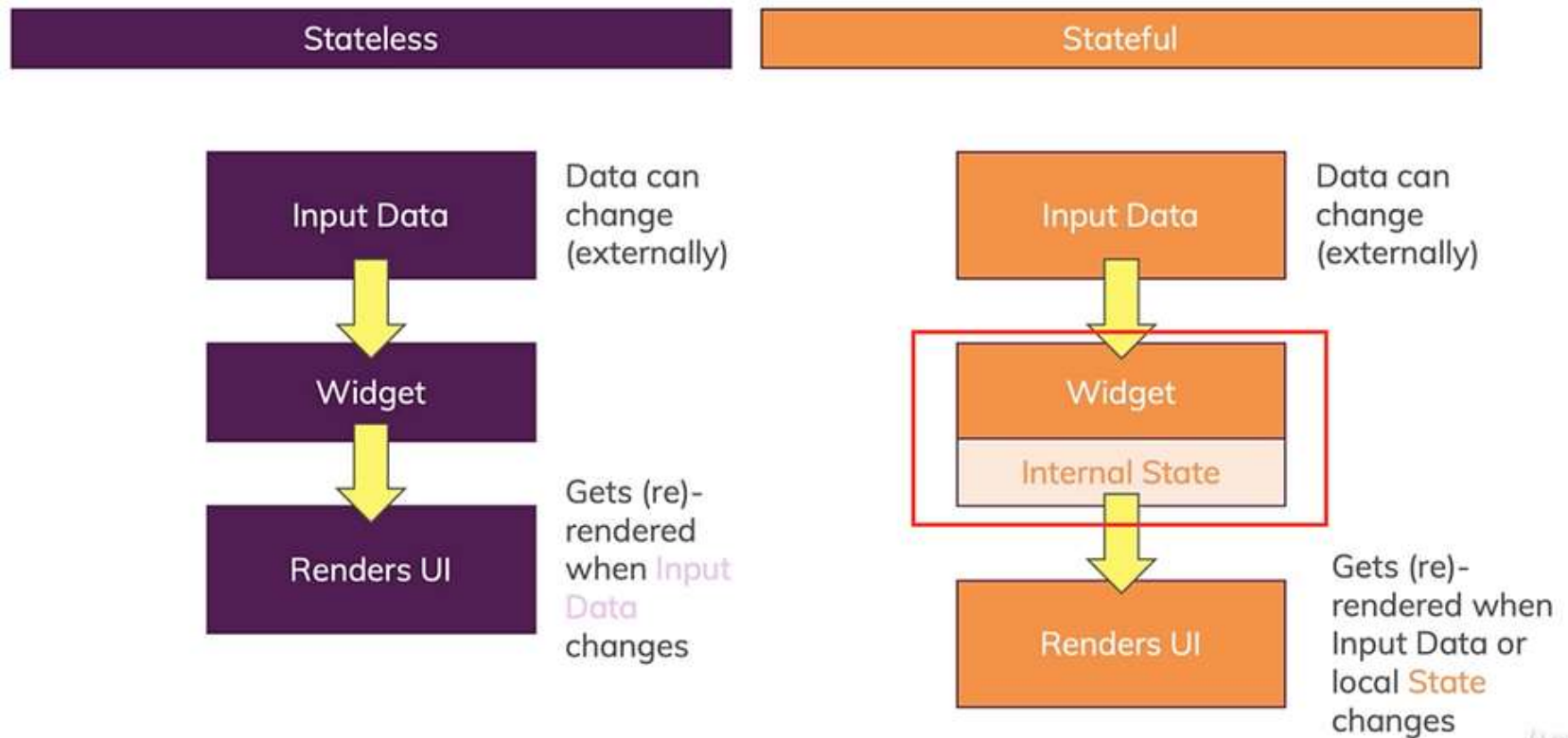
TopBar, NavBar...Navigation... *Scaffold*



State...

Composants Statefull et Stateless

Stateless vs Stateful



Composant stateless

@Composable

```
fun UserDetails() {  
    // This is the state  
    var name by remember { mutableStateOf("") }  
    var lastname by remember { mutableStateOf("") }  
    var email by remember { mutableStateOf("") }  
}
```

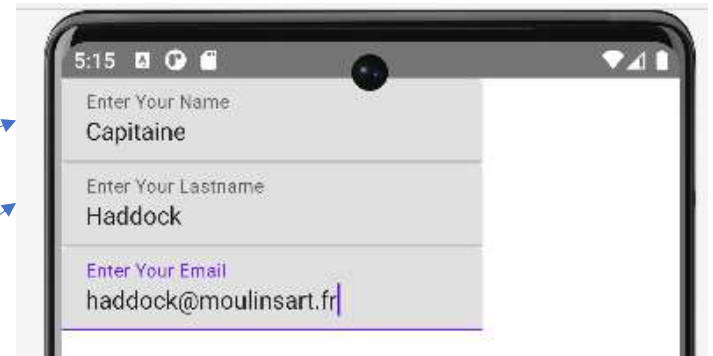
Column {

```
    TextFieldExample(  
        name = name,  
        onValueChange = { name = it },  
        label = "Enter Your Name"  
    )  
}
```

```
    TextFieldExample(  
        name = lastname,  
        onValueChange = { lastname = it },  
        label = "Enter Your Lastname"  
    )  
}
```

...

```
}
```



@Composable

```
fun TextFieldExample(  
    name: String,  
    onValueChange: (String) -> Unit,  
    label: String  
) {  
    TextField(  
        value = name,  
        onValueChange = onValueChange,  
        label = { Text(text = label) }  
    )  
}
```

Composant stateless →
réutilisable

State, MutableState

- **State<T>**

- read-only
- Si valeur change ➔ recomposition

- **MutableState<T>**

- La valeur peut être modifiée
- Si affectation/mise à jour de la valeur ➔ recomposition
- Si affectation/mise à jour de la valeur, SANS changement de valeur ➔ rien

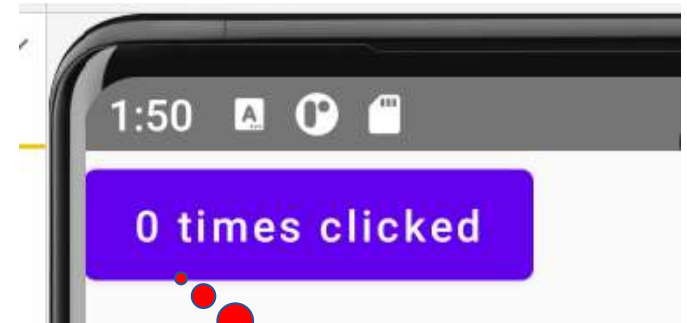
State, MutableState

But : créer un bouton qui affiche combien de fois on a cliqué sur lui.

```
@Composable
fun ExempleState() {
    var clickCount = 0
    Column {
        Button(onClick = {
            clickCount++
            Log.d("JC", "NoState: " + clickCount)
        }) {
            Text(text = "" + clickCount + " times clicked")
        }
    }
}
```

Le compteur est bien
mis à jour

On veut utiliser le
compteur...



...mais le bouton ne
se met pas à jour !

State, MutableState

```
@Composable
fun ExempleState() {
    var clickCount = 0
    Column {
        Button(onClick = {
            clickCount++
            Log.d("JC", "click")
        }) {
            Text(text = "Click $clickCount")
        }
    }
}
```

POURQUOI ?

State, MutableState

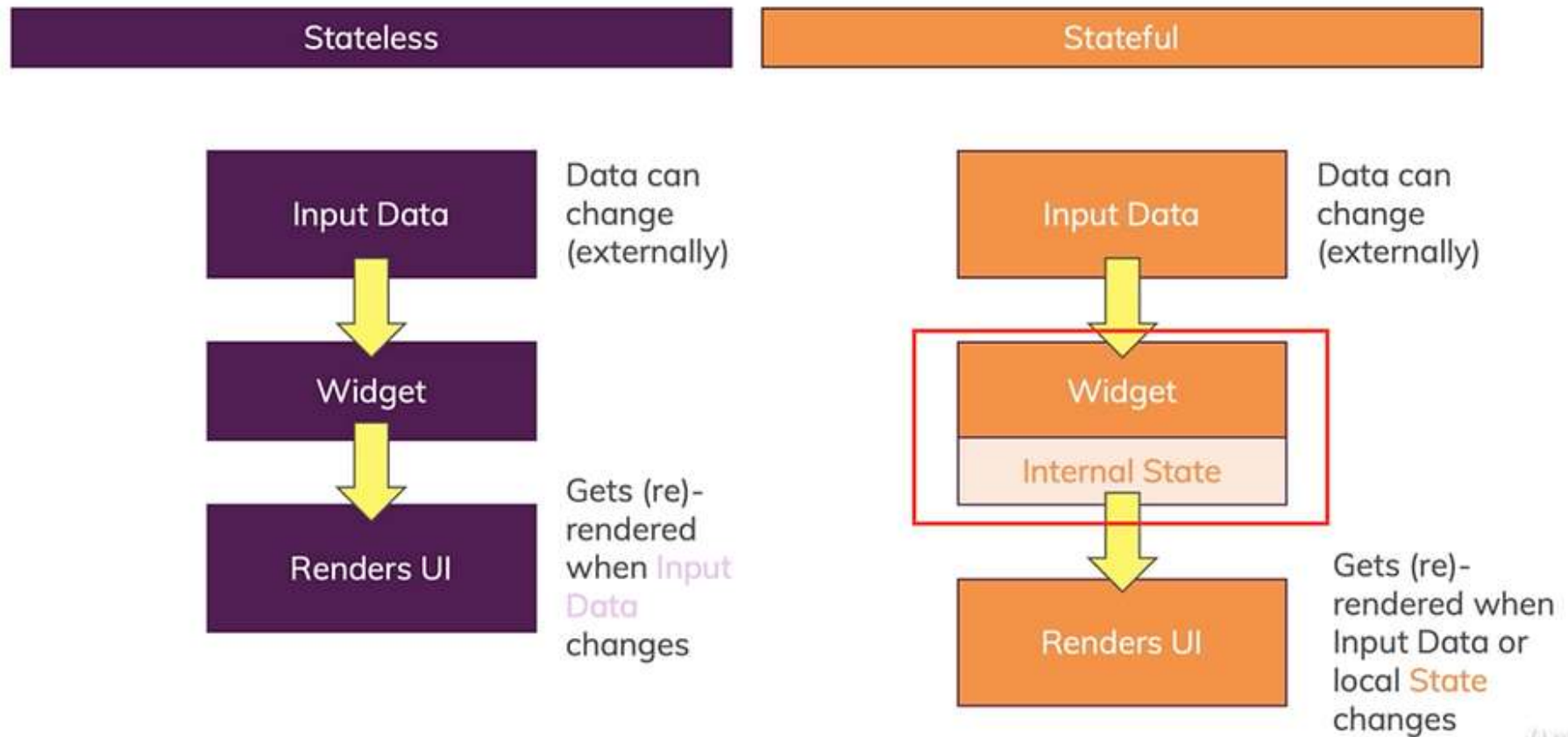
@Composable

fun ExampleState() {

Pour mettre à jour le bouton, l'IHM doit être mise à jour → états et changements d'états !
(State, MutableState)

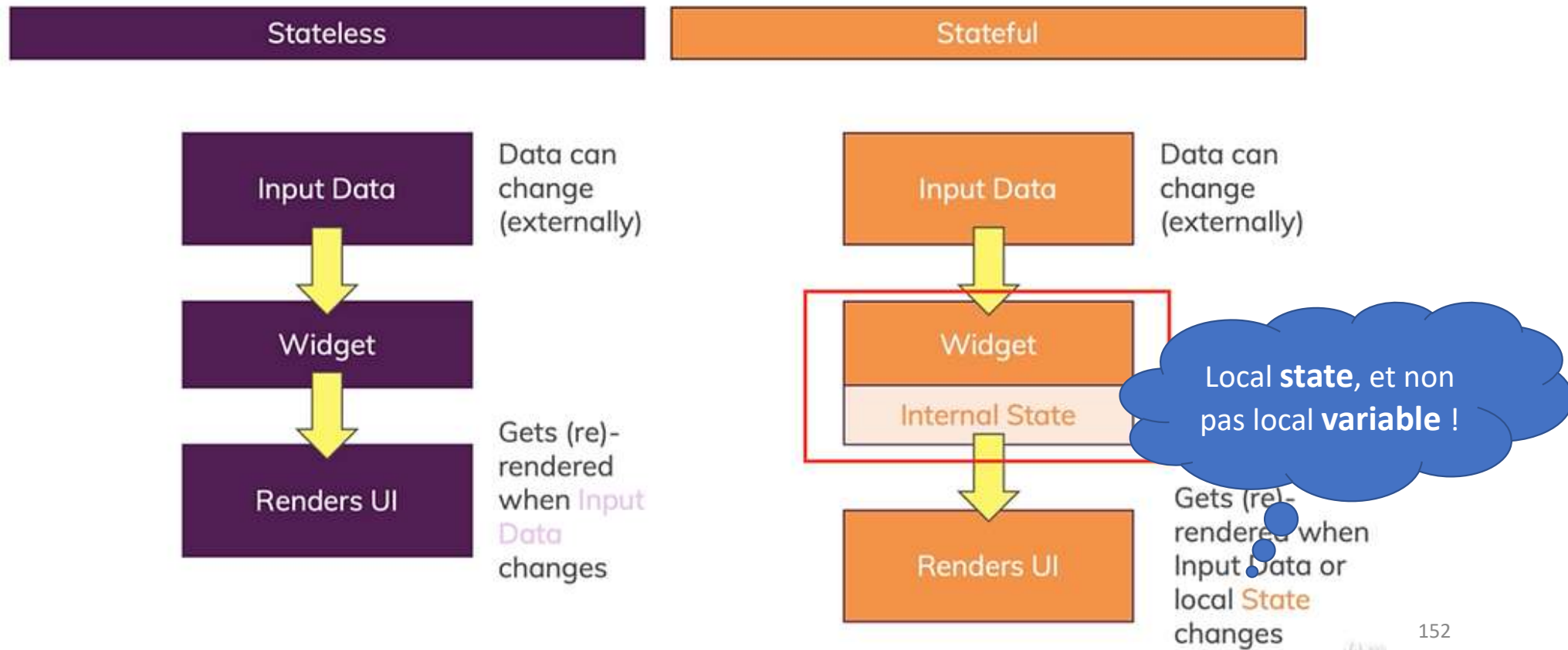
Rappel...

Stateless vs Stateful



Rappel...

Stateless vs Stateful



State, MutableState...



The screenshot shows a Kotlin Compose function `ExempleState()` with a warning from the IDE. A blue thought bubble above the warning says "Mauvaise solution, et détectée par IDE". The code defines a `clickCount` variable using `mutableStateOf(0)`, which is underlined in red. The function also contains a `Column` with a `Button` that increments `clickCount` and a `Text` widget that displays the count.

```
@Preview
@Composable
fun ExempleState() {
    var clickCount by mutableStateOf(0) //mauvaise solution
    Column { this: ColumnScope
        Button(onClick = {
            clickCount++
            Log.d( tag: "JC", msg:
        }) { this: RowScope
            Text(text = "" + cli
        }
    }
}
```

Creating a state object during composition without using remember
Provide feedback on this warning Alt+Maj+Entrée More actions... Alt+Entrée

```
public fun <T> mutableStateOf(
    value: T,
    policy: SnapshotMutationPolicy<T>
): MutableState<T>
```

Return a new `MutableState` initialized with the passed in `value`
The `MutableState` class is a single value holder whose reads and writes are observed by Compose. Additionally, writes to it are transacted as part of the `Snapshot` system.

Params: `value` - the initial value for the `MutableState`
`policy` - a policy to controls how changes are handled in mutable snapshots.

See Also: `State`,

State, MutableState...

Si vous utilisez un State dans un Composable, vous devez utiliser **remember**

Mauvaise solution, et détectée par IDE

```
@Preview
@Composable
fun ExampleState() {
    var clickCount by mutableStateOf( value: 0) //mauvaise solution
    Column { this: ColumnScope
        Button(onClick = {
            clickCount++
            Log.d( tag: "JC", msg:
        }) { this: RowScope
            Text(text = "" + cli
        }
    }
}
```

Creating a state object during composition without using remember
Provide feedback on this warning Alt+Maj+Entrée More actions... Alt+Entrée

```
public fun <T> mutableStateOf(
    value: T,
    policy: SnapshotMutationPolicy<T>
): MutableState<T>
```

Return a new **MutableState** initialized with the passed in **value**
The **MutableState** class is a single value holder whose reads and writes are observed by Compose. Additionally, writes to it are transacted as part of the **Snapshot** system.

Params: **value** - the initial value for the **MutableState**
policy - a policy to controls how changes are handled in mutable snapshots.

See Also: [State](#),

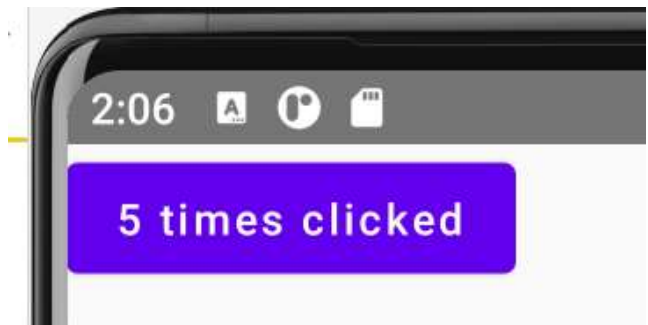
State, MutableState...

- “remember” : permet de stocker un objet qui change de valeur (ou pas).
 - Le composant se “rappelle” de sa valeur
 - Si la valeur change, cela déclenchera automatiquement une recomposition des widgets associés (rafraîchissement de l’IHM)

State, MutableState...

État « mutable » de type Int

```
@Composable
fun ExempleState() {
    var clickCount by remember { mutableStateOf(0) }
    Column {
        Button(onClick = {
            clickCount++
            Log.d("JC", "NoState: " + clickCount)
        }) {
            Text(text = "" + clickCount + " times clicked")
        }
    }
}
```



State, MutableState...

```
@Composable
fun ExempleState() {
    var clickCount by remember { mutableStateOf(0) }
    Column {
        Button(onClick = {
            clickCount++
            Log.d("JC", "clickCount: $clickCount")
        }) {
            Text(text = "clickCount: $clickCount")
        }
    }
}
```

Problème !
Si on tourne le téléphone, le
compteur se réinitialise...

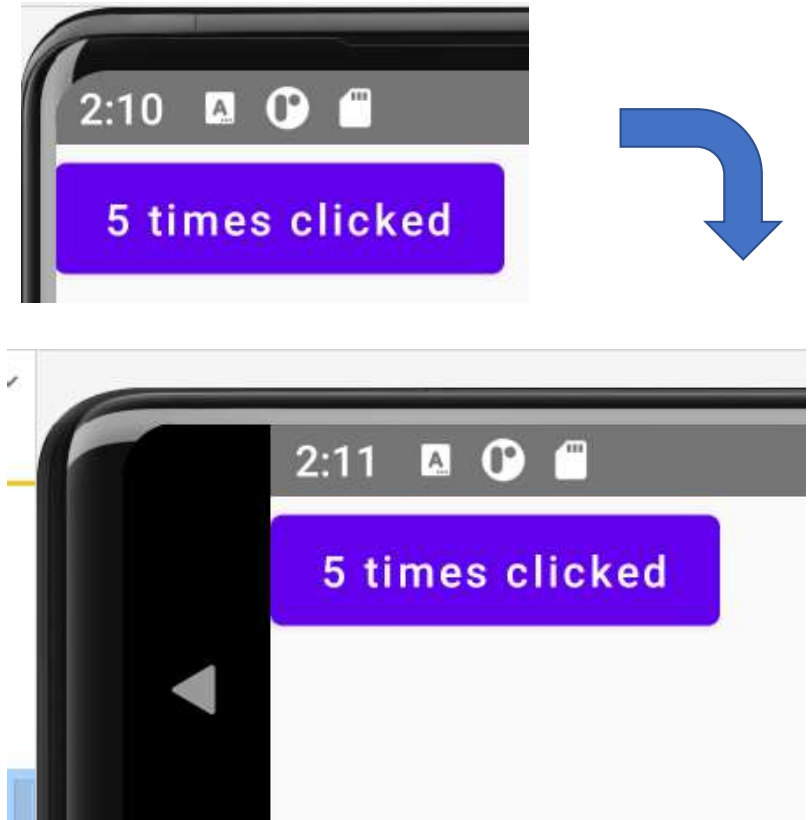
State, MutableState...

```
@Composable
fun ExempleState() {
    var clickCount by remember { mutableStateOf(0) }
    Column {
        Button(onClick = {
            clickCount++
            Log.d("JC", "clickCount: $clickCount")
        }) {
            Text(text = "Click $clickCount times")
        }
    }
}
```

Solution : rememberSaveable

rememberSaveable

```
@Composable
fun ExampleState() {
    var clickCount by rememberSaveable { mutableStateOf(0) }
    Column {
        Button(onClick = {
            clickCount++
            Log.d("JC", "NoState: " + clickCount)
        }) {
            Text(text = "" + clickCount + " times clicked")
        }
    }
}
```



« by » ou « = », c'est (presque) pareil

« **by** » convertit le mutableState dans le type de la variable associé

« = » ne fait pas la conversion ➔ **.value** obligatoire

```
var clickCount: Int
```

```
var clickCount by remember { mutableStateOf(0) }  
Column {  
    Button(onClick = {  
        clickCount++  
    }) {  
        Text(text = "" + clickCount + " times clicked")  
    }  
}
```

Valeur « int »
directement

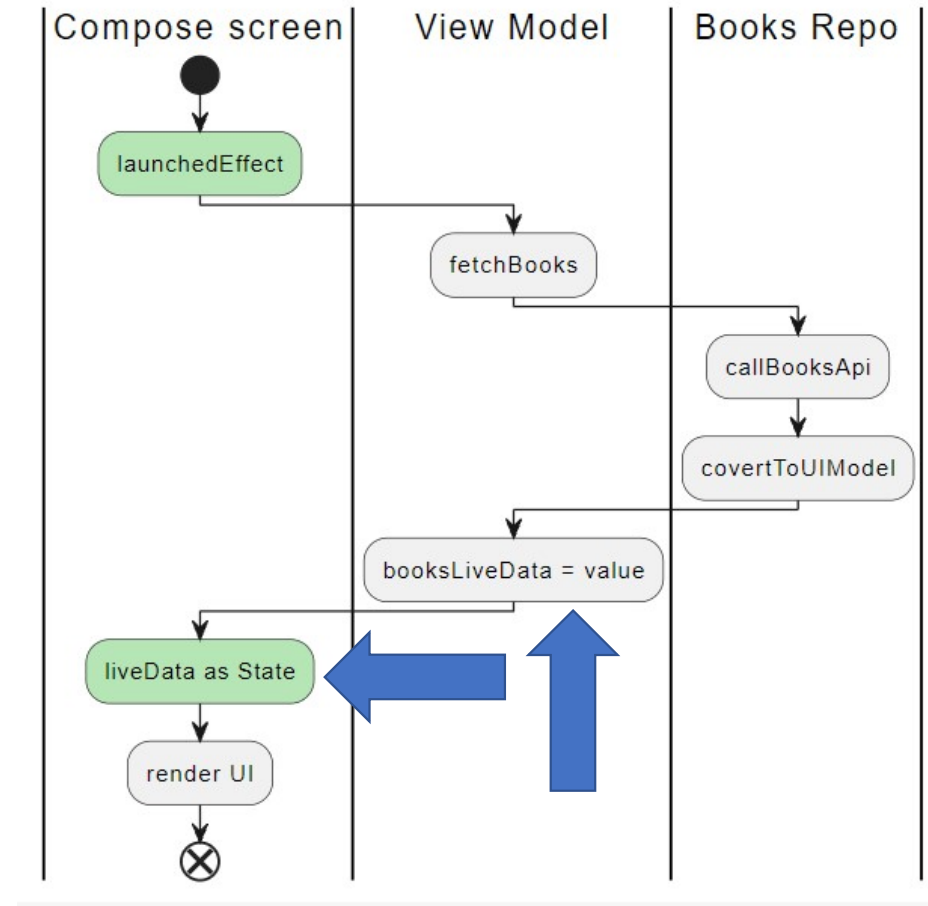
```
var clickCount: MutableState<Int>
```

```
var clickCount = remember { mutableStateOf(0) }  
Column {  
    Button(onClick = {  
        clickCount++  
    }) {  
        Text(text = "" + clickCount.value + " times clicked")  
    }  
}
```

.value
obligatoire

State et LiveData (pour info)

- On peut utiliser des LiveData comme des State
- Exemple :
 - var maVariable by
myViewModel.maData.observeAsState(0)
- Pour rappel, Kotlin fournit des « Flow », « StateFlow » et « SharedFlow », proches et différents à la fois des LiveData
 - <https://developer.android.com/kotlin/flow>
 - On en reparlera bientôt (Room, API...)



State et LiveData (pour in

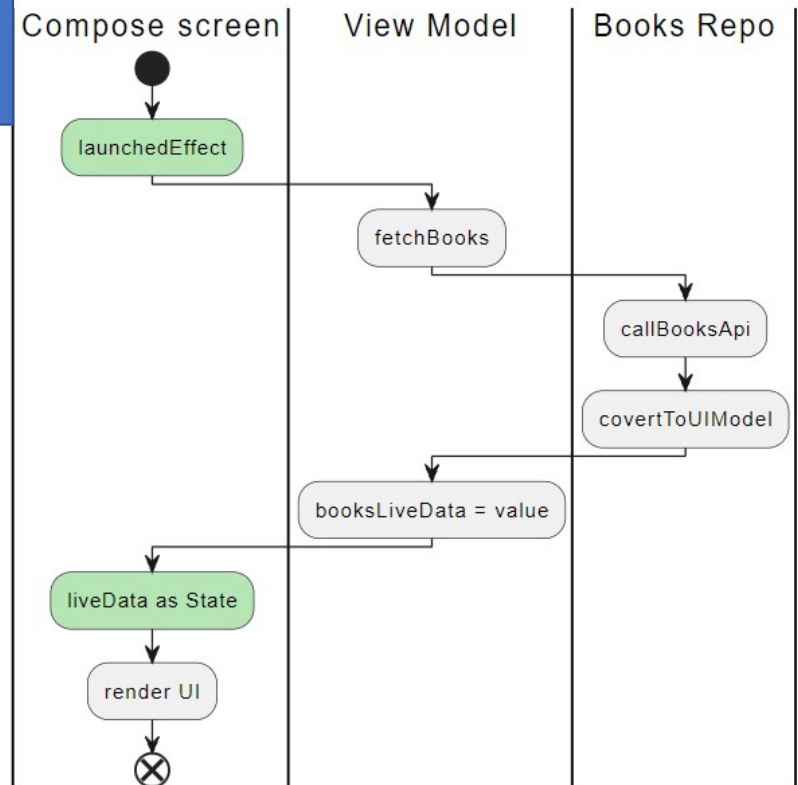
Pensez à mettre à jour le gradle :

implementation "androidx.compose.runtime:runtime-livedata:\$compose_version"

```
@Composable
fun BooksScreen(
    viewModel: BookListViewModel = ...
){
    // State
    val books = viewModel.books.observeAsState()
    ...
}

// UI
Column() {
    ...
    LazyColumn(modifier = modifier) {
        items(books) {
            // List item composable
            BookListItem(book = it)
        }
    }
}
```

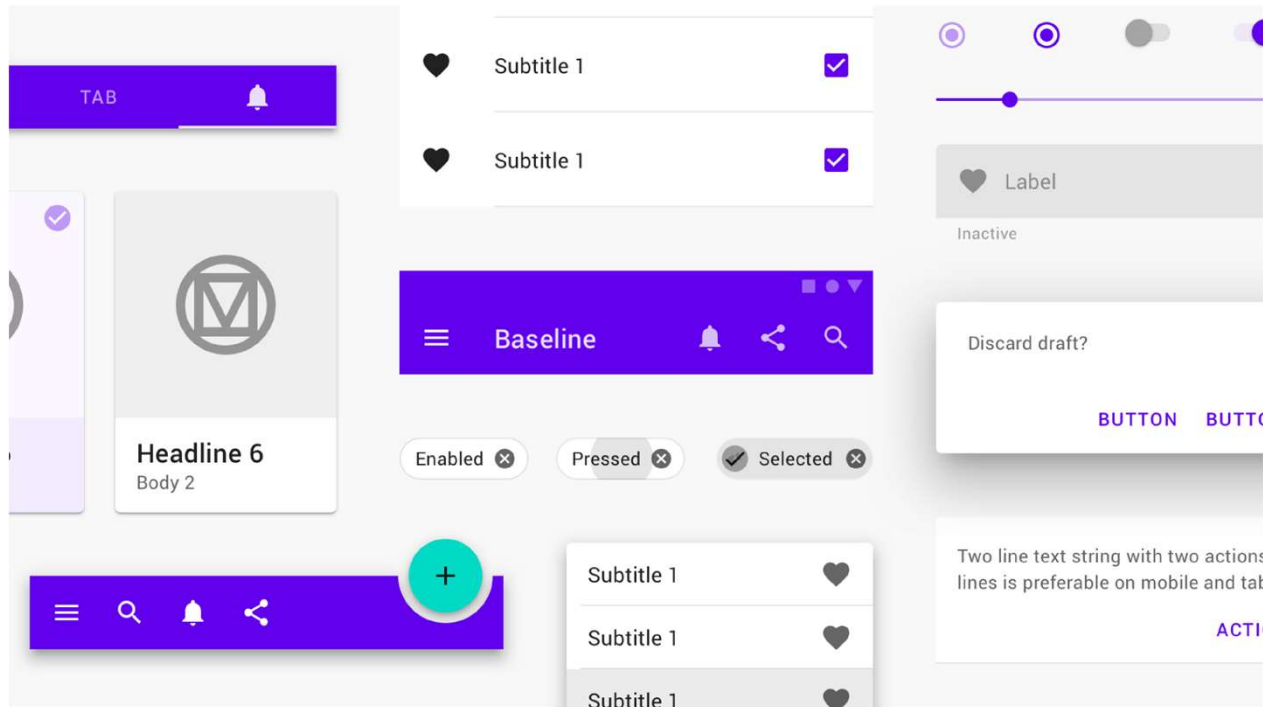
Les données ont pu arriver par Rx par exemple



Source: <https://dev.to/mahendranv/using-viewmodel-livedata-with-jetpack-compose-31h8>

Autres composants IHM

- Cv. <https://developer.android.com/jetpack/compose/layouts/material>



Tutoriels utilisés pour ce cours

- Officiel

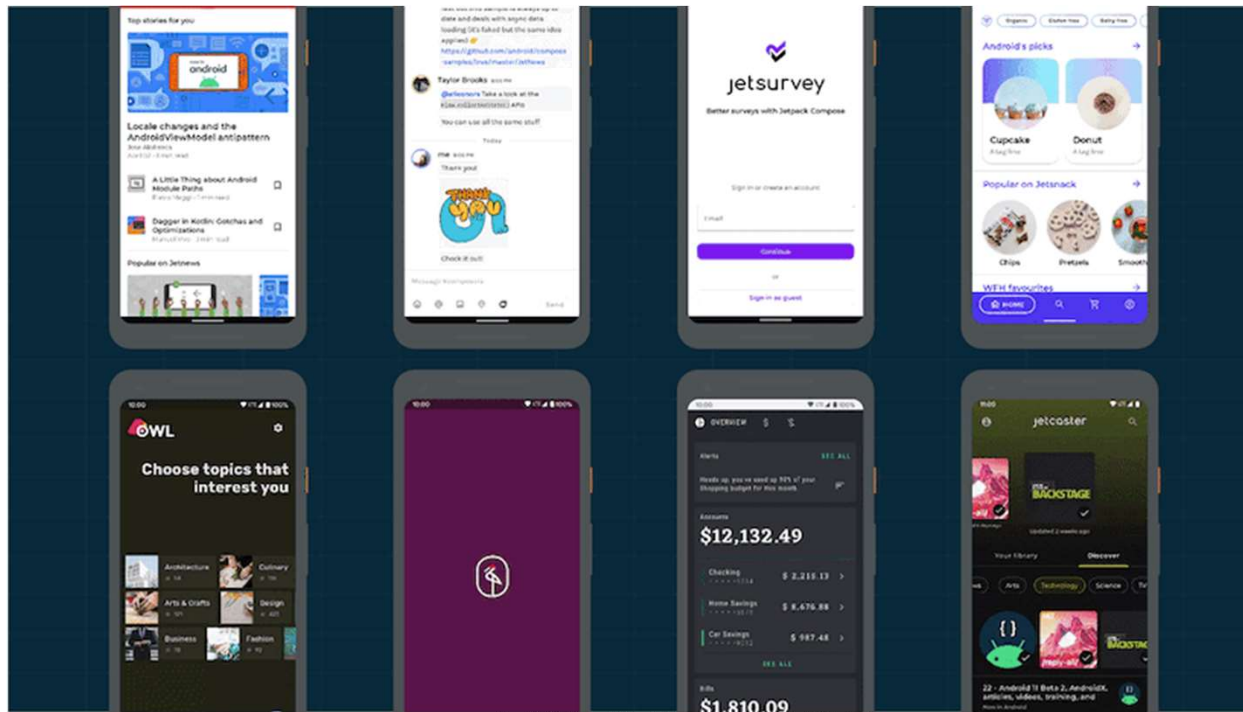
- <https://developer.android.com/jetpack/compose/tutorial>
- <https://developer.android.com/jetpack/compose/documentation>

- Autre

- <https://semicolonspace.com/jetpack-compose-tutorial>
- <https://www.jetpackcompose.net/>

Des exemples officiels à voir/consulter

- <https://github.com/android/compose-samples>
- Mis à jour régulièrement



Les tutoriels officiels

- <https://developer.android.com/courses/pathways/compose?hl=fr>

The screenshot shows the 'Jetpack Compose' tutorial page. At the top, it says 'Découvrez Compose, un kit d'outils moderne permettant de créer une interface utilisateur Android native.' and '13 activités • 1 questionnaire'. A progress card on the right shows 'Progression : 0 %' and a 'Sign in' button. Below, a list of four tutorial steps is shown, each with a number, title, type, and a dropdown arrow.

Numéro	Titre	Type	Statut
1	Tutoriel : principes de base de Jetpack Compose	Article	Activité facultative
2	Nouveautés de Jetpack Compose	Vidéo	Activité facultative
3	Raisonnement dans Compose	Article	Activité facultative
4	Principes de base de Jetpack Compose		

Passons à la pratique !

Créer des composants simples, faire afficher dans téléphone/émulateur

1. Créer un composant Text basique et lancer l'appli dans émulateur/téléphone
2. Ajouter une preview (ou des previews, par exemple avec image du téléphone, mode jour/nuit...)
3. Modifier le Text pour que la couleur de fond change par exemple, ou la police de caractères, etc.
4. Ajouter une autre composant Text
5. Ajouter un espace entre les Text
6. Si pas encore fait, passez des paramètres aux Texts

Exemple

@Composable

```
fun Greeting(name: String, ue: String, modifier: Modifier = Modifier) {
```

```
    Text(  
        text = "        ",  
        modifier =
```

```
)
```

```
    Text(  
        text = "        ",  
        fontSize = ,  
        modifier =
```

```
)
```

```
}
```

```
}
```

Bonjour JC

Bienvenue dans TAC, la
meilleure UE du monde !

Exercice

- Un bouton qui compte le nb de clics et qui s'invalide si compteur > 5 + un texte qui apparaît si le compteur > 0
 - Si vous bloquez... cf. <https://developer.android.com/codelabs/jetpack-compose-state?hl=fr#5>

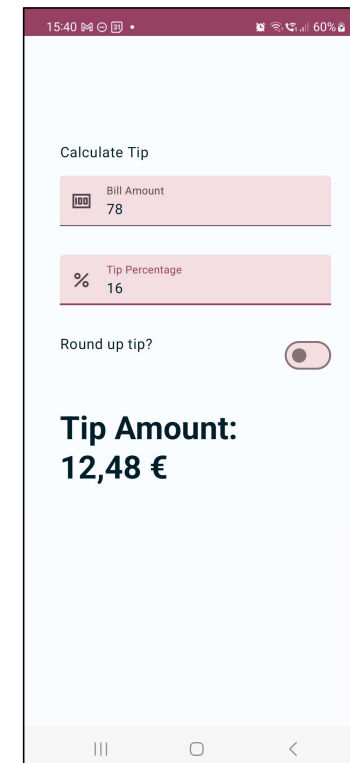


Exercice: Text stateless

- 3 champs textes sans état interne ➔ donc réutilisable

Exercice : calcul de pourboire

- Faites le tutoriel à <https://developer.android.com/codelabs/basic-android-compose-calculate-tip?hl=fr#0>



Exercice : calcul de pourboire

Information dans le tuto pour définir le clavier

4. Définir un bouton d'action

Dans l'atelier de programmation précédent, vous avez appris à utiliser la classe `KeyboardOptions` pour définir le type de clavier. Dans cette section, vous allez découvrir comment définir le bouton d'action du clavier avec la même classe, `KeyboardOptions`. Un bouton d'action est un bouton situé à l'extrémité du clavier. Voici quelques exemples dans ce tableau :

Propriété	Bouton d'action du clavier
<code>ImeAction.Search</code> Option possible lorsque l'utilisateur souhaite effectuer une recherche.	
<code>ImeAction.Send</code> Option possible lorsque l'utilisateur souhaite envoyer le texte dans le champ de saisie.	
<code>ImeAction.Go</code> Option possible lorsque l'utilisateur souhaite accéder à la cible du texte dans l'entrée.	

[Retour](#)

[Suivant](#)

Exercice : Créer un formulaire avec clavier adapté

- Mise en majuscule du nom pendant la saisie
- Initiale majuscule du prénom pendant la saisie
- Age sur 3 digits max
- Password ****
- Carte de crédit : xxxx*****
- A faire incrémentalement : IHM de base, puis jolie, puis comportement

The screenshot shows a mobile application interface with a purple header bar displaying the time 2:38 and various status icons. The form consists of several input fields: 'Nom' with the value 'HADDOCK', 'Prénom' with the value 'Capitaine', 'Mot de passe' with masked characters '*****', 'Âge' with the value '123', and 'Numéro de carte' with the value '9876*****'. Below the input fields are three buttons: 'Valider', 'Annuler', and 'Aide'. At the bottom of the screen is a numeric keypad with digits 1-9, 0, a decimal point, a minus sign, a comma, and a backspace key. The keypad is styled with a light blue background and dark text.

Exemples à regarder, et « jouer » avec...

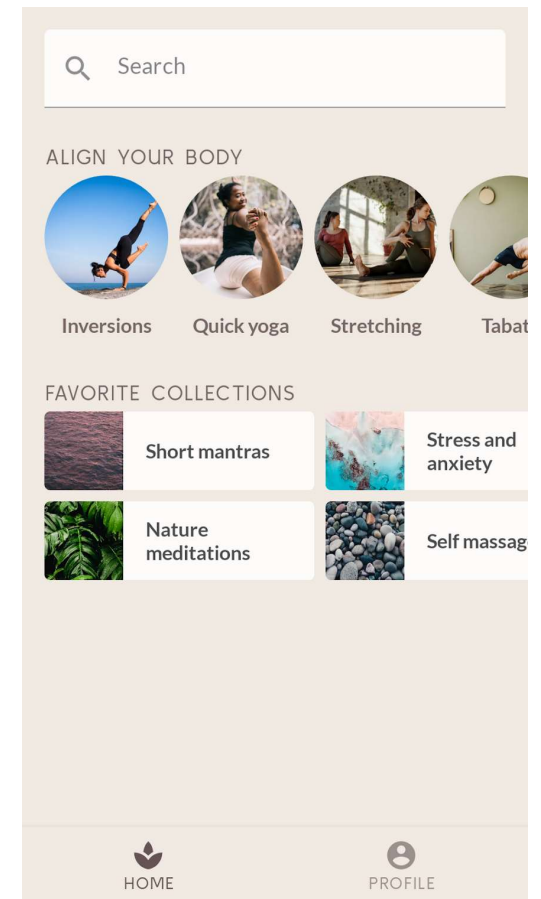
- Boutons, Navigation, State...
 - <https://gitlab.com/m1-ue-tac/compose/introduction-compose>
 - Attention, des changements dans Kotlin/Compose depuis ça... MAJ à faire !
 - Pour le moment, regardez-le plutôt pour avoir des pistes pour + tard

Exercices de groupe 😊

- Sur chaque rangée vous allez travailler ensemble pour faire les 2 exercices suivants et trouver/tester des solutions.
- Vous avez le droit de demander de l'aide aux autres rangées
- Nous ferons bien évidemment une correction

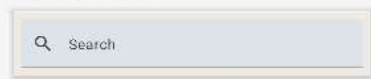
Mises en page de base dans Compose

- Exercice : recréez l'IHM ci-contre à partir du projet vide
 - <https://gitlab.com/m1-ue-tac/compose/exemple-belle-mise-en-page> (version de départ)
- Comme nous n'avons pas encore vu les barres de navigation (en bas), vous pouvez ignorer celle-ci si vous voulez.

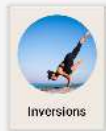


Les Preview à réaliser

SearchBarPreview



AlignYourBodyEle.



FavoriteCollectionCa..



FavoriteCollectionsGridPreview



FavoriteCollectionsGridPreview



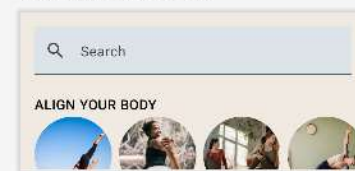
AlignYourBodyRowPreview



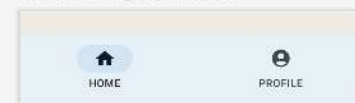
HomeSectionPreview



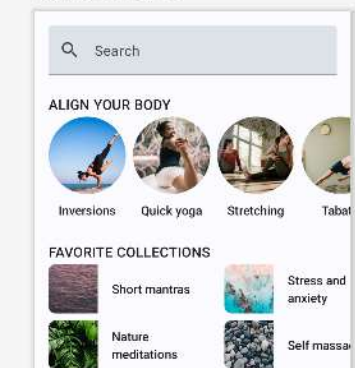
ScreenContentPreview



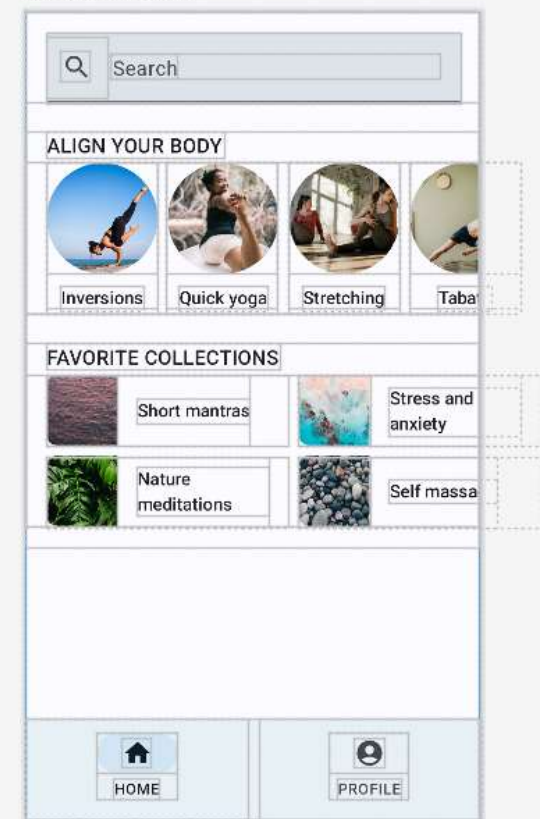
BottomNavigationPreview



MySoothePreview



MySoothePreview



2nd exercice de groupe (tiré de Kodeco : liste recettes)

Source : <https://www.kodeco.com/7032631-jetpack-compose-tutorial-for-android-getting-started>

Projet initial : <https://gitlab.com/m1-ue-tac/compose/intro-compose-kodeko-initial.git>

Exercice : refaire ce projet depuis projet initial (= projet vide + ressources fournies)

juste l'IHM, pas de comportement

